VITOR HUGO DE OLIVEIRA SILVA

MULTI-TASK LEARNING FOR POINT-OF-INTEREST CLASSIFICATION AND PREDICTION TASKS: THE ROLE OF THE CHECK-IN-LEVEL REPRESENTATION

Dissertation submitted to the Computer Science Graduate Program of the Universidade Federal de Viçosa in partial fulfillment of the requirements for the degree of *Magister Scientiae*.

Orientador: Fabrício Aguiar Silva

FLORESTAL - MINAS GERAIS
2026

Resumo

SILVA, Vitor Hugo De Oliveira, M.Sc., Universidade Federal de Viçosa, agosto de 2026. **Multi-Task Learning for Point-of-Interest Classification and Prediction Tasks: The Role of the Check-in-Level Representation**. Orientador: Fabrício Aguiar Silva.

Redes sociais baseadas em localização registram *check-ins*: visitas de um usuário a um ponto de interesse (POI). Duas propriedades da próxima visita merecem ser antecipadas, sua categoria e sua região, mas não o ponto de interesse exato; as duas tarefas leem o mesmo histórico, portanto um único modelo poderia aprendê-las em conjunto por aprendizado multitarefa (MTL). Compartilhar parâmetros pode prejudicar uma das tarefas, e se o treinamento conjunto ajuda esse par, e do que isso depende, permanecia em aberto quando esta pesquisa começou. Três estudos respondem a essa pergunta. O primeiro construiu o MTLnet, um modelo conjunto com uma representação (*embedding*) em nível de POI e compartilhamento rígido de parâmetros; ele não superou consistentemente os modelos dedicados (tarefa única). O segundo substituiu apenas a entrada, por codificadores espaciais, temporais e categóricos decompostos; o desempenho de categoria subiu acentuadamente nos três estados testados, indicando que, naquela configuração, o gargalo era a representação, e não a arquitetura. O terceiro moveu a representação para o nível do check-in, dando a cada visita seu próprio vetor, e redesenhou a topologia de compartilhamento em torno de um tronco de atenção cruzada. Os seis conjuntos de dados são cinco estados dos Estados Unidos, do Gowalla, e Istambul, do Massive-STEPS. Sob validação cruzada com usuários disjuntos entre treino e teste, com vinte modelos ajustados por configuração, quatro inicializações aleatórias sobre cinco partições fixas, e testes pareados sobre as quatro médias por inicialização, esse modelo conjunto supera os modelos dedicados na previsão da próxima categoria nos seis, por 5,3 a 9,4 pontos de macro-F1 sob uma seleção *joint-best*. Na previsão da próxima região, supera em quatro deles e equipara-se estatisticamente, com não-inferioridade dentro de uma margem de dois pontos de Acc@10 (TOST), nos outros dois. A resposta é, portanto, condicional: se o aprendizado multitarefa ajuda depende da representação de entrada e da topologia de compartilhamento construída sobre ela.

Palavras-chave:
aprendizado multitarefa
ponto de interesse
previsão da próxima categoria
previsão da próxima região
representação em nível de check-in

Abstract

SILVA, Vitor Hugo De Oliveira, M.Sc., Universidade Federal de Viçosa, August 2026. **Multi-Task Learning for Point-of-Interest Classification and Prediction Tasks: The Role of the Check-in-Level Representation**. Advisor: Fabrício Aguiar Silva.

Location-based social networks record *check-ins*: visits by a user to a point of interest (POI). Two properties of the next visit are worth anticipating, its category and its region, though not the exact next place; both tasks read the same trace, so one model could learn them together through multi-task learning (MTL). Sharing parameters can hurt one of the tasks, so whether joint training helps this pair, and what that depends on, was unresolved when this research started. Three studies answer that question. The first built MTLnet, a joint model with a place-level embedding and hard parameter sharing; it did not consistently outperform its dedicated single-task counterparts. The second replaced only the input, with decomposed spatial, temporal, and categorical encoders; category performance rose sharply in the three states tested, indicating that in that configuration the bottleneck was the representation, not the architecture. The third moved the representation to the check-in level, giving each visit its own vector, and redesigned the sharing topology around a cross-attention trunk. The six datasets are five states of the United States from Gowalla and Istanbul from Massive-STEPS. Under user-disjoint cross-validation, with twenty fitted models per configuration, four random initializations over five fixed folds, and paired tests on the four initialization means, that joint model outperforms the dedicated models on next category at all six, by 5.3 to 9.4 macro-F1 points under a joint-best selection. On next region it outperforms at four of them and statistically matches, with non-inferiority within a two-point Acc@10 margin (TOST), at the other two. The answer is therefore conditional: whether multi-task learning helps depends on the input representation and the sharing topology built on it.

Keywords:
multi-task learning
point of interest
next-category prediction
next-region prediction
check-in-level representation

List of Figures

List of Tables

List of abbreviations and acronyms

CBIC	Congresso Brasileiro de Inteligência Computacional
Check2HGI	Check-in-level representation extending Hierarchical Graph Infomax
CoUrb	Workshop de Computação Urbana
DGI	Deep Graph Infomax
FiLM	Feature-wise Linear Modulation
HGI	Hierarchical Graph Infomax
LBSN	Location-Based Social Network
MobiWac	ACM International Symposium on Mobility Management and Wireless Access
MTL	Multi-Task Learning
POI	Point of Interest
SBRC	Simpósio Brasileiro de Redes de Computadores e Sistemas Distribuídos
TOST	Two One-Sided Tests

Contents

1 Introduction

1.1 Context and motivation

Location-based social networks let a person announce where they are: a *check-in* records that a given user visited a given place, a point of interest (POI), at a given time. Such records now exist at large scale, and they carry more regularity than their noise suggests; an analysis of large-scale human traces estimated the potential predictability of an individual's next location at about 93 percent [2]. A system that anticipates two properties of the next visit, *what kind of place* it will be and *in which part of the city* it will happen, can act before the visit occurs: a recommender can rank candidates of the right type, a navigation or transit service can prepare for where the user is heading, and a platform can allocate attention and resources by area. These uses reach beyond mobility research itself. The study of human mobility informs urban planning, disease spreading, and pollution analysis, among other societal concerns [3], and place categories are the semantic characterization that location-based services rely on [4]. The field also imports its tools: part of the representation machinery used in this research, the spatial location encoders of the second study, was first validated on geospatial tasks such as species recognition and remote sensing classification [5, 6].

The two properties above are the two prediction tasks of this dissertation. The *next category* task predicts the category of the next visited place, one of seven top-level classes. The *next region* task predicts the official geographic unit of the next visit, a census tract in the United States or a mahalle in Istanbul, a target space that ranges from hundreds to several thousand classes per dataset. The exact *next place* task, predicting the specific establishment, is a third and different problem; this dissertation does not address it, and Chapter 2 keeps the three tasks formally distinct.

A fourth task also appears in this dissertation: the first two studies paired next category prediction with the static classification of a place's category, category classification, and Section 1.2 explains why the final study replaced it. Next category and next region were chosen for what a mobility-aware service can act on, and both are established end targets in the literature on the way to the harder next-place problem.

Serving both predictions raises a natural engineering wish. The two tasks read the same trace of visits, so one model could learn them together, sharing what they have in common; this is multi-task learning (MTL) [7], and it has produced single networks that handle many vision tasks at once [8], recurrent models that diagnose over a hundred clinical conditions simultaneously [9], and language models fine-tuned across dozens of tasks described by instructions [10]. The operational appeal is concrete: a single model to maintain, and a single forward pass that returns both predictions together, instead of two dedicated single-task models running side by side. The promise, however, is not automatic. Sharing parameters between tasks can hurt one of them, a failure mode known as negative transfer. Whether joint training helps for this pair of POI prediction tasks, and what the answer depends on, was unresolved when this research started.

1.2 Research question and the arc of this dissertation

This dissertation answers one question:

> **Does multi-task learning help point-of-interest prediction (next category and next region), and what does the answer depend on?**

The answer was not found in one step, and the journey is the contribution. This dissertation is a collection of three studies, and it presents them in the order they happened: a negative result, its diagnosis, and its resolution.

The first study (Chapter 3, published at CBIC 2025) built the first joint model of this research, MTLnet, from standard components: a place-level graph embedding as input and hard parameter sharing beneath the two task heads. It predicted place categories in two forms, the static category classification task and the sequential prediction of the next category. The joint model did not consistently outperform the two dedicated single-task models, and it cost more to train. The paper reported this null result as a finding, together with three candidate explanations, one of which pointed at the input representation.

The second study (Chapter 4, published at CoUrb 2026) tested the representation explanation first, as the cheapest controlled test among the three: it kept the MTLnet architecture fixed and replaced only the input, a monolithic 64-dimensional place embedding, with decomposed spatial, temporal, and categorical encoders. Category performance rose sharply at every state tested. The diagnosis followed: at that stage of the research, the input representation, not the sharing architecture, was the bottleneck.

The third study (Chapter 5, submitted to MobiWac 2026 and under review) acted on that diagnosis. Any place-level embedding assigns a place the same vector on every visit; a representation that cannot tell a weekday lunch from a Saturday night out at the same place is working against both tasks at once. That observation is the hypothesis the final study tests. It moves the representation to the check-in level: one vector per visit. Acting on another of the first study's candidate explanations, the restrictiveness of hard parameter sharing, it also redesigns the sharing topology, replacing the shared hidden layers with cross-attention between two task-specific streams. The change of representation also refines what a joint model should predict, since under a per-visit representation the static classification of places becomes a less natural fit than the sequential tasks, and the task pair settles on next category and next region. With both changes in place, one joint model finally outperforms both dedicated single-task models across the six datasets studied, five states of the United States and Istanbul: on the category task at all six, and on the region task at four of six, with statistical non-inferiority within a two-point margin (TOST) at the other two.

The task pair therefore evolved across the three studies, from static category classification plus next category in the first two to next category plus next region in the last, and this dissertation names that evolution plainly rather than narrating one fixed experiment. The arc it reports is a correction trail: each study revised what the previous one concluded, and the later chapters state precisely which earlier conclusions they supersede.

1.3 Objectives

The general objective of this dissertation is to determine whether multi-task learning helps point-of-interest prediction for the next category and next region tasks, and to identify what the answer depends on. Four specific objectives structure the work, one per chapter of the collection and its consolidation:

1. Investigate whether a standard hard-parameter-sharing joint model benefits its two category tasks, static category classification and next category prediction, relative to dedicated single-task models (Chapter 3).
2. Diagnose the role of the input representation in that result by decomposing and enriching the input while holding the architecture fixed (Chapter 4).
3. Design and validate a check-in-level representation and a joint model built on it for next category and next region prediction (Chapter 5).
4. Anchor the final answer to the research question in a user-disjoint statistical protocol, the cross-validation with paired significance and non-inferiority testing of Chapter 5, and consolidate the evidence of the three studies under it (Chapter 6).

1.4 Scope and assumptions

The empirical scope is fixed by the data and the targets. The experiments draw on observational check-in traces from two sources: the Gowalla dataset restricted to five states of the United States (Alabama, Arizona, Florida, California, Texas) and the Istanbul portion of the Massive-STEPS benchmark. The prediction targets are the next category, over the seven-class taxonomy defined in Chapter 2, and the next region, over census tracts in the United States and mahalles in Istanbul. The exact next place is not predicted anywhere in this work. The joint model operates under a single-model constraint: one trained artifact whose one forward pass over one set of inputs yields both predictions. Design-time assumptions of this kind are stated here and kept separate from the evaluation-time limitations discussed in Chapter 6.

1.5 Organization of this dissertation

In accordance with the Master's Regulations of the Federal University of Viçosa, this dissertation is organized as a collection of works, with each central chapter presented as an independent article. The chapters were re-typeset in a unified format for this dissertation; their content is faithful to the published text of each article, or, in the case of the article under review, to the submitted manuscript, and any correction applied afterward is listed in the errata appendix of the supplementary volume of this dissertation rather than silently edited.

- **Chapter 2, Fundamentals**, consolidates the background the three articles share: the POI prediction tasks, representations for mobility data, multi-task learning, and the datasets and evaluation protocol. It closes by connecting these fundamentals to the three questions the articles answer.
- **Chapter 3** presents the first article, published in English at the XVII Congresso Brasileiro de Inteligência Computacional (CBIC 2025, DOI 10.21528/CBIC2025-1191324), with this author as first author.
- **Chapter 4** presents the second article, published in Portuguese at the X Workshop de Computação Urbana (CoUrb 2026, DOI 10.5753/courb.2026.22960), held jointly with the Simpósio Brasileiro de Redes de Computadores e Sistemas Distribuídos (SBRC 2026), and reproduced here as an English translation. The article is first-authored by Tarik S. Paiva; this author is the second author, contributed the MTLnet baseline on which the study builds, and presented the paper at the event.
- **Chapter 5** presents the third article, submitted to the 23rd ACM International Symposium on Mobility Management and Wireless Access (MobiWac 2026) and currently under review, with this author as first author.
- **Chapter 6, Conclusion**, consolidates the answer to the research question, states the limitations, and derives future work from them.

Each article chapter opens with a short preface identifying its venue, its publication status, and the points on which later chapters revise its conclusions, so that the reader always knows which statements reflect the state of the research at the time of that article and which reflect the dissertation's final position.

1.6 Contributions

The contributions of this dissertation fall into four groups.

Theoretical. The finding that the input representation, together with the sharing topology built on it, dominates whether multi-task learning helps these POI prediction tasks. The arc also yields a corrected view of MTL for this task family: a null result under a place-level representation coexists with a positive result under a check-in-level one.

Software. The MTLnet framework (Chapter 3), the check-in-level representation and the joint model built on it (Chapter 5), and the reproducible training and evaluation pipelines behind the experiments of Chapters 3 and 5, with code repositories referenced in the respective chapters.

Empirical. A six-dataset benchmark of joint versus dedicated models for next category and next region prediction under user-disjoint cross-validation, with twenty fitted models per configuration (four seeds, each a complete repetition of the five-fold experiment with a different random initialization, over one fixed set of five folds), paired significance tests on the four per-seed means, that is, four paired observations, non-inferiority margins, and a leakage audit. All four seeds reuse the same fold partition, so the reported intervals do not cover uncertainty over resampled user splits.

Practical. Evidence that one deployable model can serve both prediction services at once, outperforming the dedicated models on the category task and outperforming or remaining non-inferior to them on the region task, with the operational simplicity of a single artifact and a single forward pass.

2 Fundamentals

This chapter sets out the concepts the three studies in this dissertation share, so that three papers of different shape read as one document. It defines the prediction tasks and keeps them distinct (Section 2.1); follows the line of representations for mobility from one-hot identifiers to the check-in level (Section 2.2); reviews multi-task learning and the conditions under which it helps or hurts (Section 2.3); states the datasets and metrics, and the validation protocol of the final study (Section 2.4); and closes by drawing these together into the question the following chapters answer (Section 2.5).

2.1 Point-of-interest prediction tasks

Location-based social networks (LBSNs) record where people go. Each record is a *check-in*: a user, a point of interest (POI), and a timestamp. These traces carry geographic and temporal detail at a resolution earlier mobility data lacked, and they have turned the study of cities into a data-driven discipline [11]. The behavior they capture is far from random. People return to a small set of places on a daily rhythm and travel farther only occasionally [12], and the regularity has a measurable limit: an entropy analysis of movement traces places a potential predictability of about 93 percent on where an individual goes next [2]. This bound is specific to next-location prediction at coarse spatial resolution; it shows that mobility is far from random and is learnable at all, and Section 2.4 states the reference points that actually bound the category and region tasks studied here.

Deep-learning methods for human mobility address several distinct problems, from crowd-flow estimation to next-location prediction [3], so the target of a model must be stated precisely before results can be compared. This dissertation separates three prediction targets and keeps them apart throughout. *Next-place prediction* asks for the exact next POI a user will visit. *Next-category prediction* asks only for the semantic class of that next POI, one of seven top-level categories (Community, Entertainment, Food, Nightlife, Outdoors, Shopping, Travel). *Next-region prediction* asks for the map partition the next POI falls in, a census tract in the United States datasets and a *mahalle* in Istanbul. A fourth, non-sequential task, *category classification*, labels a POI with its semantic category from static features rather than from a sequence [4]. The distinction between these targets is not cosmetic. The number of candidate places runs to tens of thousands, whereas there are seven categories and, depending on the dataset, from a few hundred to several thousand regions, so the tasks differ in difficulty, in the baselines that apply, and in what a correct answer means. The work reported here predicts the next category and the next region. It does not predict the exact next place; that target is named only to hold it apart from the two the dissertation studies.

Most of the field has pursued next place. The sequential framing began with recurrent models: ST-RNN augments a recurrent network with time-specific and distance-specific transition matrices so that the interval and the displacement between visits shape the prediction [13]. DeepMove adds an attention mechanism over long and sparse histories, recovering the periodic structure of a user's movement [14]. HST-LSTM folds spatial-temporal transition context into a hierarchical recurrent cell [15], and Flashback attends back over past hidden states to cope with the gaps that sparse trajectories leave [16]. Attention then became the backbone rather than an add-on. STAN models spatio-temporal correlations between non-adjacent visits and shows that visits far apart in a sequence still carry signal [17]; GeoSAN encodes geography through a hierarchical grid and a self-attention layer [18]; and GETNext brings a global trajectory-flow map into a transformer so that collaborative signal from many users informs a single prediction [19]. This lineage is background for the present work, not its target: every model named in this paragraph predicts the exact next place.

Two observations from that line matter downstream. First, a place is not a fixed object across visits. CTLE learns context- and time-aware location embeddings in which the same POI takes different vectors under different circumstances, the closest prior work to a per-visit view of a place [20]. Section 2.2 returns to this point, because it is the hinge of the representation argument. Second, the region and the category have appeared in next-place systems, but as means rather than ends. HMT-GRN predicts place and region in a hierarchy and uses the region prediction to constrain a beam search over places, so the region is auxiliary to place [21]. Category-aware recommenders such as CatDM predict a category to prune the place candidate set [22]. Both treat the category or the region as an intermediate signal on the way to a place. A smaller body of work makes one of them the end target directly: activity-region prediction studies the region level in its own right [23], and POI-RGNN predicts the category of the next place as its output [24]. The dissertation takes this second stance for both targets at once, predicting next category and next region as co-equal ends, which is the setting the following sections build the tools for.

2.2 Representations for mobility

How a place is represented determines what a model can learn from it. This section follows one line of development, from symbolic identifiers to the check-in-level representation this dissertation adopts, because the central argument of the work is that the representation, more than the architecture, decides whether multi-task learning helps.

The starting point is the one-hot identifier, a sparse vector that marks a place or a category by position and encodes no relationship between them. Distributed representations replace it. The skip-gram model learns dense vectors in which proximity reflects co-occurrence, so that related symbols sit close together in a continuous space [25]. Carrying that idea to graphs, DeepWalk samples random walks and treats them as sentences, learning node embeddings from local neighborhoods [26], and node2vec generalizes the walk so that a single objective can favor either homophily or structural role [27]. Graph neural networks then made the aggregation explicit. The graph convolutional network combines a node's features with those of its neighbors through a localized spectral rule [28]; graph attention networks weight each neighbor by learned attention rather than by fixed degree [29]; and GraphSAGE learns aggregator functions that generalize to nodes unseen during training, making the embedding inductive [30].

These methods still need a training signal. For places, labels are scarce, so the representation is learned without them, by an information objective. Mutual information between two high-dimensional variables can be estimated and maximized by gradient descent [31], and Deep InfoMax turns this into a representation learner by maximizing the mutual information between a local feature and a global summary of the input [32]. Deep Graph Infomax (DGI) applies the principle to graphs: it learns node representations, unsupervised, by maximizing the mutual information between local patch representations and a global graph summary [33]. In this project DGI produces the place embeddings used as input to the first models. Hierarchical Graph Infomax (HGI) builds directly on it. HGI extends the infomax objective across a hierarchy and is trained by maximizing the mutual information among the POI, region, and city levels, yielding region-aware place embeddings [34]. HGI is the place-level baseline representation the later chapters measure against, and it is the direct base of the representation the dissertation contributes. Two qualifications belong with that use. Huang et al. present HGI as a method for urban region representation and evaluate it on region-level estimation tasks, so the work reported here repurposes its POI-level output as a place embedding for a sequential prediction task their paper does not address. The baseline was also tuned rather than taken as published: the cross-region edge weight of their Equation 2, set to 0.4 for the dense Chinese cities they study, was raised to 0.7 for the sparser United States state datasets used here. The sweep that fixed that value ran on Alabama over four settings of the weight, 0.4, 0.5, 0.6, and 0.7, each measured over five folds with a budget of 50 epochs, and the category F1 rose monotonically across them, from 0.7388 ± 0.0205 at the published setting to 0.8186 ± 0.0123 at the adopted one, on a zero-to-one scale, with the spread taken across the five folds.

A place embedding, however it is trained, shares one property: it assigns each place a single fixed vector. That vector is the same whether the place is visited on a weekday morning or a Saturday night, by a commuter or a tourist. Yet a place serves different functions in different contexts, and a representation that cannot express this cannot distinguish two visits to the same place that mean different things. CTLE makes the point concrete by learning location embeddings that are context- and time-aware, so that the vector for a place depends on the visit [20]. This is the limitation the rest of the dissertation responds to: a per-place vector is static across visits, and moving below the place level, to the check-in itself, is the way to make the representation reflect the visit.

Before that step, several general encoders supply the context a static place vector omits. They are worth naming at their own origin, because they were developed outside mobility and are imported as components. Time2Vec is a model-agnostic encoding of time that combines a linear term with periodic terms, so that hour-of-day and day-of-week enter a model as continuous features [35]. SIREN shows that networks with sinusoidal activations represent continuous signals and their derivatives well, which is the general basis for periodic coordinate encoders [36]. Space2Vec encodes absolute position and spatial context at multiple scales [37], and Sphere2Vec encodes points on a sphere so that the encoding respects spherical geometry rather than a flat approximation [5]. A related encoder uses spherical harmonics together with sinusoidal representation networks, again to honor the spherical domain that flat sine-and-cosine features distort [38]. Feature-wise linear modulation (FiLM) is a general conditioning mechanism that scales and shifts a layer's features according to a side input [39]. In the models of Chapters 3 and 4 it conditions the shared network on a per-task identity, so that one trunk can serve both tasks. The context these encoders carry enters the model separately, and how it enters is the turning point of this dissertation. Chapter 3 feeds a single place-level embedding and adds only the per-task conditioning. Chapter 4 keeps that model but decomposes the input, replacing the single embedding with separate spatial, temporal, and categorical components combined at the input, and it is this change to the representation, rather than any change to the architecture, that moves the results.

The check-in-level representation, Check2HGI, completes the line. It extends the graph-infomax hierarchy with a fourth level below the place, the check-in, and is trained without task labels in the same infomax spirit, so that each visit, rather than each place, carries its own vector.

The training objective makes that extension concrete, and it is worth stating here because it is what the fourth level costs. The hierarchy has three boundaries between adjacent levels, check-in to place, place to region, and region to city, and the total objective is a fixed-weight sum of one term per boundary,

$$\mathcal{L} = 0.4\,\mathcal{L}_{c2p} + 0.3\,\mathcal{L}_{p2r} + 0.3\,\mathcal{L}_{r2c}, \tag{2.1}$$

where $\mathcal{L}_{c2p}$ is the check-in-to-place term, $\mathcal{L}_{p2r}$ the place-to-region term, and $\mathcal{L}_{r2c}$ the region-to-city term. Each term is built from a bilinear discriminator that scores how compatible two embeddings are,

$$\mathcal{D}(\mathbf{e}_1, \mathbf{e}_2) = \sigma(\mathbf{e}_1^\top \mathbf{W} \mathbf{e}_2), \tag{2.2}$$

in which $\mathbf{e}_1$ and $\mathbf{e}_2$ are the two embeddings being compared, $\mathbf{W}$ is a learned weight matrix, and σ is the logistic function, so the score lies between 0 and 1. A boundary's term then rewards a high score on a true pair and a low score on a false one,

$$\mathcal{L}_* = -\log \mathcal{D}(\mathbf{e}^+, \mathbf{e}^+) - \log\left(1 - \mathcal{D}(\mathbf{e}^+, \mathbf{e}^-)\right), \tag{2.3}$$

where $\mathbf{e}^+$ denotes an embedding from a true pair, a check-in with the place it actually occurred at, and $\mathbf{e}^-$ an embedding substituted from elsewhere in the batch to form a false pair. No target label appears in any of the three equations, which is the sense in which the representation is trained without task labels.

Where a place embedding answers "what is this place," a check-in-level representation answers "what is this visit," and Chapter 5 develops this representation and the joint model built on it. Table 1 traces the full progression, from DGI through HGI to the check-in level and the joint model that uses it.

The last step of that chain deserves one clarification, because the joint model is a descendant of MTLnet and not a replacement written from nothing. In the released implementation the joint model is a specialization of the MTLnet class: it inherits the per-task input encoders, the task heads, and the partition of parameters into shared and task-specific groups, and it overrides exactly one component, the shared middle. Where MTLnet conditions a stack of shared residual layers on task identity through FiLM, the joint model puts a stack of cross-attention blocks in that position, each block letting one task stream read the other's features while each stream keeps its own feed-forward weights. A second specialization then adds the private spatial path, which hands the region output the region sequence directly, before the two streams mix. So the two models differ in how the tasks share and in what the region output may read, and they agree everywhere else. The architectural claim of Chapter 5 is a change of sharing topology inside a lineage, which is why the negative result of Chapter 3 and the positive result of Chapter 5 can be read against each other at all.

Table 1 – Representation and model lineage threaded through this dissertation, from the place-level graph-infomax embeddings to the check-in-level representation and the joint model built on it. Status of the Check2HGI representation and the joint model is *submitted, under review*.

Model / method	What it added	Reference
DGI	Unsupervised place embeddings by maximizing mutual information between local patches and a global graph summary.	[33]
HGI	Extends graph infomax over the POI, region, and city hierarchy to yield region-aware place embeddings.	[34]
MTLnet	First joint model: place embedding, FiLM conditioning, and hard parameter sharing. Null result for that configuration.	[1]
ST-MTLNet	Keeps MTLnet, replaces the place-embedding input with decomposed spatial, temporal, and categorical encoders.	Chapter 4
Check2HGI	Check-in-level representation: graph infomax extended to the check-in, so each visit carries its own vector.	Chapter 5
Joint model	Cross-attention model on Check2HGI: one model for both tasks, next category and next region.	Chapter 5

2.3 Multi-task learning

Multi-task learning (MTL) trains one model on several related tasks at once, in the expectation that a representation shared among them generalizes better than one learned for a single task in isolation [7]. The appeal for the present work is direct: predicting the next category and the next region from the same check-in history may let each task inform the other. Whether it does is the question the dissertation opens with, and the answer turns out to depend on choices this section lays out.

Deep MTL is organized around how much the tasks share. The two poles are hard parameter sharing, where the tasks use a common trunk and split only at the output heads, and soft parameter sharing, where each task keeps its own parameters and the tasks are coupled by a weaker constraint [40]. Between the poles lie architectures that learn what to share. Cross-stitch units place a learned linear combination of two task networks' activations at each layer, letting the model decide how much to mix them [41]. The multi-gate mixture-of-experts shares a pool of expert subnetworks but gives each task its own gate over them, so that routing, not a fixed trunk, determines sharing [42]. Progressive layered extraction sharpens this by separating shared experts from task-specific ones in layered gates [43], and DSelect-k makes the expert selection itself differentiable and sparse [44]. The principle these architectures share, keeping shared and task-specific components side by side rather than forcing one common trunk, is the one the joint model of Chapter 5 adopts, though it realizes it with cross-attention rather than expert gating.

Sharing is not free. When tasks pull the shared parameters in different directions, joint training can leave a task worse off than its single-task model, a failure known as negative transfer. Which tasks help each other and which interfere cannot be assumed away, and joint training can hurt as easily as it helps depending on the pairing [45]. The effect is not incidental: because the tasks can conflict, minimizing a weighted sum of their losses is only valid when they do not, and multi-task learning is more honestly cast as a multi-objective problem with no single optimum [46].

A family of methods tries to manage the conflict at the level of the gradients or the losses. Uncertainty weighting sets each task's loss weight from its homoscedastic uncertainty, learned jointly with the model [47]. GradNorm rescales per-task gradient magnitudes so that tasks train at comparable rates [48], and dynamic weight averaging sets the weights from the recent rate of change of each loss [49]. Other methods act on the gradient directions: PCGrad projects a task's gradient off any conflicting task's gradient [50], CAGrad maximizes the worst per-task improvement among updates that stay within a ball around the average gradient, with convergence guarantees [51], Nash-MTL treats the combination of task gradients as a bargaining game and takes the Nash solution as the joint direction [52], and Aligned-MTL aligns the principal components of the gradient system and uses its condition number as a stability criterion [53]. FAMO reduces the cost of balancing by tracking loss decreases in constant time and space rather than reading every task gradient at each step [54]. A recurring result tempers the family. Random loss weighting is a competitive baseline [55]; a controlled study finds that current MTL optimizers often do not outperform a well-tuned fixed-weight baseline [56]; and a direct defense of plain loss summation shows that a unitary scalarization with standard regularization matches or improves upon the specialized optimizers [57]. A recent survey and a dense-prediction survey both consolidate the architectures and the balancing methods and set out when sharing helps [58, 59]. The dissertation takes the cautious position these results support: a fixed-weight baseline is a serious competitor, and a balancer earns its place only by outperforming it.

In mobility, MTL has been used almost entirely in the service of next place. MCARNN predicts activity and next place together with a context-aware recurrent unit, pairing the two in parallel [60]. A cascade line runs the other way: CSLSL imposes a causal chain, time then activity then location, so that the activity or category is an intermediate step toward the place [61]. Across these systems the region and the category appear as auxiliary signals or as stages on the path to a place; no multi-task model among them predicts the next region as a co-equal end target alongside the next category. Single-task work on region prediction exists, as Section 2.1 notes, but the joint setting is open. That absence is the opening the dissertation addresses. Its own starting model, MTLnet, applies hard parameter sharing on a place-level embedding and does not outperform the dedicated single-task models, a result that holds for that configuration and motivates the representation argument the next chapters develop [1].

2.4 Datasets and evaluation

A claim about mobility prediction is only as trustworthy as the data it is measured on and the protocol that measures it. This section fixes both: the datasets the dissertation uses, the metrics and reference points each result is read against, the validation protocol that keeps the estimate honest, and the tests that license the verbs used to report a comparison.

Two check-in datasets serve as the ground. Gowalla is the dataset of record, introduced with the study of periodic and socially driven movement in LBSNs, and this work uses five United States states from it [12]. For a non-United-States setting, the Massive-STEPS benchmark supplies Istanbul check-ins and argues that the field has leaned too long on decade-old datasets, which is part of the reason for evaluating across more than one city [62]. The Foursquare check-in dataset from New York and Tokyo is a further widely used benchmark [63]; it is named here for context, as the dissertation evaluates on Gowalla and the Istanbul data rather than on it.

The category label distribution is imbalanced. The Food class alone accounts for roughly a third of the check-ins in a representative state, so plain accuracy is inflated by the majority class and must not be the headline for next category. Macro-averaged F1 is the primary category metric instead: it is the unweighted mean of the per-class F1 scores, so it reads as average per-class quality with every class counting equally, and it does not reward a model that serves only the frequent classes [64]. That choice of metric, rather than a reweighted loss, is how the imbalance is answered here. The joint model of Chapter 5 trains on plain unweighted cross-entropy; class weighting, tested there on both outputs, lowered both region accuracy and category macro-F1. For next region, the primary metric is accuracy at 10 (Acc@10), the fraction of cases where the true region appears in the model's ten highest-ranked predictions; it is a ranking metric and says nothing about the probability mass placed on the true region, so a model can rank the true region tenth as cheaply as first. In the reported region results a region never seen in training counts as a miss, so the figure is not inflated by unpredictable new regions; Chapter 5 gives the full definition. The two tasks are scored separately throughout this dissertation, each against its own reference points, so the comparison a joint model has to win is made per task, against the dedicated single-task model for that task. Where the two scores are combined at all, it is to select a training checkpoint rather than to report a result: Chapter 5 reads its joint model at the epoch of the best joint validation score, the geometric mean of the two task metrics.

Every metric is read against reference points. The majority-class floor, which always predicts the most frequent category, is the level a learned category model must clear. A first-order transition model, a mobility Markov chain over the training partition, is the corresponding non-learned floor for the sequential targets [65]. The entropy analysis of human movement that reports a potential predictability of about 93 percent sets a bound for predicting the next location at coarse resolution, and it frames why mobility is learnable at all [2]; it is not, however, a ceiling on seven-class category macro-F1 or on region ranking, which are different label spaces. The operative ceiling for the two tasks studied here is the dedicated single-task model, since the question throughout is whether one joint model can match or outperform the models built for each task alone.

The validation protocol guards against the most damaging error in this setting, a user whose check-ins appear in both training and test. Estimates use stratified k-fold cross-validation [66], and the folds are formed so that no user spans a split: a grouped, stratified splitter keeps all of a user's check-ins on one side of every fold, so that measured accuracy reflects generalization to new users rather than memorization of familiar ones [67]. That protocol is the one Chapter 5 uses, and it strengthened from one study to the next: Chapters 3 and 4 both stratify by sample rather than by user, so that the check-ins of one user may appear in both training and validation, and only Chapter 5 splits by user. The statistical treatment is scoped the same way: Chapters 3 and 4 report fold means and standard deviations and run no significance test, so the tests set out below license verbs in Chapter 5 alone. Comparisons there are made across the folds and repetitions with paired tests. The paired Wilcoxon signed-rank test compares two models across the paired results without assuming normality [68]. Its exact one-sided p has a floor set by the number of pairs, so a superiority claim tested on four repetitions is reported with a paired t on the per-repetition means and the Wilcoxon test alongside it, on the individual folds (Chapter 5); either test licenses the verb "outperforms", and the Holm step-down procedure controls the family-wise error when several datasets are tested at once [69]. A superiority test cannot, by itself, support a claim of equivalence, so where the dissertation states that one model matches another it uses two one-sided tests to establish statistical non-inferiority within a two-point margin [70]. Wherever this dissertation reports a test, the verb and the test are bound together: “outperforms” follows only from a paired superiority test, and “matches” only from a non-inferiority test within the stated margin.

2.5 Relevance

The preceding sections are not a catalog; they are the parts of a single argument this dissertation makes, and this section draws them together before the paper chapters put them to work. No new literature enters here. The point is to show how the tasks, the representations, the multi-task machinery, and the evaluation protocol combine into one problem.

The three prediction targets are different problems, and the field has mostly studied the hardest of them, the exact next place. Predicting the next category and the next region is not a lesser version of that task but a different one, with its own baselines and its own notion of a correct answer, and treating both as co-equal end targets is the stance this work takes. That stance only pays off if the model can represent a visit well enough to serve both. Here the representation is the lever. The field moved from one-hot identifiers to place embeddings, and hierarchical graph infomax gives a strong place-level vector, but any place embedding assigns a place the same vector on every visit. A vector that stays the same across visits carries nothing about the visit being predicted, and the check-in level is the response to that limit.

Multi-task learning is the mechanism that would let the two tasks share what they have in common, but it is not a free gain. Naive hard sharing can leave a task worse than its single-task model, and the elaborate gradient balancers proposed to prevent this frequently do not outperform a well-tuned fixed-weight baseline. Whether joint training helps therefore cannot be assumed; it has to be measured, against the dedicated single-task models, under a protocol that does not flatter it. That protocol is the last piece. User-disjoint cross-validation, macro-F1 and Acc@10 read against named floors and the single-task ceiling, and verbs bound to paired tests and to non-inferiority testing are what separate a real improvement from a hopeful one. The value of the fundamentals in this chapter is that they make the central question answerable on honest terms.

That question presses because its parts have not been brought together. The field has a place-level representation but not a check-in-level one built for these targets; it has multi-task learning but almost no treatment of the next region as an end target; and it has strong evaluation practice that mobility studies do not always apply. The dissertation takes up the three in order. It first asks whether naive multi-task learning helps at all, given a place-level embedding and hard parameter sharing, and reports an honest answer for that configuration. It then asks whether the representation, rather than the architecture, is the lever, by decomposing and enriching the input the same model receives. It finally asks what a representation built for check-ins enables in a redesigned joint model, one that, by paired superiority tests, outperforms the dedicated single-task models on the next category everywhere it is tested and on the next region at four of six datasets, and matches the dedicated single-task model within a two-point margin, by non-inferiority testing, at the other two. Chapter 5 reports these tests in full. The chapters that follow answer these three questions in turn.

3 Multi-Task Learning for POI Category and Next-POI Prediction

This chapter reproduces the article "An Investigation into Multi-Task Learning for Point-of-Interest Category Classification and Next-POI Prediction", published at the Congresso Brasileiro de Inteligência Computacional (CBIC 2025), DOI 10.21528/CBIC2025-1191324, with Vitor H. O. Silva as first author. The text was reformatted from the two-column conference layout to the dissertation format; typographical and tabulation errata were corrected, and each correction is listed in Appendix B of the supplementary volume of this dissertation. Its conclusions are the conclusions of the time, for the configuration studied here: with a place-level embedding and hard parameter sharing, multi-task learning did not consistently improve on the dedicated single-task models. The two chapters that follow revise that verdict, and they change different things to do it. Chapter 4 keeps this architecture and replaces the input representation, which is what identifies the representation as the binding constraint. Chapter 5 then changes the architecture as well, moving from hard parameter sharing to a cross-attention sharing topology, and changes the task pair with it. The chapter's preference for the Nash-MTL optimizer is likewise a conclusion of the time, weakened by a later finding about the optimizer implementation, and Chapter 5 does not rely on it. A note on terminology: the term "Next-POI Prediction" as used in the reproduced article denotes the frame's next category *task, that is, predicting the category of the next visited place, not the exact place itself; the dissertation reserves* next place *for the exact-POI task, which is not studied here.*

3.1 Introduction

Multi-Task Learning (MTL) is a machine learning paradigm where multiple related tasks are learned jointly, sharing representations and inductive biases to improve generalization performance [7]. By using shared information, MTL can potentially mitigate overfitting and yield more robust feature extractors compared to single-task approaches. Its success in domains like computer vision [8], natural language processing [10] and recommendation systems [71] motivates its application to Location-Based Social Networks (LBSNs), where understanding user interactions with Points-of-Interest (POIs), specific physical locations that someone might find useful or interesting, such as restaurants, shops, or landmarks, is fundamental.

In this chapter, we investigate the application of MTL to two critical POI-related tasks:

1. **POI Category Classification**: Classify the semantic category (e.g., shopping, travel) of a POI based on its features and context within a user's trajectory.

2. **Next-POI Prediction**: Predicting the category of the next POI a user will visit, given the sequence of their prior visits.

These tasks are ostensibly related, as both draw upon the same underlying user mobility data. Accurate POI category representations could inform next-location predictions, and conversely, sequential patterns could provide context for category classification. However, the tasks possess fundamentally different natures. POI Category Classification is primarily a **static classification task** that relies on the intrinsic, context-rich features of individual locations. In contrast, Next-POI Prediction is a **dynamic, sequential task** that depends on capturing temporal patterns and transitions.

This dichotomy raises a critical question: can a single, shared representation effectively serve two tasks with such distinct underlying characteristics? The assumption that MTL will be beneficial is not guaranteed. It is possible that forcing a shared encoder to learn features for both a static and a sequential task could result in **negative transfer**, where the joint training process actually hinders performance by learning a "compromise" representation that is not specialized enough for either task.

Therefore, the central hypothesis of this study is that a standard hard parameter-sharing MTL architecture will face significant limitations when applied to this specific task pairing, failing to achieve substantial and consistent improvements over well-tuned, specialized single-task models. Our work is thus framed as an empirical investigation into the boundary conditions of MTL's effectiveness, exploring whether the presumed task relatedness is sufficient to overcome their inherent structural differences.

To test this hypothesis, we conducted experiments on the Gowalla LBSN dataset [12, 72], comparing our MTL model against strong single-task baselines (HMRM [73] and MHA+PE [74]). Our results lend weight to our hypothesis. While the proposed models perform well, particularly in POI category classification, the MTL framework does not deliver the consistent gains often expected. The performance differences between the MTL and single-task models were frequently marginal and fell within standard deviations, suggesting their statistical performance was largely comparable. This outcome indicates that, for this problem, the architectural constraints and task dissimilarities may have offset the potential benefits of joint learning.

The main contributions of this chapter are as follows:

- We introduce an MTL framework to simultaneously address POI category classification and next-POI prediction.[1]

- We design task-specific heads to capture category co-occurrence and sequential transition dynamics, respectively.
- We provide a critical analysis of the limitations of a hard parameter-sharing MTL approach for this task pairing. Our findings highlight that significant task dissimilarity can challenge the effectiveness of MTL, serving as a case study on the importance of task compatibility in model design.

The rest of this chapter is organized as follows. In Section 3.2, we review the theoretical foundations of MTL and prior work on POI modeling. Section 3.3 describes our joint architecture, data preprocessing, and training protocol. Section 3.4 presents experimental results. Finally, Section 3.5 summarizes our findings and discusses future research directions.

3.2 Theoretical Foundations and Related Work

3.2.1 POI Classification and Next-POI Prediction

POI Category Classification refers to the task of inferring the semantic category of a location based on contextual information such as geographic coordinates, visit frequency, or user behavior.

Next-POI Prediction, in contrast, aims to predict which specific location a user is likely to visit next, given their past movement history.[2] It is a sequential task that requires modeling temporal and spatial patterns within user trajectories.

The prediction and classification of POIs are fundamental challenges in applications such as recommendation systems and urban planning. Xu et al. [4] propose the Tree-guided Multi-task Embedding (TME) model, which aims to improve the semantic annotation of POIs using a hierarchical structure of categories and mobility-based representation learning.

POI Category Classification has been extensively addressed in location-based recommendation systems, with various techniques applied to capture patterns and improve prediction accuracy. In this context, Lim et al. [21] propose the Hierarchical Multi-Task Graph Recurrent Network (HMT-GRN), which uses multi-task learning to simultaneously predict the next POI and its geographic region.

1 The complete source code is publicly available at <https://github.com/VitorHugoOli/PoiMtlNet> for full reproducibility.

2 This states the general formulation of the task as the literature poses it. The variant studied in this chapter predicts the *category* of the next visited place rather than the place itself, as defined in Section 3.1 and in the chapter preface.

Although these approaches have made significant progress in the POI classification and next-location prediction fields, they tend to focus only on single-task objectives. None of the cited models propose a unified multi-task learning (MTL) framework that jointly optimizes both POI category classification and next-POI prediction tasks. This motivates the development of solutions that use shared representations across related POI tasks.

3.2.2 Multi-Task Learning

MTL was formalized by Caruana [7] as an inductive transfer mechanism that enhances generalization by learning shared representations across related tasks. Early neural architectures employed *hard parameter sharing*, where a common set of hidden layers is used for all tasks, yielding strong regularization and reduced training time.

Recent surveys [59, 75] organize contemporary MTL research along five methodological dimensions: (i) *parameter sharing* (hard vs. soft); (ii) *relationship learning* (discovering task affinity or hierarchy); (iii) *feature routing* (e.g., cross-stitch, sluice networks, attention gating); (iv) *optimization* (conflict-aware gradient techniques); and (v) *pre-training and instruction tuning*. MTL has evolved from small, homogeneous task sets to dozens of heterogeneous objectives spanning computer vision, natural language processing, and recommender systems.

A fundamental architectural challenge in MTL is deciding *how much*, *where*, and *when* to share parameters across tasks. Three canonical sharing schemes have emerged:

Hard Parameter Sharing

A single encoder is shared by all tasks, followed by task-specific heads. This remains the simplest and most popular baseline, providing effective regularization [7].

Soft Parameter Sharing

Each task has its own encoder, but networks exchange information through learned cross-connections, such as Cross-Stitch units [41] or Sluice networks [76].

Mixture-of-Experts (MoE)

Multi-Gate Mixture-of-Experts (MMoE) architectures maintain a pool of shared expert subnetworks; each task learns a gating function to combine experts, outperforming monolithic sharing in large-scale recommendation settings [42].

To mitigate task interference and loss imbalance, several optimization techniques have been proposed: GradNorm equalizes gradient magnitudes across tasks [48]; MGDA finds Pareto-optimal descent directions [46]; PCGrad removes conflicting gradient components [50]; and

Dynamic Weight Averaging (DWA) reweights losses based on their relative learning speeds [49].

Despite these successes, MTL still faces several key challenges:

- **Negative Transfer**: Unrelated or adversarial tasks can degrade shared representations, harming individual task performance [76, 75].
- **Gradient Conflict**: Simultaneous optimization may produce opposing gradient directions, slowing or destabilizing convergence [50, 46].
- **Data Heterogeneity**: Variations in modality, label granularity, and dataset size complicate sampling strategies and minibatch construction [52, 45].
- **Scalability**: Routing complexity, memory footprint, and evaluation costs often grow super-linearly as the number of tasks increases [75, 59].

Recent work addresses these issues via task clustering [45], dynamic curricula, conflict-aware optimizers, and parameter-efficient adapters [59]. Nevertheless, principled criteria for task grouping, theoretical guarantees of Pareto efficiency, and energy-efficient training of very large MTL foundation models remain open research directions.

3.2.3 Multi-task learning applied in POI

MTL has been explored in problems related to Points of Interest (POIs), primarily due to its ability to improve generalization by sharing information across correlated tasks. In POI modeling scenarios, such as user trajectory prediction and recommendation, MTL allows the learning of complementary objectives simultaneously, allowing for more accuracy.

Early MTL-based approaches such as MCARNN [60] employ recurrent neural networks with temporal attention mechanisms to jointly predict user activities and future visited locations. Similarly, the iMTL framework [71] uses an LSTM architecture to model next-activity prediction, incorporating temporal dynamics in user behavior modeling.

More recent works have begun incorporating contextual and semantic signals. TLR-M [77], for example, applies Transformer-based architectures to simultaneously predict the next POI and queue waiting time, demonstrating the value of attention-based models in capturing spatio-temporal dependencies. MTPR [78] combines LSTMs and adversarial learning to address uncertainty in check-ins and improve multi-task POI recommendation both location and temporal context with a generative component.

Despite these advances, existing MTL approaches often focus on joint modeling of next-POI prediction and temporal or behavioral signals, while the relationship between POI category classification and next-POI prediction remains underexplored. Some Models such as TME [4] address category annotation using graph-based encoders, but treat prediction and classification separately.

This creates a gap in the literature regarding the joint modeling of semantic and sequential POI tasks. In contrast, our proposed model introduces a unified MTL framework that jointly performs POI category prediction and next-POI classification using a hard parameter-sharing architecture. By modeling both tasks simultaneously, our approach promotes knowledge transfer across shared spatio-temporal representations, enabling more robust predictions even in data-sparse environments.

3.3 Methodology

This section details the proposed methodology for Point-of-Interest (POI) category classification and next-POI prediction using a MTL approach. We first describe the generation of POI embeddings and the preparation of data for both tasks. Subsequently, we present the architectures of the single-task baselines, which are also used as task-specific heads within our MTL framework. Finally, we elaborate on the proposed MTL architecture, including the hard parameter-sharing strategy and the Nash-MTL optimizer employed for gradient aggregation.

3.3.1 POI Embeddings and Data Preparation

This chapter computes POI embeddings using graph attention convolution along with contrastive learning based on Deep Graph Infomax (DGI). These embeddings are the input for the MTL model discussed in Section 3.3.2.

3.3.1.1 POI Embeddings

A point of interest is defined as a tuple $\langle id, lat, long, cat\rangle$, where id serves as a unique identifier, lat and $long$ represent its coordinates, and cat denotes its category, such as restaurant or pharmacy. A place suffers from complementarity effect [79], implying that a location is affected by its neighbors (e.g. leisure zones are often near residential districts). Hence, we propose a graph-based contrastive method to capture this spatial locality behavior, generating a POI embedding to serve as input to our MTL model.

To capture the spatial locality and capture neighborhood information, we model the weighted graph $G(V, E)$ as a Delaunay Triangulation, where each vertex $v \in V$ represents a POI and edge $e_{ij} \in E$ defines the connection between POI p_i and a POI p_j. Besides that, following [80] the weight of an edge e_{ij} is defined as $w_{ij} = \log((1 + D^{1.5}/1 + d_{ij}^{1.5}))$, where D is the diagonal length of bounding box that encloses the coordinates of POIs, and d_{ij} is the geodesic distance between p_i and p_j. The node feature matrix is based on category one-hot encoding of each POI, represented by $X \in \mathbb{R}^{n_c}$, where n_c is the number of possible categories.[3]

Therefore, to encode the Delaunay graph we apply a graph attention convolution layer [29], yielding an embedding $h_i \in \mathbb{R}^d$, where d is the dimension of the output. In addition, we utilize a Deep Graph Infomax (DGI) [33] as a contrastive learning method to maximize the mutual information between local representations (node embeddings) and the global graph embedding compared to a corrupted version of a graph, which is guided by the Equation 3.1.

$$\mathcal{L} = \frac{1}{2|V|}\left(\sum_{i=1}^{|V|} \mathbb{E}_{(X,G)}[\log \mathcal{D}(\vec{h}_i, \vec{s})] + \sum_{j=1}^{|V|} \mathbb{E}_{(\tilde{X},G)}[\log(1 - \mathcal{D}(\tilde{\vec{h}}_j, \vec{s}))]\right) \tag{3.1}$$

where X and G denote the positive samples for the node feature matrix and graph adjacency, respectively, while $\tilde{X}$ is the corrupted version of the feature matrix. To corrupt the features, we employ a random shuffling method. Additionally, $\vec{s}$ is the global graph embedding derived by applying global mean pooling to the embedding matrix $H = [h_i, h_{i+1}, \ldots, h_n]$ in our model. In addition, $\mathcal{D}$ is a discriminator that differentiates each positive example from its corrupted version, learning from their dissimilarities; in our POI Embedding, this discriminator is implemented as a linear model.

3.3.1.2 Data Processing for POI Category Classification

Each POI is represented by its 64-dimensional DGI embedding $\mathbf{e} \in \mathbb{R}^{64}$. The dataset therefore consists of pairs $(\mathbf{e}, c)$, where c is the POI's ground-truth category to be predicted.

3.3.1.3 Data Processing for Next-POI Category Prediction

This task predicts the category of the next POI a user will visit from their recent check-in history.

1. **Trajectory building.** Check-ins (user, POI, timestamp) are ordered chronologically per user; users with fewer than five visits are discarded.
2. **Sequence extraction.** From each trajectory we draw non-overlapping windows of length $L_h = 9$: $(p_1, \ldots, p_9)$ as context and p_{target} as the next POI.
3. **Feature/label construction.**

[3] The released implementation feeds the network the mean of the one-hot vectors of a POI's graph neighbours, with the POI's own vector excluded, rather than the POI's own one-hot vector (`research/embeddings/dgi/preprocess.py`). Appendix B of the supplementary volume of this dissertation records the departure. The distinction matters for how the embedding should be read: the input describes a POI's neighbourhood, so the static task the embedding supports is spatial homophily rather than recall of the POI's own label.

- The input is the concatenation of the 64-dimensional embeddings of p_1–p_9, yielding a $9\times 64 = 576$-dim vector. Histories shorter than nine are padded by a special POI whose embedding is $\mathbf{0}$.
- The label is the category of p_{target}.

This format allows the model to infer the category of the next POI solely from the embeddings of the previous visits.

3.3.2 Multi-Task Learning Architecture

Our proposed Multi-Task Learning (MTL) architecture, depicted in Figure 1, is built upon a hard parameter-sharing scheme for the joint training of POI Category Classification and Next-POI Prediction. This design integrates Feature-wise Linear Modulation (FiLM) layers to condition the shared representations, a technique aimed at modulating task interactions and mitigating negative transfer [39, 45].

Architecture Overview

Input features for POI Category Classification ($\mathbf{x}^{(c)}$) and Next-POI Prediction ($\mathbf{x}^{(n)}$) are first processed by separate, task-specific encoders. These encoders, implemented as MLPs, map the raw input features into a shared latent space of dimension d_{shared}.

Feature-wise Linear Modulation (FiLM)

To allow for task-specific adaptation within the shared pathway, we employ FiLM layers [39]. A unique, learnable task embedding $\mathbf{e}_t$ for each task t is used to generate a scaling vector γ_t and a shifting vector β_t. These vectors modulate the encoded features $\mathbf{h}_{\text{enc}}^{(t)}$ from each task-specific encoder:

$$\text{FiLM}(\mathbf{h}_{\text{enc}}^{(t)} \mid \mathbf{e}_t) = \gamma_t \odot \mathbf{h}_{\text{enc}}^{(t)} + \beta_t$$

This lightweight operation conditions the features before they enter the fully shared layers of the network.

Shared Processing Layers

The FiLM-modulated representations are processed by a sequence of shared layers. This module begins with a linear transformation followed by a LeakyReLU activation, LayerNorm, and dropout, and continues through L_{shared} residual blocks. This hard parameter-sharing approach regularizes the model by forcing it to learn a common, robust feature representation for both tasks [81].

Task-Specific Heads

Finally, the processed shared representations are passed to dedicated, unshared task-specific heads. These heads generate the final predictions for each task, allowing for specialized output generation and isolated gradient computation during backpropagation.

Rationale for Hard Parameter Sharing

The choice of hard parameter sharing is motivated by its efficiency and regularization benefits, which are critical for our target application. This approach is supported by several factors:

- **Computational Efficiency:** Unlike soft-sharing methods that often introduce task-specific parameters and increase computational overhead, hard sharing maintains a compact model. This is crucial for deployment on resource-constrained edge devices (e.g., NVIDIA Jetson platforms).
- **Implicit Regularization:** By constraining the hypothesis space, hard sharing acts as a regularizer, often leading to more generalizable models, especially when tasks are related [76]. Seminal work by Caruana demonstrated significant error reduction with this approach [7].
- **Empirical Performance:** In practice, sharing one network across tasks reduces inference cost, since one network is evaluated instead of one per task [45].[4]

This architecture strategically balances model capacity with knowledge transfer, prioritizing efficiency for potential real-time recommendation scenarios.

3.3.3 Nash-MTL Optimizer

MTL necessitates a unified update direction that benefits all tasks, even when their gradients conflict. Nash-MTL formulates this as a cooperative K-player bargaining game [52], where each task acts as an “agent” seeking to maximize its utility (loss reduction). The optimizer selects a step that is (i) beneficial to all tasks, (ii) scale-invariant to arbitrary loss re-weightings, and (iii) proportionally fair, ensuring no alternative direction increases one task’s utility without

[4] The published sentence read “hard parameter sharing frequently matches or exceeds the performance of more complex architectures on many benchmarks, while offering faster training and inference”. The accuracy and training-speed halves are corrected here rather than reproduced. The cited work names improved accuracy and reduced training time among the benefits joint training may have “in theory”, and then argues the other way empirically, reporting that joint training “often leads to inferior overall performance as task objectives can compete”; its contribution is a framework for assigning tasks to several networks so that competing tasks are separated, and it describes that framework as costly at training time. See Appendix B.1 of the supplementary volume of this dissertation.

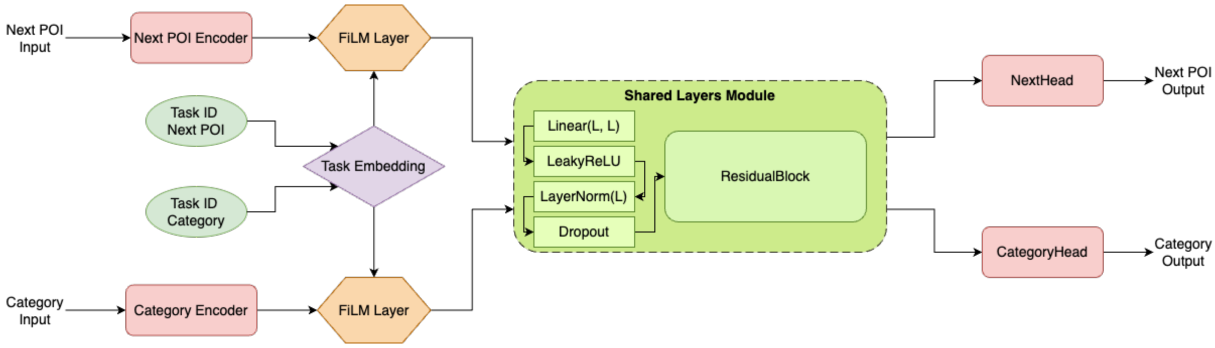

Figure 1 – The proposed Multi-Task Learning (MTL) architecture. Task-specific encoders process inputs for POI Category Classification and Next-POI Prediction. Feature-wise Linear Modulation (FiLM) layers adapt these features using learnable task embeddings. The modulated representations are then processed by a shared path of residual blocks. Finally, dedicated task-specific heads generate the respective outputs.

decreasing the product of all utilities [52]. The unique solution satisfying these axioms is the Nash Bargaining Solution (NBS). This approach allows Nash-MTL to "negotiate" at each iteration, yielding a compromise descent direction that improves every loss while maximizing the joint progress of the entire system.

Formal Formulation.

Given task-specific objectives $\{\mathcal{L}_k(\boldsymbol{\theta})\}_{k=1}^K$ and their gradients $\mathbf{g}_k = \nabla_{\boldsymbol{\theta}}\mathcal{L}_k$ at current parameters $\boldsymbol{\theta}$, Nash-MTL frames gradient aggregation as a bargaining game to determine the common update direction [52]. The utility for each task k is its signed improvement, $u_k(\Delta\boldsymbol{\theta}) = \mathbf{g}_k^\top \Delta\boldsymbol{\theta}$, where $\Delta\boldsymbol{\theta}$ is the parameter update within an ε-ball [52]. The NBS is found by maximizing the weighted geometric mean of utilities:

$$\Delta\boldsymbol{\theta}^* = \arg\max_{\Delta\boldsymbol{\theta}\in\mathcal{B}_\varepsilon} \sum_{k=1}^{K} \log\big(u_k(\Delta\boldsymbol{\theta})\big) \quad \text{s.t. } u_k(\Delta\boldsymbol{\theta}) > 0,\ \forall k. \tag{3.2}$$

By setting $\Delta\boldsymbol{\theta} = \sum_k \alpha_k \mathbf{g}_k$ with positive weights $\boldsymbol{\alpha}$, the solution simplifies to solving the nonlinear system $\big(\mathbf{G}^\top\mathbf{G}\big)\boldsymbol{\alpha} = \boldsymbol{\alpha}^{-1}$ [52], where $\mathbf{G} = \big[\mathbf{g}_1 \ \dots \ \mathbf{g}_K\big]$ [52]. The parameter update is then $\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta\,\mathbf{G}\boldsymbol{\alpha}$, with η chosen to ensure monotonic loss decrease and convergence to a Pareto-stationary point [52].

Practical Aspects.

Nash-MTL is architecture-agnostic.[5] Its invariance to the scale of the individual task gradients, the second of the axioms listed above, balances heterogeneous tasks without manual

[5] The published sentence continued "and requires only two matrix-vector products per iteration". That cost claim is corrected here rather than reproduced: it is not supported by the method's own paper, and it understates the cost. See Appendix B.1 of the supplementary volume of this dissertation.

re-weighting. For efficiency, task weights can be updated less frequently, significantly reducing runtime while maintaining performance [52].

Parameter Partition.

In our model (Figure 1), we explicitly expose two disjoint parameter sets:

$$\begin{aligned}\Theta_{\text{shared}} &= \{\texttt{task\_emb}, \texttt{film}, \texttt{shared\_layers}\},\\ \Theta_{\text{specific}} &= \{\texttt{category\_encoder}, \texttt{next\_encoder},\\ &\qquad \texttt{CategoryHead}, \texttt{NextHead}\}.\end{aligned}$$

This partition enables independent learning-rate scheduling or regularization if domain shifts between tasks emerge during fine-tuning.

The Nash-MTL optimizer was employed for its ability to aggregate tasks gradients in a balanced and principled manner, formulating the optimization process as a bargaining problem and aiming for joint progress across all tasks. In our evaluation, Nash-MTL was compared with different strategies, including PCGrad [50] and an approach with no optimizer. Among these, it was found that Nash-MTL consistently yielded a better overall performance of the MTL model, with a lower combined multi-task loss, which supported our decision to adopt it as our choice of optimizer.

3.4 Experimental Setup and Results

3.4.1 Dataset and Evaluation Metrics

To evaluate our model, we use a subset of the public *Gowalla* check-in dataset, originally collected by Cho *et al.* [12]. We focus our analysis on the data from the U.S. state of Florida, which is one of the most active regions in the original dataset. This subset comprises 20,301 users, 65,009 unique Points-of-Interest (POIs), and 990,518 check-ins.

Following previous work, each POI is mapped to one of seven high-level categories: *Community*, *Entertainment*, *Food*, *Nightlife*, *Outdoors*, *Shopping*, and *Travel*. These categories will be used for the POI category prediction task.

Evaluation protocol

To ensure a robust evaluation of our models, all experiments were conducted using a 5-fold cross-validation methodology. The folds are formed by a stratified splitter over the samples rather than over the users, so the check-ins of one user may appear in both training and validation; Chapter 5 adopts a stricter user-disjoint protocol. For the category task the sample unit is the place, so no place spans two folds. The code of record pins a single random seed, so the five folds constitute one repetition of the experiment rather than several, and the means and standard deviations reported below are the spread across those five folds at that seed; Chapter 5 repeats its five-fold experiment at four random initializations. Within a fold, training runs for the full number of epochs configured, without early stopping, and each task is read at the epoch of its own highest validation macro-F1, measured on the same fold the score is reported on. For every category, we report precision, recall, and F_1-score, and utilize the macro-average of these metrics to summarize overall performance. Class-wise metrics are crucial because category frequencies are highly skewed in real LBSN data.

3.4.2 Discussion and Comparison with Established Models

In this section, we discuss the performance of our proposed MTL model and its single-task (Single) counterpart. For a comprehensive evaluation, we selected two state-of-the-art approaches as baselines. The Human Mobility Representation Model (HMRM), introduced by Chen et al. (2020) [73], is designed for POI category classification. HMRM calculates POI embeddings by considering users' temporal visiting patterns and the co-occurrence of locations within an individual's historical trajectory. To quantify the relationship between locations and visiting times, it employs Point-wise Mutual Information (PMI). The model then utilizes matrix factorization techniques to generate latent representations (embeddings) from various inputs, and these embeddings are subsequently used to train a Support Vector Machine (SVM) for predicting POI categories.

For the task of next POI category prediction, we compare our models against MHA+PE, a solution proposed by Zeng et al. (2019) [74]. This model enhances a recurrent neural network with a Multi-Head Attention (MHA) mechanism, originally developed for machine translation by Vaswani et al. (2017) [82], and incorporates Positional Encoding (PE). The MHA mechanism allows the model to extract correlations from different parts of a sequence, effectively capturing the relevance of various elements within a user's trajectory, while positional encoding helps to propagate the order of records in a sequence.

3.4.2.1 POI Category Classification

As shown in Table 2, both our MTL and Single models outperform HMRM [73] in every POI category in terms of F1-score, precision, and recall. For instance, in the 'Shopping' category, our MTL model achieves an F1-score of 62.51 ± 0.94 compared to HMRM's 46.69 ± 0.81. Similarly, for 'Food', the MTL model scores 57.43 ± 1.46 against HMRM's 28.44 ± 0.42. While both MTL and Single models are competitive, the Single model shows marginally better F1-scores in several categories like 'Community' (53.11 ± 0.58) and 'Outdoors' (47.75 ± 0.89), whereas the MTL model excels in 'Food' and 'Shopping'. Overall, our approaches demonstrate a substantial improvement over the HMRM baseline for this task.

Table 2 – POI category classification results for Florida (per-category F1-score, precision, and recall, mean ± standard deviation over the 5 folds; the better of the MTL and Single values per row in bold, as in the published table).

		MTL	**Single**	**HMRM**
F1	Community	52.13 ± 0.90	**53.11** ± **0.58**	20.25 ± 0.52
	Entertainment	41.44 ± 1.29	**41.73** ± **1.65**	13.07 ± 1.14
	Food	**57.43** ± **1.46**	56.70 ± 0.84	28.44 ± 0.42
	Nightlife	29.94 ± 2.27	**34.22** ± **2.08**	25.54 ± 1.73
	Outdoors	47.19 ± 1.01	**47.75** ± **0.89**	16.11 ± 0.68
	Shopping	**62.51** ± **0.94**	61.88 ± 1.01	46.69 ± 0.81
	Travel	43.16 ± 1.43	**43.20** ± **1.94**	15.25 ± 1.19
Precision	Community	**51.39** ± **1.77**	49.54 ± 1.70	21.13 ± 0.75
	Entertainment	44.08 ± 1.46	**44.47** ± **3.66**	14.52 ± 1.34
	Food	58.05 ± 1.23	**59.04** ± **1.12**	48.94 ± 0.88
	Nightlife	**36.07** ± **2.05**	32.52 ± 1.83	18.34 ± 1.34
	Outdoors	46.39 ± 1.12	**47.74** ± **1.09**	13.00 ± 0.48
	Shopping	60.24 ± 1.21	**61.59** ± **1.44**	39.63 ± 0.82
	Travel	**46.48** ± **1.12**	44.69 ± 4.89	24.58 ± 1.91
Recall	Community	52.93 ± 1.35	**57.37** ± **2.46**	19.48 ± 0.99
	Entertainment	39.14 ± 1.69	**39.88** ± **4.93**	11.90 ± 1.08
	Food	**56.99** ± **3.71**	54.60 ± 2.01	20.05 ± 0.32
	Nightlife	25.80 ± 3.54	**36.77** ± **5.86**	42.13 ± 3.28
	Outdoors	**48.09** ± **2.11**	47.79 ± 1.51	21.27 ± 1.78
	Shopping	**65.04** ± **2.69**	62.28 ± 2.83	56.82 ± 0.98
	Travel	40.39 ± 2.73	**43.08** ± **7.15**	11.07 ± 0.95

3.4.2.2 Next POI Category Prediction

The results for next POI category prediction, detailed in Table 3, indicate a competitive performance landscape. The MHA+PE model [74] shows strong F1-scores in several categories, notably ‘Food’ (43.47 ± 0.50) and ‘Shopping’ (43.53 ± 1.66), outperforming our MTL and Single models in these instances. However, our MTL model achieves the best F1-score in ‘Travel’ (64.61 ± 1.11) and ‘Nightlife’ (22.07 ± 0.52), surpassing MHA+PE in these two categories. The Single model also shows competitive results, leading in ‘Entertainment’ (26.06 ± 1.01) and ‘Outdoors’ (22.45 ± 0.81). This suggests that while MHA+PE is highly effective for certain POI categories, our MTL and Single models offer superior or comparable performance in others, particularly where different sequential patterns might be exploited. One further point concerns how these leads are distributed. The MTL and Single models do not lead consistently across the categories, each obtaining the better F1-score in some of them, and this uneven distribution could make either model appear worse in the categories where the other one leads.

Overall, while our proposed models show strong performance in POI category classification against HMRM [73], the next POI category prediction task presents a more competitive

Table 3 – Next POI category prediction results for Florida (per-category F1-score, precision, and recall, mean ± standard deviation over the 5 folds; best value per row in bold, second-best underlined).

		MTL	Single	MHA+PE
F1	Community	34.79 ± 1.05	<u>34.90 ± 0.61</u>	**35.83 ± 1.70**
	Entertainment	24.21 ± 1.75	**26.06 ± 1.01**	<u>25.48 ± 0.86</u>
	Food	<u>28.06 ± 3.55</u>	26.35 ± 2.97	**43.47 ± 0.50**
	Nightlife	**22.07 ± 0.52**	<u>21.79 ± 0.30</u>	1.55 ± 1.50
	Outdoors	<u>21.61 ± 0.50</u>	**22.45 ± 0.81**	12.56 ± 1.94
	Shopping	<u>42.58 ± 2.06</u>	42.18 ± 1.13	**43.53 ± 1.66**
	Travel	**64.61 ± 1.11**	<u>64.32 ± 0.88</u>	26.91 ± 0.97
Precision	Community	<u>30.22 ± 1.36</u>	30.04 ± 1.08	**42.59 ± 1.48**
	Entertainment	<u>32.64 ± 2.55</u>	28.63 ± 2.85	**38.72 ± 1.56**
	Food	39.05 ± 0.70	**39.87 ± 1.48**	35.91 ± 0.81
	Nightlife	<u>17.71 ± 2.40</u>	15.27 ± 0.67	**39.67 ± 23.94**
	Outdoors	<u>19.63 ± 2.93</u>	19.50 ± 2.08	**46.83 ± 1.82**
	Shopping	<u>42.57 ± 0.78</u>	**43.47 ± 1.08**	39.04 ± 1.06
	Travel	<u>60.64 ± 1.87</u>	**63.21 ± 0.67**	36.64 ± 1.74
Recall	Community	41.54 ± 5.27	**41.92 ± 3.49**	31.16 ± 3.20
	Entertainment	19.52 ± 3.18	**24.16 ± 2.16**	19.05 ± 1.25
	Food	<u>22.19 ± 4.32</u>	19.94 ± 3.75	**55.12 ± 1.46**
	Nightlife	32.13 ± 9.47	**38.35 ± 3.11**	0.80 ± 0.79
	Outdoors	25.06 ± 4.26	**27.07 ± 3.40**	7.30 ± 1.32
	Shopping	42.87 ± 4.56	41.05 ± 2.33	**49.37 ± 3.74**
	Travel	**69.36 ± 4.15**	<u>65.52 ± 2.25</u>	21.30 ± 1.03

landscape when compared against a sophisticated sequential model like MHA+PE [74]. Comparing the MTL and Single models directly reveals that the multi-task learning approach did not yield the anticipated substantial improvements over the single-task baseline across both tasks. Crucially, many of these observed differences in F1-scores are minor and often fall within the reported standard deviations, suggesting that the performance of the MTL and Single models is largely comparable, without a clear or consistent advantage for the multi-task learning setup in these experiments.

3.4.3 Convergence Comparison

To further evaluate the practical implications of the MTL approach compared to single-task models, we conducted a convergence experiment. We measured the wall time, number of epochs, and Mega Floating Point Operations (MFLOPs) required for the MTL model and the individual single-task models (SingleClass and SinglePred) to reach predefined target average F1-scores. The target F1-score for the *Category* prediction task was set to 47, and for the *Next* POI category classification task, it was set to 32.2. These specific F1 values were chosen because they are close to the best-achieved results for each respective task, thereby representing a comparable and significant level of predictive performance for the models to attain. To ensure robust measurements for this experiment, all models were evaluated through a 5-fold cross-validation process. The specific metrics are detailed in Table 4.

The convergence measurements show a clear wall-time disadvantage for the joint model, while the MFLOPs column does not follow the same pattern. Table 4 reports the three metrics for each model.

Table 4 – Convergence metrics to reach the target F1-score for each model (wall time in seconds, number of epochs, and MFLOPs; 5-fold cross-validation).

Model	**Time (s)**	**Epochs**	**MFLOPs**
Category	16.26	3.8	2.315
Next	18.71	3.2	0.012
MTL	80.88	3.2	0.234

The results from this experiment were contrary to the potential expectation that an MTL framework might offer training efficiencies. Our findings indicated that the MTL model took substantially longer to converge to the target F1-scores. Specifically, the MTL approach required 80.88 s of wall time, about 2.3 times the cumulative 34.97 s of the individual single-task models. In terms of MFLOPs, in contrast, the table does not show a higher cost for the MTL setup: the MTL model required 0.234 MFLOPs, against 2.315 MFLOPs for the Category model and 0.012 MFLOPs for the Next model. This suggests that while attempting to learn multiple tasks simultaneously, the MTL model, in this configuration, incurred a significantly higher overhead in wall time. Given these observations, for deployment scenarios where convergence speed is a critical factor, employing separate, optimized single-task models would likely be a more convenient strategy.[6]

3.5 Conclusion and Future Work

In this chapter, we introduced a Multi-Task Learning (MTL) framework employing hard parameter-sharing, task-specific encoders, and FiLM layers to jointly address POI Category Classification and Next-POI Prediction using DGI-based POI embeddings.

Essentially, the MTL approach did not consistently demonstrate superior performance over its single-task counterparts and exhibited higher computational demands in terms of convergence wall time. These findings indicate that the anticipated benefits of MTL are not universally guaranteed and are highly dependent on task compatibility and architectural design.

[6] Experiments conducted on Apple M2 Pro (10-core CPU, 16-core GPU) with 32GB unified memory running macOS 15.5. Key software versions: Python 3.9.6, PyTorch 2.6.0.dev20241014+cpu, NumPy 1.26.4, scikit-learn 1.5.2, CVXPY 1.5.2. Training utilized PyTorch MPS backend for Apple Silicon acceleration.

The lack of significant improvement from the MTL model prompts an analysis of the potential underlying causes, especially since optimizers like Nash-MTL mitigated overt gradient conflicts. We hypothesize three primary factors contributed to this outcome:

- **Subtle Negative Transfer due to Task Dissimilarity:** Although related, the two tasks possess fundamentally different natures. POI Category Classification is a static task that relies on the intrinsic, context-rich features of a single POI embedding. In contrast, Next-POI Prediction is a dynamic, sequential task that depends on capturing temporal patterns and transitions within a user's trajectory. The shared encoder may have been forced to learn a "compromise" representation that was not specialized enough for either task, thus failing to outperform the focused single-task models.
- **Task Difficulty and Representation Mismatch:** The performance gap between the two tasks (with category classification achieving higher F1-scores) suggests an imbalance in difficulty. The representation learned by the shared layers might have become biased towards the features required for the simpler, static classification task, inadvertently hindering its effectiveness for the more complex sequential prediction task. The shared representation may not have been rich enough to simultaneously encode both semantic properties and sequential dynamics effectively.
- **Architectural Restrictiveness:** Our choice of a hard parameter-sharing architecture, while efficient and strongly regularizing, may have been too restrictive for the tasks. The distinct nature of the tasks might require more flexible computational pathways. A single shared block may be insufficient to learn a representation that benefits both, suggesting that soft-sharing or expert-based models could be more appropriate.

These insights contribute to the broader understanding of MTL's practical challenges. Future research will proceed along several directions motivated by these hypotheses. We plan to explore alternative parameter-sharing mechanisms, such as **soft sharing (e.g., Cross-Stitch Networks) or Mixture-of-Experts (MoE) models**, to test the hypothesis that the hard-sharing architecture was overly restrictive. In addition, investigating **advanced multi-task optimizers and loss-balancing schemes** beyond gradient conflict mitigation could address the task difficulty imbalance. Finally, a deeper analysis of **task relatedness**, perhaps using techniques to measure feature representation overlap, could help quantify the degree of negative transfer and guide future architectural choices.

4 ST-MTLNet: Spatio-Temporal POI Representations for Multi-Task Learning

This chapter is a translated reproduction of the paper "ST-MTLNet: Representações Espaço-Temporais de Pontos de Interesse para Aprendizado Multitarefa," published in Portuguese in the Anais do X Workshop de Computação Urbana (CoUrb 2026, an SBRC workshop), pages 323 to 336, DOI 10.5753/courb.2026.22960 [83]. Tarik S. Paiva is the first author of the paper; the author of this dissertation is the second author, presented the paper at the workshop, and is the first author of the baseline model MTLnet, introduced in Chapter 3. The evaluation protocol of this study stratifies the cross-validation split by sample rather than by user, a weaker protocol than the user-disjoint cross-validation adopted in Chapter 5; the conclusions reported here are those of the time, for that configuration. This chapter isolates the representation effect with MTLnet as its only baseline; it does not revisit the MTL-versus-single-task question, which Chapter 5 reopens. As in Chapter 3, the term "Next-POI Prediction" used here denotes the frame's next category *task (the category of the next visited place), not the exact-place task the dissertation calls* next place. *Appendix B of the supplementary volume of this dissertation records what we established after publication about the scope of this chapter's static task, and that record should be read alongside the category results reported here.*

4.1 Introduction

Location-Based Social Networks (LBSNs), such as Gowalla [12, 72], record user *check-ins* with geographic location, visit time, and category of the visited place. In this context, a Point of Interest (POI) corresponds to a semantically identifiable physical place, such as restaurants, stores, airports, or parks. The analysis of these data is relevant to applications such as recommendation and urban mobility. In this scenario, two tasks stand out:

1. **POI Category Classification**: classify the semantic category (e.g., *Food*, *Shopping*, *Travel*) of a POI from its features.
2. **Next-POI Prediction**: predict the category of the next POI that a user will visit, given the historical sequence of their *check-ins*.

In principle, these tasks may be treated as related, since they share the same mobility dataset and the same semantic space of categories, which makes multi-task learning (MTL) a promising strategy to exploit information shared between them [7]. However, they impose distinct demands. POI Category Classification is a non-sequential task, since it depends mainly on the attributes of the POI itself and on its spatial context. Next-POI Prediction is explicitly sequential, requiring the modeling of temporal order, recurrence, and transitions along the user trajectory. Thus, the information useful for one task may not benefit the other in the same way.

MTLnet, proposed in [1], exploits this potential by modeling the two tasks in a single multi-task architecture with hard parameter sharing. However, the model uses exclusively vector representations based on the graph structure, generated by *Deep Graph Infomax* (DGI), which encodes spatial and categorical information in a single 64-dimensional vector, without explicitly incorporating continuous geographic coordinates or temporal visitation patterns.

Given this, the question arises of whether decomposing the input into specialized *encoders* for space, time, and category produces a base more suitable for the two tasks than the monolithic *embedding* of DGI. The hypothesis is that decoupled representations capture complementary dimensions of human mobility that are not explicitly modeled by the original approach.

In this sense, this chapter proposes ST-MTLNet (*Spatial-Temporal MTLNet*), which integrates three independent representations: a continuous spatial representation for geographic coordinates, a temporal representation (Time2Vec) for visitation patterns, and a hierarchical categorical representation for regional and structural relationships between POIs. In the spatial dimension, we evaluate two architectures with distinct assumptions, SIREN and Sphere2Vec-M, originally validated in geospatial tasks of remote sensing and ecology [6], but still little explored in multi-task learning of POIs in LBSNs.

The experiments were carried out with the public Gowalla dataset [72] in the states of Florida, California, and Texas. In categorical classification, ST-MTLNet outperforms MTLnet in all evaluated category-state combinations, with average gains per state of 20.2 to 22.0 percentage points, considering the better of the two spatial encoders in each combination. In Next-POI Prediction, the proposed model outperforms the *baseline* in most scenarios, with emphasis on categories such as *Food*. The comparison between SIREN and Sphere2Vec-M also shows that, although the modular approach is consistently superior to the *baseline*, the most suitable spatial *encoder* depends on the geographic distribution of the POIs in each territory.

Therefore, the main contributions of this chapter are:

- The proposal of ST-MTLNet, which incorporates continuous spatial, temporal, and hierarchical categorical representations into MTLnet.
- The comparative evaluation of SIREN and Sphere2Vec-M in three states with distinct

territorial structures.

- The public release of the source code and the experimental *pipeline*.[1]

This chapter is organized as follows: Section 4.2 presents the related work. Section 4.3 describes the proposed methodology. Section 4.4 discusses the experimental results. Finally, Section 4.5 presents the conclusions and future work.

4.2 Related Work

POI prediction and classification are fundamental challenges in location-based recommendation systems and in urban mobility analysis. This section organizes the related work into four axes: graph-based representations, spatial encoders, temporal representations, and multi-task learning applied to POIs.

4.2.1 Graph-Based POI Representations

Modeling POIs through graphs allows capturing spatial and semantic neighborhood relationships between places. The POI2Vec model [84] adapts the Word2Vec architecture to generate POI *embeddings* that incorporate the geographic influence between places through a geographic binary tree structure. CATAPE (*Category-Aware Location Embedding*) [85] extends this idea by incorporating categorical and sequential information, capturing the geographic influence between POIs based on the temporal sequence of user visits.

In a self-supervised approach, *Deep Graph Infomax* (DGI) [33] maximizes the mutual information between local (node) and global (graph) representations to generate *embeddings* without explicit supervision. HGI (*Hierarchical Graph Infomax*) [34] advances this line by operating at multiple hierarchical levels (POI, region, and city), producing representations enriched with regional information through graph convolutions and multi-attention mechanisms.

However, these approaches encode spatial information implicitly through the graph topology, without directly using the geographic coordinates as an input signal for the representation.

4.2.2 Spatial Representations

The direct use of geographic coordinates to generate latent spatial features is an approach distinct from graph-based representations. The TorchSpatial *framework* [6] establishes a unified *benchmark* for evaluating spatial representation strategies, allowing systematic comparison between different architectures.

[1] The complete source code is available at <https://github.com/TarikSalles/Spatial_Embeddings>

The SIREN methodology (*Sinusoidal Representation Networks*) [36], applied to the geographic context [38], models continuous functions through sinusoidal activations, being able to represent high-frequency signals in spatial data. Sphere2Vec [5] operates directly on spherical coordinates with multiple resolution scales, preserving geodesic distance properties that are lost in planar projections.

Although these strategies have been evaluated on geospatial tasks such as species prediction and population estimation, their application can be extended to POI prediction and classification models as proposed in this chapter.

4.2.3 Temporal Representations

The temporal dimension is central to POI prediction tasks, since user visitation patterns present regularities (e.g., meal times, weekly movements). The LSTPM model (*Long- and Short-Term Preference Modeling*) [86] captures short- and long-term preferences using a Geo-Dilated RNN that relates non-consecutive POIs through temporal sequences of visits and geographic information. TransTARec [87] proposes an adaptive temporal translation model that unites representations of the POI, the *timestamp*, and user preferences for Next-POI Prediction.

These models incorporate the temporal dimension mainly in the context of the user's visit sequence. In this sense, Time2Vec [35] proposes a temporal representation that combines linear terms with sinusoidal functions of learnable frequency and phase, capturing periodic and non-periodic patterns from continuous temporal values.

4.2.4 Multi-Task Learning for POI Tasks

Multi-task learning (MTL) [7] has been explored in POI tasks mainly to combine Next-POI Prediction with temporal, behavioral, or regional signals. MCARNN [60] integrates sequential dependencies and temporal regularities to simultaneously predict future activities and places. MTPR [78] jointly models location and temporal context through geographic LSTMs and adversarial learning. In a more recent line, Halder et al. [88] propose a multi-task model based on *Transformers* to recommend the next POI and simultaneously predict the waiting time.

In categorical classification, TME [4] explores a hierarchical category structure for semantic annotation of POIs, while HMT-GRN [21] combines Next-POI Prediction and prediction of its geographic region in a recurrent model with hierarchical graphs. Together, these works indicate that MTL is promising in POI problems, but there is still little exploration of architectures that jointly address *POI Category Classification* and *Next-POI Prediction* with continuous spatial and explicit temporal representations. This gap motivates the investigation proposed in this chapter.

4.2.5 The MTLnet framework

The baseline of this study is MTLnet, the joint architecture introduced in Chapter 3 [1]. MTLnet addresses POI Category Classification and Next-POI Prediction in one model with hard parameter sharing: task-specific encoders project the input *embeddings* into a shared latent space, FiLM modulation conditions the shared layers on the task identity, and specialized heads produce the two outputs, with Nash-MTL balancing the task gradients during training. Its input is a single 64-dimensional *embedding* per POI, produced by Deep Graph Infomax over a spatial graph.

The evaluation in Chapter 3 found that this joint model performed on par with the dedicated single-task models at a higher training cost, a conclusion of the time for that configuration, and hypothesized that the shared input representation might not be rich enough for the two tasks. This chapter takes that hypothesis as its starting point: it keeps the MTLnet architecture unchanged and replaces only its input representation, as detailed in Section 4.3.

4.3 Methodology

As discussed in Section 4.2, MTLnet [1] uses a single vector $\mathbf{E}_{DGI} \in \mathbb{R}^{64}$ as input for both tasks, jointly encoding spatial and categorical information through the graph topology. One limitation of this monolithic approach is that it does not incorporate continuous geographic coordinates or temporal visitation patterns, factors that, as shown by works on spatial [6] and temporal [35] encoding, carry information complementary to the topological relationships of the graph.

The central design decision of this chapter is to decompose the unimodal DGI representation into three independent 64-dimensional components, trained separately and integrated by concatenation: a continuous spatial *encoder*, a temporal *encoder* (Time2Vec), and a hierarchical categorical *encoder* (HGI). The underlying hypothesis is that a decoupled representation, with specialized *encoders* trained with loss functions suited to each dimension of the information, produces more expressive representations than a unimodal representation that encodes all dimensions jointly. To evaluate this hypothesis and investigate the impact of the spatial encoding strategy, two *encoders* with distinct architectural assumptions are compared: SIREN and Sphere2Vec-M.

The proposed methodology is illustrated in Figure 2. The following describes the MTLnet model, the *baseline* with DGI, the rationale and operation of each proposed *encoder*, and the strategy for integrating the *embeddings*.

MTLnet [1] is a multi-task network built on hard parameter sharing [7], which simultaneously processes the POI Category Classification and Next-POI Prediction tasks. The architecture uses task-specific *encoders* implemented as MLPs with LayerNorm and Dropout,

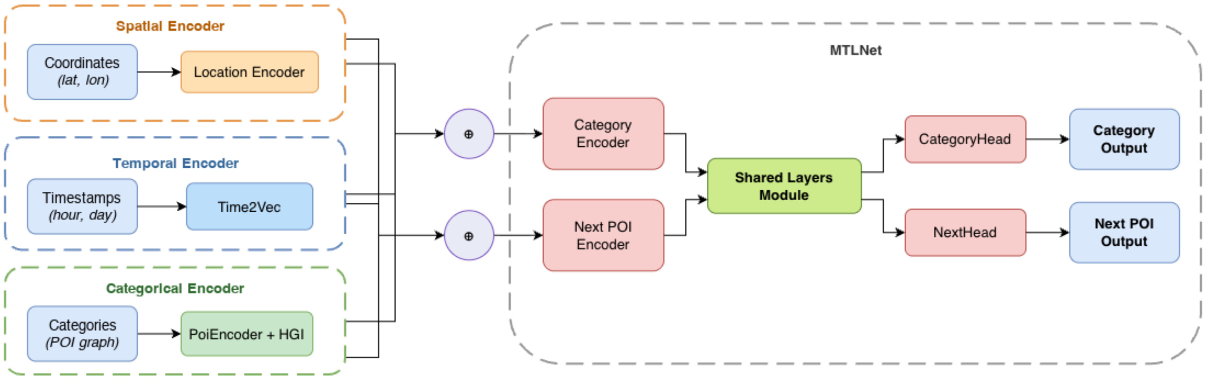

Figure 2 – Architecture based on MTLnet [1]. The spatial, temporal, and categorical *encoders* are trained in a decoupled manner and generate their respective *embeddings*, which are integrated as input to the model. The integrated input is processed by the model's shared layers (*Shared Layers Module*) and by the task-specific layers, producing the *Category Output* and *Next POI Output*.

which project the input *embeddings* into a shared latent space of dimension $d_{\text{shared}} = 256$. Next, FiLM modulation (*Feature-wise Linear Modulation*) [39] is applied, conditioned by a task-identifier *embedding* $\mathbf{e}_t$, which generates scale γ_t and shift β_t parameters to modulate the encoded representations:

$$\text{FiLM}(\mathbf{h}_{\text{enc}}^{(t)} \mid \mathbf{e}_t) = \gamma_t \odot \mathbf{h}_{\text{enc}}^{(t)} + \beta_t \tag{4.1}$$

The modulated representations are processed by four shared residual blocks with LayerNorm, LeakyReLU, and Dropout, allowing the model to learn a common representation for both tasks [81].

At the output, each task has a specialized *head*. The category classification module uses a set of three parallel MLPs with varying depths (2, 3, and 4 layers), whose outputs are concatenated and projected to the 7 POI classes. The next-POI *head* employs a *Transformer encoder* with 8 attention *heads* and 4 layers, a causal mask for autoregressive processing, and attention-weighted pooling using attention weights learned over the *timesteps* of the input sequence.

Multi-task training uses the Nash-MTL regularizer [52], which formulates gradient balancing as a cooperative Nash bargaining game among K tasks. Given the gradients of each task, Nash-MTL seeks the update direction that maximizes the product of the utilities of all tasks. Away from a Pareto-stationary point, meaning a point at which some convex combination of the task gradients is zero, and under the method's assumption that the gradients are linearly independent there, that direction is a descent direction for every task, avoiding the dominance of one task over the other.

4.3.1 Baseline: MTLnet with DGI

The *baseline* model corresponds to the original MTLnet proposed in [1], whose internal architecture is kept unchanged in this chapter. In this model, each POI is represented by a single monolithic *embedding* $\mathbf{E}_{DGI} \in \mathbb{R}^{64}$, obtained via *Deep Graph Infomax* (DGI) [33] over a spatial graph built by Delaunay triangulation and categorical node attributes, as described in detail in [1]. Thus, DGI jointly encodes spatial and categorical relationships in a single vector, without incorporating explicit temporal components. In contrast, the proposed approach replaces this monolithic representation with the concatenation of three specialized *encoders*, totaling 192 input dimensions.

4.3.2 Data Preparation

For the *POI Category Classification* task, each POI is represented by the *embedding* resulting from the concatenation of the three components, $\mathbf{E}_{cat} = [\mathbf{E}_{HGI} \| \mathbf{E}_{loc} \| \mathbf{E}_{time}] \in \mathbb{R}^{192}$, forming pairs $(\mathbf{E}_{cat}, c)$ where c is the real category of the POI. In the case of the *baseline*, $\mathbf{E}_{DGI} \in \mathbb{R}^{64}$ is used directly.

For the *Next-POI Prediction* task, the *check-ins* are ordered chronologically per user, and users with fewer than five visits are discarded. Non-overlapping windows of size $L_h = 9$ are extracted: the *check-ins* $(p_1, \ldots, p_9)$ form the context, and p_{target} is the next POI whose category must be predicted. Each *check-in* in the window is represented by the corresponding *embedding*, resulting in an input of $9 \times 192 = 1728$ dimensions for ST-MTLNet and $9 \times 64 = 576$ dimensions for the *baseline*. Shorter sequences are filled with padding of zero vectors.

4.3.3 Spatial Encoder

The original MTLnet model encodes spatial information implicitly through the topology of the Delaunay graph, without directly using geographic coordinates as an input signal. However, as discussed in Section 4.2, continuous spatial encoders have shown effectiveness in various geospatial tasks [6], although their application in multi-task POI models remains little explored. This chapter selects two *encoders* that represent distinct spatial encoding paradigms: SIREN [38], which models continuous functions through sinusoidal activations with controllable frequencies, and Sphere2Vec-M [5], which operates directly on spherical coordinates preserving geodesic distance properties. This diversity allows evaluating how different geometric and architectural assumptions impact performance on POI tasks.

All *encoders* produce $\mathbf{E}_{loc} \in \mathbb{R}^{64}$ and are trained with the same binary contrastive loss based on the geographic distance between pairs of coordinates:

$$\mathcal{L}_{\text{loc}}(i, j) = -\left[y_{ij} \log \sigma\left(\frac{\text{sim}(z_i, z_j)}{\tau}\right) + (1 - y_{ij}) \log \left(1 - \sigma\left(\frac{\text{sim}(z_i, z_j)}{\tau}\right)\right)\right] \tag{4.2}$$

in which σ denotes the sigmoid function, $\mathrm{sim}(\cdot, \cdot)$ is the cosine similarity, and $y_{ij} \in \{0, 1\}$ indicates whether the pair is positive ($\Delta d_{ij} \leq 10$ km) or negative ($\Delta d_{ij} \geq 70$ km), with temperature $\tau = 0.15$. Using the same loss function for the two spatial *encoders* allows a controlled comparison, isolating the effect of the encoding architecture.

4.3.3.1 SIREN

The SIREN model (*Sinusoidal Representation Networks*) [38] models a continuous function $f_\theta : \mathbb{R}^2 \to \mathbb{R}^{64}$ that directly maps normalized geographic coordinates into a vector *embedding*. Each layer of the network uses sinusoidal activations with controllable frequencies, allowing high-frequency signals in spatial data to be represented. The output is the *embedding* $\mathbf{E}_{loc} \in \mathbb{R}^{64}$.

4.3.3.2 Sphere2Vec-M

Sphere2Vec-M [5] is a multi-scale encoder formulated to operate directly on spherical coordinates (λ, ϕ), generating representations that preserve distance properties on the surface of the sphere. The *sphereM* variant is used, in which $S = 16$ multi-resolution scales are employed with geometrically spaced factors between 10 km and 10,000 km. Interaction terms between λ and ϕ are constructed by keeping one of the components at the highest scale level, allowing the angular resolution of latitude and longitude to be controlled independently. The resulting vector is projected by an MLP to produce $\mathbf{E}_{loc} \in \mathbb{R}^{64}$.

4.3.4 Temporal Encoder

Human mobility patterns exhibit cyclical regularities, such as meal times and weekly movements, which carry discriminative information about the functional nature of the visited POIs [86]. However, the original MTLnet model does not incorporate explicit temporal representations to capture this information.

To fill this gap, Time2Vec [35] is used, a generic and decoupled temporal representation capable of capturing periodic and non-periodic patterns without the need for manual feature engineering. The architecture combines linear and periodic terms to map the temporal values of each *check-in* (hour of day and day of week, in a coupled manner) into an *embedding* vector. For an *embedding* of dimension D, the function is defined as:

$$\mathbf{f}(\tau)[i] = \begin{cases} \omega_0 \tau + \phi_0 & \text{if } i = 0 \\ \sin(\omega_i \tau + \phi_i) & \text{if } 1 \leq i \leq D - 1 \end{cases} \tag{4.3}$$

in which ω_i and ϕ_i are trainable parameters (weights and *bias*) that capture the periodicity of the temporal visitation patterns. The linear term ($i = 0$) captures global temporal trends, while the sinusoidal terms model cyclical patterns at different frequencies.

The model is trained using the discrete temporal values (hour of day and day of week) in a coupled manner for each *check-in*, with the same binary contrastive formulation employed in the spatial encoder, denoted $\mathcal{L}_{tempo}$. The final output is the *embedding* $\mathbf{E}_{time} \in \mathbb{R}^{64}$, which represents the *timestamp* of each *check-in*.

4.3.5 Categorical Encoder

The DGI used in the original model encodes categorical information through a *one-hot* matrix of 7 classes processed by the graph structure, without explicitly capturing hierarchical or regional relationships among POI categories. To enrich this dimension, the generation of categorical *embeddings* is carried out in two complementary phases. The POI Encoder learns vector representations that capture co-occurrences among categories from random walks on the spatial graph, and the HGI model [34] incorporates hierarchical information at multiple levels (POI, region, and city), producing representations that reflect both the local and the regional context.

4.3.5.1 POI Encoder

The category *embedding* $\mathbf{E}_{\text{POI\_category}} \in \mathbb{R}^{64}$ is generated by the POI Encoder, which is trained independently. The POIs are organized into a spatial graph built by Delaunay triangulation over the geographic coordinates, with edges weighted by logarithmic decay of the Haversine distance and penalization for connections between distinct counties (GEOIDs).

Over this graph, random walks (*random walks*) are executed following the Node2Vec methodology [27]. Each walk is converted into a sequence of secondary categories (*fclass*), resulting in local and global co-occurrences among categories. The model learns the *embeddings* using the *skip-gram* strategy with *negative sampling* [25, 89]:

$$\mathcal{L}_{\text{SGNS}}(i, j) = -\log \sigma\left(\mathbf{e}_j^\top \mathbf{e}_i\right) - \sum_{k=1}^{K} \log \sigma\left(-\mathbf{e}_{n_k}^\top \mathbf{e}_i\right),$$

in which $\mathbf{e}_i$ is the *embedding* of the target class, $\mathbf{e}_j$ the positive context *embedding*, and $\mathbf{e}_{n_k}$ represent the negative samples. The implementation incorporates a hierarchical regularization term [90] between category and *fclass*: $\mathcal{L}_{\text{hier}} = \sum_{(c,s)\in\mathcal{H}} \|\mathbf{e}_s - \mathbf{e}_c\|_2^2$, in which $\mathcal{H}$ contains the (category, *fclass*) pairs. The total loss is $\mathcal{L} = \mathcal{L}_{\text{SGNS}} + \lambda_{\text{hier}} \mathcal{L}_{\text{hier}}$. In the end, the resulting *embedding* is generated per category and remapped to each POI.

4.3.5.2 Hierarchical Graph Infomax (HGI)

After the generation of the *embeddings* $\mathbf{E}_{\text{POI\_category}}$, they are used as input to the Hierarchical Graph Infomax (HGI) model [34], which enriches the representation by incorporating regional geographic information. The model optimizes mutual information at two hierarchical levels (POI-region and region-city):

$$\mathcal{L} = \alpha\, \mathcal{L}_{\text{POI–region}} + (1 - \alpha)\, \mathcal{L}_{\text{region–city}},$$

in which $\alpha = 0.5$ controls the balance between the hierarchical levels. Both losses use structural negative sampling to reinforce the separability between regions and between POIs. The output is the enriched *embeddings* $\mathbf{E}_{\text{HGI}} \in \mathbb{R}^{64}$, which capture both local and regional categorical relationships.

4.3.6 Embedding Integration

The three representation components are trained independently and concatenated to form the input of both MTLnet tasks:

$$\mathbf{E}_{input} = [\mathbf{E}_{HGI} \| \mathbf{E}_{loc} \| \mathbf{E}_{time}] \in \mathbb{R}^{192} \tag{4.4}$$

The choice of concatenation as the integration strategy preserves the individual information of each component without imposing premature interactions between the representations, delegating to MTLnet the task of learning how to combine these dimensions through its shared layers and FiLM modulation. The input dimensionality of ST-MTLNet ($\mathbb{R}^{192}$) is higher than that of the *baseline* ($\mathbb{R}^{64}$), a natural consequence of the decomposition into three specialized components.

However, the MTLnet architecture projects any input to the same shared latent space of dimension $d_{\text{shared}} = 256$ through the task-specific *encoders*, so that the capacity of the shared layers and task heads remains unchanged across the evaluated models. Even so, the difference in input dimensionality may influence part of the observed gains. For this reason, an additional experimental control equalizing the dimensionality of the representations would allow validating more precisely whether the gains occur mainly from the semantic specialization of the *encoders*, and not only from the increase in input dimensionality.

Two variants of the proposed model are evaluated, named ST-MTLNet (*Spatial-Temporal MTLNet*), each using a different spatial *encoder*. The *baseline* MTLnet [1] uses only the monolithic *embedding* $\mathbf{E}_{DGI} \in \mathbb{R}^{64}$ as input for both tasks. The proposed variants replace this representation with the concatenation of the decoupled *embeddings*, keeping the temporal (Time2Vec) and categorical (HGI) *encoders* fixed and varying the spatial *encoder* between SIREN and

Sphere2Vec-M.

All models share the same hyperparameters of the MTLnet architecture, differing only in the input *embeddings*. This configuration allows isolating the effect of the representation strategy on performance, evaluating both the gain of the modular approach relative to the monolithic *baseline* and the impact of the choice of the spatial *encoder*.

4.4 Results

4.4.1 Experimental Setup

The performance of the models is evaluated using the Average F1-Score per category, reported as mean and standard deviation over 5 *folds* with a stratified split, using 80% of the data for training and 20% for validation, preserving the proportion between categories. The split is stratified by sample, not by user, so the *check-ins* of one user may appear in both training and validation; Chapter 5 adopts a stricter user-disjoint protocol. The released code of record pins a single random seed, so the five folds constitute one repetition of the experiment rather than several, and the reported standard deviations are the spread across folds at that seed; Chapter 5 repeats its five-fold experiment at four random initializations. Within a fold, training runs for the full number of epochs configured, without early stopping, and each task is read at the epoch of its own highest validation macro-F1, measured on the same fold the score is reported on.

The experiments were conducted with the Gowalla *dataset* [12, 72] in the states of Florida, California, and Texas, whose statistics are presented in Table 5. Texas concentrates the largest volume of *check-ins* of the three states, while Florida provides the smallest. The choice of these states is due to the high volume of *check-ins* and the presence of distinct urban patterns and spatial distributions, which allows evaluating the behavior of the models in scenarios with different levels of density and geographic dispersion.

Table 5 – Total number of *check-ins*, POIs, and users for Florida, California, and Texas.

States	*Check-ins*	Points of Interest	Users
Florida	990,518	65,009	20,301
California	2,535,573	148,314	36,106
Texas	3,355,419	135,570	37,522

4.4.2 POI Category Classification

The results of the POI Category Classification task are presented in Table 6.

In the three evaluated states, the two variants of ST-MTLNet outperform the original MTLnet in all 21 category-state combinations, showing that replacing the monolithic *embedding*

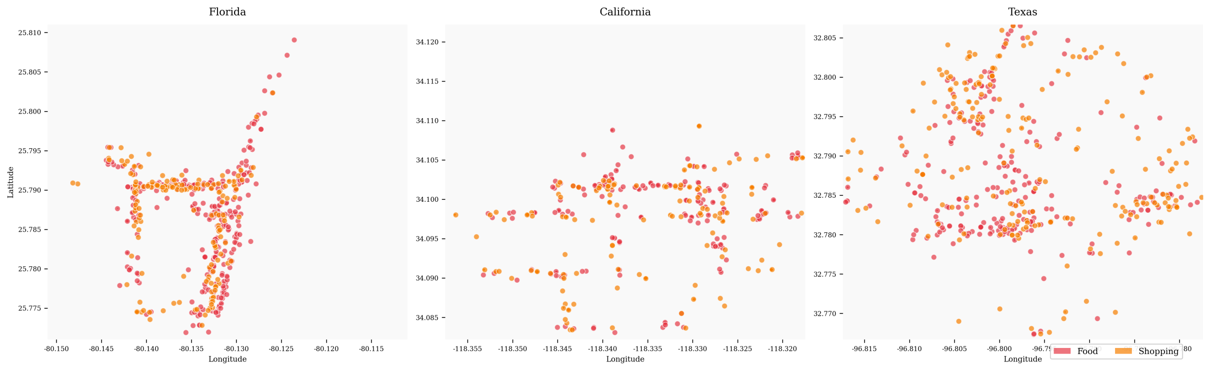

Figure 3 – Spatial distribution of POIs of the Food (red) and Shopping (orange) categories in the densest sub-region of Florida, California, and Texas. The sub-regions were selected by density in a *grid*, with about 100 POIs per region. Each panel plots the POI coordinates (longitude on the horizontal axis, latitude on the vertical axis) for one state; the dense, overlapping clusters of the two categories illustrate the co-location pattern discussed in the text.

of DGI with decoupled representations consistently enriches the characterization of the POIs. The average gains per state are 20.2 to 22.0 percentage points, considering the better of the two spatial encoders in each combination. The gains are particularly substantial in categories such as *Nightlife* and *Travel*. In *Nightlife*, for example, ST-MTLNet with Sphere2Vec-M reaches 62.44 ± 2.00 in Florida, 60.78 ± 1.63 in California, and 64.57 ± 1.03 in Texas, always well above the *baseline*. In *Travel*, the best results reach 64.89 ± 1.20 in Florida with SIREN, 63.59 ± 1.45 in California with SIREN, and 64.73 ± 0.72 in Texas with Sphere2Vec-M.

The improvements observed in *Food* and *Shopping* reinforce that the geographic location carries discriminative signals relevant to the semantic category of the POI. Figure 3 shows that these categories tend to form dense clusters in the analyzed sub-regions, favoring the capture of spatial patterns by the continuous *encoders*. Although the two proposed variants present close performance, a complementary behavior is observed: SIREN obtains more outstanding results in Florida and California, while Sphere2Vec-M stands out more frequently in Texas. Together, these results show that the modular approach consistently outperforms the monolithic paradigm of DGI in this task.

4.4.3 Next-POI Prediction

The results of the Next-POI Prediction task are presented in Table 7: the proposed variants keep the higher Average F1-Score per category in most category-state combinations, while the *baseline* retains six of them.

Unlike classification, this task presents a more heterogeneous scenario, but still favorable to the proposed variants: counting the better of the two spatial encoders per combination, the spatio-temporal models outperform the original MTLnet in 15 of the 21 evaluated combinations, with one additional technical tie in *Outdoors* in Florida, where the *baseline* mean exceeds the best

Table 6 – Average F1-Score (%) per model and state for the POI Category Classification task.

		MTLnet	**ST-MTLNet$_{SIREN}$**	**ST-MTLNet$_{Sphere2Vec-M}$**
Florida	Community	51.86 ± 0.73	**70.00 ± 0.81**	69.84 ± 0.98
	Entertainment	41.24 ± 1.83	**64.45 ± 1.82**	63.92 ± 1.61
	Food	55.47 ± 1.55	72.92 ± 0.59	**73.22 ± 0.66**
	Nightlife	32.59 ± 1.96	**62.60 ± 1.96**	62.44 ± 2.00
	Outdoors	47.71 ± 1.60	65.64 ± 1.73	**66.35 ± 1.51**
	Shopping	62.96 ± 0.62	**77.48 ± 0.36**	77.47 ± 0.72
	Travel	45.49 ± 1.20	**64.89 ± 1.20**	64.77 ± 1.54
California	Community	53.22 ± 0.37	**70.21 ± 0.17**	70.08 ± 0.44
	Entertainment	40.89 ± 1.32	**63.60 ± 0.83**	63.21 ± 0.76
	Food	60.68 ± 0.69	**75.78 ± 0.39**	75.74 ± 0.19
	Nightlife	26.81 ± 1.71	60.59 ± 1.35	**60.78 ± 1.63**
	Outdoors	51.59 ± 1.15	**67.94 ± 1.52**	67.37 ± 1.95
	Shopping	60.43 ± 1.01	76.84 ± 0.29	**77.00 ± 0.33**
	Travel	38.88 ± 1.32	**63.59 ± 1.45**	63.25 ± 0.99
Texas	Community	57.37 ± 0.69	73.45 ± 0.34	**76.20 ± 0.46**
	Entertainment	46.69 ± 0.46	64.09 ± 1.24	**67.73 ± 1.43**
	Food	54.31 ± 0.42	69.80 ± 0.80	**73.10 ± 0.17**
	Nightlife	34.44 ± 1.21	60.62 ± 1.32	**64.57 ± 1.03**
	Outdoors	45.99 ± 1.70	64.21 ± 0.42	**68.72 ± 0.70**
	Shopping	62.70 ± 0.29	76.40 ± 0.44	**79.67 ± 0.14**
	Travel	39.37 ± 1.29	57.53 ± 1.08	**64.73 ± 0.72**

variant by 0.02 percentage points, a gap within one standard deviation. The largest gains occur in *Food*, a category in which both variants outperform the *baseline* in the three states. The best result is obtained by Sphere2Vec-M in Texas, with 48.67 ± 0.66, while in California the same model reaches 51.28 ± 0.57, outperforming DGI by a wide margin. Consistent improvements also appear in *Shopping* and *Community*, suggesting that the combination between continuous spatial information and temporal context favors the modeling of recurrent transitions between categories.

The co-location pattern shown in Figure 3 helps to interpret this behavior, since the presence of dense clusters of categories such as *Food* and *Shopping* favors frequent local transitions along the user's trajectory. Even so, the *baseline* maintains an advantage in some cases, especially in *Travel* in Florida and California, in addition to specific categories in California, such as *Entertainment*, *Nightlife*, and *Outdoors*.

This behavior may be related to the nature of the *Travel* category itself, which tends to involve sparser movements between distant regions. In these cases, the graph topology used by DGI may be more efficient for preserving relationships between geographically distant POIs. On the other hand, spatial *encoders* such as SIREN and Sphere2Vec-M are coordinate-based, being more suitable for capturing local patterns, showing that continuous coordinate encodings

Table 7 – Average F1-Score (%) per model and state for the Next-POI Prediction task.

		MTLnet	**ST-MTLNet**$_{SIREN}$	**ST-MTLNet**$_{Sphere2Vec-M}$
Florida	Community	34.33 ± 1.33	**38.31 ± 0.90**	37.98 ± 1.22
	Entertainment	25.34 ± 1.07	**31.38 ± 1.72**	31.24 ± 2.41
	Food	27.12 ± 2.79	**41.91 ± 1.93**	41.65 ± 3.15
	Nightlife	21.33 ± 1.61	23.32 ± 2.74	**23.98 ± 1.94**
	Outdoors	**21.61 ± 0.99**	21.29 ± 1.59	21.59 ± 1.91
	Shopping	39.62 ± 3.40	44.00 ± 2.04	**44.66 ± 2.38**
	Travel	**64.47 ± 1.02**	45.00 ± 1.10	44.93 ± 1.11
California	Community	32.14 ± 0.89	**37.86 ± 0.20**	37.80 ± 1.11
	Entertainment	**19.38 ± 0.94**	17.28 ± 1.73	17.01 ± 1.39
	Food	29.26 ± 1.67	50.42 ± 1.15	**51.28 ± 0.57**
	Nightlife	**18.24 ± 0.53**	16.34 ± 1.95	15.10 ± 2.70
	Outdoors	**25.01 ± 0.81**	24.84 ± 1.70	24.07 ± 2.46
	Shopping	38.41 ± 2.64	**39.04 ± 1.72**	37.82 ± 1.31
	Travel	**46.05 ± 0.84**	36.94 ± 1.70	37.82 ± 1.04
Texas	Community	34.22 ± 0.43	35.99 ± 0.93	**36.97 ± 0.81**
	Entertainment	25.56 ± 1.56	**28.30 ± 1.37**	27.59 ± 1.64
	Food	29.56 ± 4.10	47.48 ± 2.42	**48.67 ± 0.66**
	Nightlife	25.46 ± 1.13	26.06 ± 1.36	**26.14 ± 2.66**
	Outdoors	23.02 ± 1.03	**24.62 ± 0.46**	23.04 ± 1.44
	Shopping	38.79 ± 4.12	**45.40 ± 1.30**	43.18 ± 2.19
	Travel	29.71 ± 1.32	**34.26 ± 0.90**	33.05 ± 1.51

and graph-based representations capture complementary aspects of mobility. Even with this limitation, the general performance indicates that the proposed representations are more effective in most sequential prediction scenarios.

4.5 Conclusion and Future Work

This chapter investigated whether replacing the monolithic *embedding* of DGI with decoupled and specialized representations could produce a base more suitable for the tasks of POI Category Classification and Next-POI Prediction. The results obtained in the three evaluated states indicate that it can. By incorporating a continuous spatial *encoder*, a temporal *encoder* (Time2Vec), and a hierarchical categorical *encoder* (HGI), ST-MTLNet consistently outperforms the *baseline* based only on DGI.

In the POI Category Classification task, the proposed variants outperform MTLnet in all 21 category-state combinations, with average gains per state of 20.2 to 22.0 percentage points, considering the better of the two spatial encoders in each combination. In the Next-POI Prediction task, the performance is also mostly favorable to ST-MTLNet, which outperforms the *baseline* in 15 of the 21 evaluated combinations, with one additional technical tie, with emphasis on categories such as *Food*. Taken together, these results show that decomposing the representation into spatial, temporal, and categorical components allows capturing patterns that DGI, by itself, does not model sufficiently.

The comparison between SIREN and Sphere2Vec-M also shows that there is no single universally superior spatial *encoder*. SIREN presents better performance more frequently in Florida and California, while Sphere2Vec-M stands out in Texas, suggesting that the suitability of the *encoder* depends on the geographic distribution of the POIs in each territory. Thus, the main gain of this chapter lies not only in the choice of a specific spatial architecture, but in the very adoption of decoupled representations as an alternative to the monolithic paradigm of DGI.

A relevant limitation remains in the *Travel* category of the Next-POI Prediction task, in which the original MTLnet still obtains the best results in part of the scenarios. This suggests that long-distance movements and topological relationships between regions are still better captured by the graph structure used by DGI. In addition, since the three proposed components are used together, this chapter does not isolate the individual contribution of each *encoder*, which restricts a more detailed analysis of the relative weight of each source of information.

Another limitation is related to the exclusive use of Gowalla, a dataset commonly used in the literature but composed of mobility records collected between February 2009 and October 2010. Consequently, the results should be interpreted with caution regarding their generalization to current urban mobility patterns.

As future work, we highlight the investigation of more flexible multi-task architectures, the exploration of more sophisticated fusion strategies between the *encoders*, and the combination of graph-based representations and continuous coordinate encodings. These directions may broaden the observed gains and help reduce the limitations still present in categories such as *Travel*.

5 A Check-in-Level Multi-Task Study of Next Category and Region

This chapter reproduces the article "Predicting the Next Category and Region of a Visit: A Check-in-Level Multi-Task Study on Mobility Data," submitted to MobiWac 2026, under review at the time of writing (EDAS #1571313639). It closes the investigation that Chapters 3 and 4 open: with a check-in-level representation and a redesigned sharing topology, a single joint model outperforms the dedicated single-task category model at all six datasets studied (five U.S. states and Istanbul) and the dedicated region model at four of the six, and it matches the dedicated region model (statistically, within two points) at the other two. The article was reformatted from its two-column original into the dissertation layout: sections, figures, and tables were renumbered, and the figures were regenerated at single-column width. No result, claim, or conclusion was altered; every departure from the submitted text is recorded in Appendix B of the supplementary volume of this dissertation.

5.1 Introduction

Location-based social networks, such as Gowalla [12], let people share the places that they visit as check-ins, which together record how people move through a city. Individual mobility is highly predictable in principle [2], so a service that anticipates the next move can prepare ahead of time, caching content where a user is heading [91] or provisioning capacity at the destination before demand arrives. Handover predictions already let cellular services adapt in advance at the network level [92]; at the city scale, check-in mobility analysis is likewise a design input for mobility-aware services [93].

Predicting the next place exactly, a single point of interest (POI), is hard and often more than a service needs. Two coarser questions are usually enough: the next category, the type of place such as food or shopping, and the next region, the part of the city that a user is heading to (a neighborhood-scale unit such as the U.S. census tract). The first captures intent, the second captures geography; we study whether one model should learn both at once.

Sharing one representation across tasks has a cost: in multi-task learning (MTL), one model does several jobs at once by sharing most of its parts, so the shared parameters can converge to a compromise optimal for neither task, helping one while hurting the other [7]. Our earlier work reported no consistent multi-task advantage for the paired category tasks and attributed it, in part, to this effect [1]; the useful question is where sharing helps, what it costs, and how to share so the gains hold and the cost stays small.

We propose two enhancements. First, we build a check-in-level representation: instead of one fixed vector per place, each check-in gets its own vector that holds its context (the time, nearby places, and recent visits). This adds a fourth level, the check-in, beneath the place, region, and city levels of hierarchical graph infomax [34, 33] (Figure 4). Second, we train one model that predicts the next category and the next region in a single forward pass, with a shared trunk (a cross-attention stack where the two tasks exchange semantic context) and a private spatial path for the region task (Figure 5). We evaluate on two settings chosen to differ: five U.S. states of different sizes (Gowalla) and one non-U.S. city (Istanbul).[1]

Our contributions are as follows.

- A check-in-level representation that extends hierarchical graph infomax from the place to the individual visit, improving next-category prediction over a standard place embedding by about +28 to +40 points of macro-F1 (a balanced score that counts every category equally) across all six datasets (Table 9). A controlled test attributes most of the gain to the per-visit context, not to extra training signal. The gap is large because a single fixed vector per place cannot separate places that serve several purposes, which per-visit context recovers.

- A single model for joint next-category and next-region prediction, sharing semantic context across tasks while keeping a private spatial path for the region task; to our knowledge, the first work to treat fine-grained region as an end target of equal standing (Section 5.2.3). In one forward pass, it outperforms a dedicated single-task category model on every dataset (by about +5 to +9 points of macro-F1), even though each dedicated model is tuned per dataset. On region, it outperforms at four of the six datasets and stays statistically non-inferior within a two-point margin (TOST, a test of equivalence) at the remaining two (Table 10).

- An empirical account, across five U.S. states and one non-U.S. city, of how the joint gain scales with the number of regions. The category gain holds on all six datasets. Across the U.S. states, the region result moves monotonically with the region count: the joint model matches the dedicated model (TOST, ±2 percentage points, or pp) at the small region counts and outperforms it at the large region counts, with the state with the most regions (California) showing the largest region gain (Figure 7). At Istanbul, the dataset with the fewest regions, the joint model is also ahead on region (+0.19 Acc@10). All joint and dedicated results average four random initializations (Section 5.6.2).

[1] Code (model, representation, baselines, statistical tests): <https://github.com/VitorHugoOli/PoiMtlNet/tree/mobiwac>. Both data sources are public: the category-annotated Gowalla dump (<https://figshare.com/articles/dataset/gowalla_data/22126586>) and Massive-STEPS [62].

The remainder of this chapter presents related work (Section 5.2), the problem (Section 5.3), the method and setup (Sections 5.4 and 5.5), results (Section 5.6), and discussion and conclusions (Sections 5.7 and 5.8).

5.2 Background and Related Work

5.2.1 The MTLnet framework and the representation diagnosis

Two chapters of this dissertation precede this study, and this chapter answers the question that they leave open. Chapter 3 introduced MTLnet, the first joint model for this task pair: task-specific encoders feed shared layers conditioned on task identity, and a gradient-balancing method weights the two tasks during training [1]. On a place-level embedding with hard parameter sharing, that joint model did not outperform the dedicated single-task models, the conclusion of the time for that configuration. Chapter 4 kept the MTLnet architecture unchanged and replaced its single 64-dimensional place-level input with decomposed spatial, temporal, and categorical encoders (ST-MTLNet); the category score rose at each of the three states studied, so the input representation, not the sharing architecture, was identified as the bottleneck [83]. This chapter carries that diagnosis one step further: the representation moves from the place level to the check-in level, the sharing topology is redesigned around it, and the earlier negative conclusion is revisited under this new configuration.

5.2.2 From place embeddings to per-visit embeddings

A place embedding transforms a POI into a vector so that similar or nearby places sit close together in the vector space. Deep Graph Infomax (DGI) [33], a general self-supervised method for graphs, learns such vectors on a place network linked by similarity, time, and distance, training them to tell the real network from a copy with shuffled node features. Hierarchical Graph Infomax (HGI) [34] adds geographic structure, organizing the network as place, then region, then city. Both produce one vector per place, so two visits to the same coffee shop look identical to the model. Contextual check-in representations instead give each visit its own vector: CTLE [20], the closest example, learns a vector per visit by masking and reconstructing parts of a user's check-in sequence. Our representation, Check2HGI, also works at the check-in level but through a different construction. CTLE is a sequence model, a Transformer that reads the check-in sequence itself; Check2HGI remains a network model, keeping the hierarchical place-to-region-to-city network and the same infomax objective, now extended one level deeper, from individual places down to individual check-ins (Figure 4). The order of a user's visits still enters the network, as links between consecutive check-ins (Section 5.4.1).

The novelty is this specific combination: per-visit context inside a hierarchical graph-infomax representation. We compare against CTLE and a feature-concatenation control to separate the combination from contextualization alone and from feature injection.

5.2.3 Predicting the next category and the next region

A large body of work predicts the next place that a user will visit, the exact POI, with recurrent and attention models such as DeepMove [14], STAN [17], and GETNext [19]. In contrast, we target two properties of the next visit, its category and its region, rather than the exact place. We choose them because they are what a mobility-aware service can act on, not to make the task easier; the region alone spans hundreds to several thousand classes. Predicting over a partition of the map is also the standard formulation in the human-mobility literature, with a grid cell as the target [3]; our next-region task substitutes official neighborhood-scale units for grid cells. Our earlier work [1] paired static category classification with next-category prediction and found no consistent multi-task gain; this chapter introduces the next-region task and adds the check-in-level representation, on which sharing helps instead of hurting (Section 5.6.2).

The field increasingly models several granularities at once; in those systems, category and region are auxiliary signals that help a primary next-place task (MCMG [94], HMT-GRN [21]). We study the pair as the object itself. To our knowledge, fine-grained region as an end target of equal standing, rather than an auxiliary coarse grid cell, is underexplored. The nearest exceptions do not study our exact pairing: DRRGNN [23] forecasts a person's next activity region jointly with a mobility-intention label, but over regions discovered per person rather than a fixed citywide partition; a generative recommender [95] predicts category and region together, but only as auxiliary steps toward its next-place ranking.

5.2.4 Multi-task learning for mobility

MCARNN [60] jointly predicts activity and location, and a recent hierarchy-aware model continues the pattern [96]; in both, the location target is the exact place. The closest alternative to our setting is the cascade: predict the category first, then the place given the category [97]. CSLSL [61] is its current form, predicting in a chain (when, then what, then where), with location as the primary output and category as an intermediate step; CatDM [22] likewise uses the predicted category only to filter candidate places. We instead predict category and region in parallel, as end targets of equal standing in one model with a single forward pass, and we drop the next-place target entirely, so neither task is an intermediate step toward a third. Since the cascade is the pattern that the field uses, we test the choice directly (Section 5.6.2). On optimization, we are conservative by design: joint training with a fixed loss weighting is standard practice [7], not itself our contribution. Reports find that gradient-balancing methods, which re-weight the tasks' updates during training (PCGrad [50], Nash-MTL [52]), rarely improve on a well-tuned fixed weighting with two tasks [56]. We confirm this at scale: of nineteen loss and gradient balancers screened at their default configurations at a single seed on two datasets, Alabama and Florida, including the two methods named above, none improved on a tuned fixed task weighting across both tasks and both datasets. Two exceed equal weighting on next-category at Alabama, Nash-MTL by 0.68 points and scale normalization by 0.19, and neither holds that position elsewhere: Nash-MTL loses it on the second dataset, and scale normalization gains on next-category there while collapsing on next-region. The reason is visible in the gradients. Measured during development on the same joint architecture (on an earlier preparation of the data), the cosine similarity between the next-category and next-region updates on the shared trunk averages +0.001 across training (four seeds each on four Gowalla states: Alabama, Arizona and Florida, which are three of the five United States datasets reported here, and Georgia, which this dissertation does not otherwise use; the largest per-dataset mean in absolute value is +0.0032). A balancer therefore has no conflict to resolve, consistent with the mechanism test in Section 5.6.2; this is a finding for this pair of tasks, not a general rule.

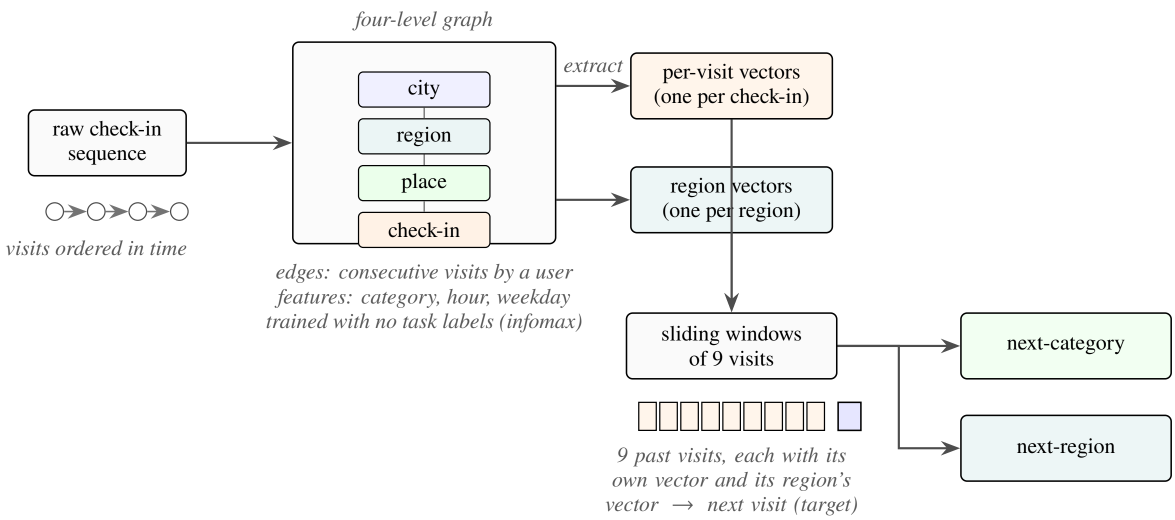

Figure 4 – From a check-in sequence to two predictions: each visit is embedded in a four-level graph (check-in, place, region, city) trained without task labels. The graph yields two sets of vectors, one per visit and one per region; sliding nine-visit windows of each feed a single model that outputs the next category and the next region.

5.3 Problem and Tasks

Given a user's time-ordered check-in history, we predict two properties of the next visit. The first is its *category*, one of seven fixed labels: Community, Entertainment, Food, Nightlife, Outdoors, Shopping, and Travel. Istanbul's source collection maps its places onto the same seven labels. The second is its *region*, the place's census tract (for Istanbul, the *mahalle*, a municipal neighborhood). We do not predict the exact next place; both properties are easier to learn and, for most uses, enough. Region, unlike category, is a large label set: we frame next-region as classification over candidate regions, from 520 classes (Istanbul) to 8,501 (California; Table 8).

The motivation is practical: the category says what type of place comes next, hence what a user will want; the region says where, hence where to prepare. The mobility literature supports such preparation: city services have long been built on location-based social network traces [11]. A recent analysis of tourist check-in mobility finds that visits concentrate on a few hub places, argues that crowds and demand therefore concentrate, and ties this structure to infrastructure planning and adaptive strategies [93]. That analysis reads past visits; we predict two properties of the next one, and aggregated over users, those predictions point to where demand is heading before it arrives. A census tract is a neighborhood, not a radio cell. We therefore scope our claims to neighborhood-level preparation: demand and load anticipation, caching content ahead of time, and capacity planning. Cell association and handover are radio-level decisions and remain out of scope. We build and evaluate no such service here; Section 5.7 quantifies what these predictions would give such a service.

5.4 Method

5.4.1 The check-in-level representation

We want each visit, not each place, to have its own vector; we call the resulting representation Check2HGI (Section 5.2.2). Figure 4 shows the full data flow, from check-ins to the two outputs.

We build a graph with four levels: the check-in, its place, the place's region (a census tract), and the city. Edges connect each level to the one above it and link a user's consecutive check-ins with a weight that decays as the time gap between the visits grows; same-place visits connect through their shared place node one level up, and nearby places are linked at the place level. Each visit's category, time of day, and day of week enter as input features of its node, not as edges. This keeps a hierarchical graph's place-to-region-to-city geography and adds the check-in as a bottom level. We train the graph mainly with an infomax objective, under which each vector learns to match its real neighborhood and reject a shuffled one [33, 34]. Two small label-free auxiliary terms are added (weights 0.3 and 0.1): a masked reconstruction of each place's aggregated category features, and an anchor to a place embedding pre-trained, label-free, on the same data. The training uses no task label: it never sees the next category or the next region. From the trained graph, we extract one 64-dimensional vector per check-in (its region nodes have trained vectors as well), so a model sees a sequence of per-visit vectors rather than repeated per-place ones.

5.4.2 The single multi-task model

We then train one model that reads a window of recent per-visit vectors and predicts the next visit's category and region at once. Figure 5 shows the design. Each task has its own input.

The category task reads the window of per-visit vectors (the semantic stream); the region task reads the same window of visits, each visit now represented by the trained vector of its region node from the same graph (the spatial stream). Both pass through private per-task encoders (a small input network per task, with no shared weights) into the shared trunk, a cross-attention stack of two blocks, so each prediction still uses both inputs. In each block, attention lets each stream read the other's features, while each keeps its own feed-forward weights. The tasks therefore share by exchanging information between per-task streams, not by owning hidden layers in common. The trunk then feeds a category output and a region output; the region output also keeps a private spatial path: a small branch inside the one model (not a second model) that reads the spatial input window, bypassing the shared trunk. The category task does not touch this branch. Together, the region task's encoder, its output, and this private path form the region pathway.

The training loss is a fixed-weight sum of the two outputs,

$$L = 0.75\, L_{\text{cat}} + 0.25\, L_{\text{reg}}, \tag{5.1}$$

where L_{cat} and L_{reg} are plain unweighted cross-entropy losses. The weights were tuned once on validation during development and held fixed across all six datasets. Class-weighting, tested on both outputs, lowered both region accuracy and category macro-F1. This is standard fixed-weight joint training [7], kept simple by design so that any improvement over the dedicated single-task models comes from the shared representation, not from an adaptive weighting scheme.

The joint model includes the shared trunk and both task outputs, so it is larger than either dedicated model, and a forward pass costs more compute than running the two small dedicated models: the joint model has about 4.2 million parameters at Alabama against 1.1 million for the two dedicated models combined (5.2 against 2.0 at California). What the single model provides is operational rather than arithmetic: one artifact to train, version, and deploy, and one forward pass whose one set of inputs produces both answers at once.

5.5 Experimental Setup

5.5.1 Data

Table 8 summarizes our six datasets. Five come from Gowalla, a location-based social network, covering the U.S. states of Alabama, Arizona, Florida, Texas, and California [12]; the sixth is Istanbul, from the Massive-STEPS collection [62], a non-U.S. check on the findings. The states range from about 114,000 check-ins and 1,109 regions in Alabama to about 3.2 million check-ins and 8,501 regions in California. Every dataset uses the seven place categories of Section 5.3; Istanbul's source collection maps its own labels onto them. The region unit is a census tract for the Gowalla states and a mahalle for Istanbul.

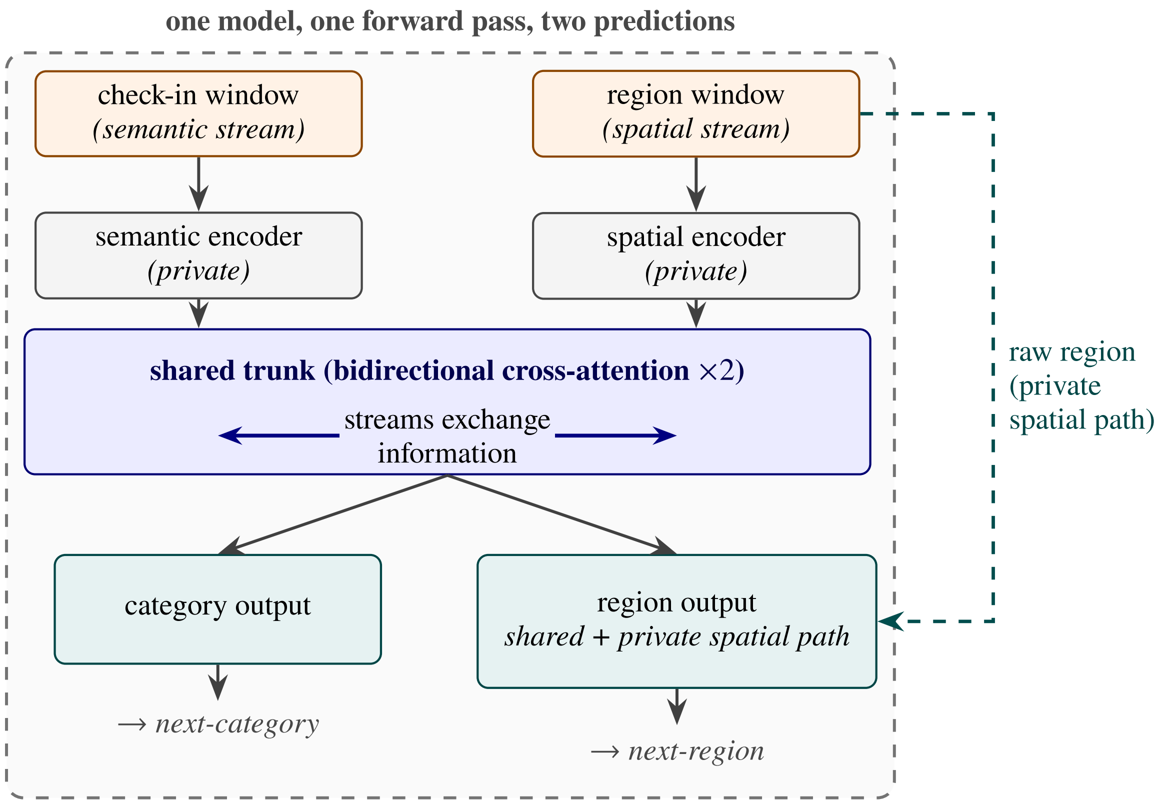

Figure 5 – The single multi-task model. A check-in (semantic) window and a region (spatial) window pass through private encoders into the shared trunk, the only place where the two tasks interact. The category output uses the trunk's output alone; the region output combines it with a private spatial path. One model, one forward pass, two predictions.

Table 8 – Dataset statistics, ordered by region count: five Gowalla states and Istanbul (Massive-STEPS). All share the same seven root categories; the region unit is a census tract (a mahalle for Istanbul). Windows: sliding nine-visit inputs plus the next-visit target; lengths: per-user check-in counts; density: 100 × check-ins/(users × POIs); Majority: share of next-visit labels in the most common category (Food in every dataset).

Dataset	**Source**	**Check-ins**	**Users**	**POIs**	**Regions**	**Windows**	**Max len**	**Avg len**	**Density (%)**	**Majority (%)**
Istanbul	Massive-STEPS	462,615	23,694	29,816	520	271,666	817	19.5	0.065	33.4
AL	Gowalla	113,846	3,858	11,848	1,109	96,326	3,835	29.5	0.249	34.2
AZ	Gowalla	236,450	7,869	20,666	1,547	200,895	5,589	30.1	0.145	34.0
FL	Gowalla	1,407,034	21,052	76,544	4,703	1,274,418	16,679	66.8	0.087	24.7
TX	Gowalla	4,089,892	38,644	160,938	6,553	3,830,414	42,300	105.8	0.066	31.0
CA	Gowalla	3,171,380	37,090	169,145	8,501	2,925,466	14,855	85.5	0.051	32.7

5.5.2 Windows, splitting, and the integrity of the representation

Windows. For each user with at least ten visits, we build time-ordered overlapping sliding windows of nine visits, one starting at each visit, with the next visit as the target, giving both tasks more examples. Near the end of a user's history, every start position within the last nine visits yields a shorter, padded window whose target is the same final visit; we keep only the full-length window ending there and drop these padded duplicates. The resulting prediction horizon is short but heavy-tailed: the median time from the last visit in a window to its target ranges from 0.4 hours in Florida to 5.5 hours in Istanbul, while 5 to 27 percent of targets lie over 3 days ahead.

Splitting. We split by user with stratified five-fold cross-validation, so all of a user's windows fall in the same fold and overlap cannot leak: a test user's visits never appear in training. The held-out fold is the validation data; we reserve no third split. The split is stratified on the next-category label because the seven-class task is imbalanced: Food is the majority class in every dataset, from about 25 percent of visits in Florida to 34 percent in Alabama (Table 8).

Integrity of the representation. We train the representation once on the whole dataset and feed it to every model, dedicated and joint. A representation built this way could pass information about a test visit along more than one channel. We therefore bound the channels that we can measure, and state for each what the measurement covers, on four grounds.

First, its training objective is label-free: it contrasts real graph neighborhoods against shuffled ones and never sees a next-category or next-region target. That bounds the training signal and not the inputs, since each visit's own category enters as a node feature (Section 5.4.1); the fourth ground below measures what that feature can carry between visits.

Second, we measured the transductive channel, what a graph fitted over all users passes to the test side that a graph fitted over training users alone would not. Rebuilding the representation per fold, from that fold's training users only, moves both tasks by at most a third of a point (region −0.33 to +0.01; category 0.00 to +0.29, at Alabama, Arizona, and Florida), within fold noise. This measurement covers the visits whose places appear in training (67 to 87 percent); visits to places unseen in training are the one part it cannot reach.

Third, the one component that could pass information between visits, a region-transition prior (a table of how often one region follows another), is built per fold from training data only, after an earlier whole-dataset version inflated region accuracy by 13 to 27 points. Our joint and dedicated models do not use this prior; it appears only in the HMT-GRN baseline (Section 5.5.4).

Fourth, we probe the channel that the graph's own links between consecutive visits open. A visit's node is linked to the visit that follows it, and category is a node input feature, so a per-visit vector could in principle absorb the category of the next visit. A screening audit run during development tests exactly this: it predicts the next category from the last window slot alone. The audit reads each encoder against a clean reference encoder rather than against the category labels directly. At Florida the graph encoder of our own lineage is that reference, at 0.4090 macro-F1 on standardized features and 0.4074 on raw ones, on a zero-to-one scale, and the residual variant that the encoder we ship descends from sits at the same level, 0.4197 and 0.4182; an attention-based encoder screened beside them reached 0.4976 and 0.4863, clear of the reference by more than the screen's margin, and was disqualified on that evidence.

A second reference can be computed directly from the category sequence, with no encoder involved. We call it the label-history benchmark: the best macro-F1 reached by four specified predictors that read only the genuine category history of the input window (Appendix D of the supplementary volume of this dissertation). It is not an upper bound on what a model may score. The four predictors are a specified set, and a better predictor of the same restricted information could exceed them. The benchmark lies below the clean reference: 0.3617 at Florida, and 0.2800 to 0.3617 across the five datasets where the inputs are available. The clean references therefore lie above the label-history benchmark, by four to six points at Florida, and so does every other candidate screened beside them on the raw run of the probe. One candidate falls below the benchmark on the standardized run and above it on the raw one; Appendix D of the supplementary volume of this dissertation names it and explains why it is not a counter-example.

The measurement bounds this channel rather than closing it, and three limits set how far it reaches: the probe is linear, it was run at Florida alone at one random initialization over five user-grouped folds, and it was run on those ancestor builds of the representation rather than on the one that produced the results reported here. The same record shows why the linear form is a screen and not a proof, since one encoder that passed it leaked under a downstream sequence model. The baseline representations meet the same standard: HGI is pre-trained once on the whole dataset, like ours, and CTLE per fold on training users only.

5.5.3 Metrics and statistical tests

For the category task, we report macro-averaged F1 (macro-F1), which averages the per-class F1 so each of the seven categories counts equally; its reference point is the majority-class floor, the macro-F1 of always predicting the most common category. For the region task, we report accuracy at ten (Acc@10), the fraction of test visits whose true region appears among the model's ten highest-scoring guesses (a visit whose true region is absent from that fold's training data counts as an error); its reference point is the dedicated model.

A claimed gain and a claimed match require different tests, because a difference that fails to reach significance is not by itself evidence of a match. A written analysis plan, fixed during development and before any result was read, assigned one test to each task, with the two-point margin pinned there: *superiority* for next-category, *non-inferiority* for next-region. It assigned the tests per task, not per dataset, and did not cover next-region superiority, so the four next-region gains of Section 5.6.2 are secondary results outside it. The plan registered a paired Wilcoxon signed-rank test; at four seeds its exact one-sided p cannot fall below 0.0625, so we report a paired t on the per-seed means, with the 90% confidence interval of the paired difference, and the registered test alongside it. Both, and this departure, are released with the code. A *seed* is one complete repetition of the five-fold experiment, over the same folds, with a different random initialization; folds and seeds are the two axes of repetition. On every dataset, both models use four seeds ($4 \times 5 = 20$ measurements) and the tests pair the per-seed means (n=4), with a Holm correction [69] across the six next-category comparisons and, separately, across the four next-region ones. The *non-inferiority* claim is that the joint model is no worse than the dedicated model by more than a two-point margin; we test it with the two one-sided tests (TOST) procedure [70]. The two-point margin is fixed in advance as well: a mobility-aware service acts on which region will be busy, not on a single rank position (Section 5.3), so a two-point shift in Acc@10 is below the granularity at which such a service would behave differently. The precision of the equivalence test is measured on this fixed partition: the paired difference's standard deviation is 0.01 to 0.18 points across the datasets, and the intervals pass a margin as small as one point at Alabama and Arizona (Section 5.6.2).

5.5.4 Baselines

The comparisons play three roles. The first role is played by the per-task state of the art, on our data, under user-disjoint five-fold protocols. For next-category, this is POI-RGNN [24], re-implemented from its published architecture and hyperparameters, and a Markov baseline that predicts the category that most often follows the recent ones (Markov-K, best order per dataset); for region, we also compute a first-order Markov floor over region transitions, under the same sliding-window protocol and fold splits as our models. For next-region, the primary comparison is HMT-GRN [21], an end-to-end recurrent model (trained as one piece from the raw check-ins) evaluated on the same data, folds, and initialization. We keep its shared multi-task skeleton and add a region-transition prior built per fold from training data. We drop its graph components and hierarchical beam search, which exist to serve its next-place target, a target that we do not predict. The result is a region-native model (region is one of its own prediction targets), not a reproduction of the complete published system. We also run STAN [17], re-implemented from its published architecture and trained from the raw check-ins with its own embeddings and sequence construction under one fixed configuration. We adapt its output layer to rank region candidates, spatial objects that still support its candidate-distance matching; we do not adapt it to category, where distance matching has no meaning. A ReHDM [98] reference is reported under its own published protocol.

The second role is played by two representation controls that attribute the category gain to our specific design, not to contextualization in general or to extra features: CTLE [20], the closest prior contextual check-in embedding, run end to end at Florida and with frozen weights (no fine-tuning) at Alabama, Arizona, and Istanbul; and a feature-concatenation control appending raw per-visit features to a standard place embedding under the same model.

The third role is a comparison between the two ways of coupling the tasks. The dominant published coupling is the directed cascade, predicting the category first and conditioning the place prediction on it [97, 22, 61]. We test the cascade inside our own model rather than re-implementing those systems: we remove the shared trunk, where the two streams exchange information, and feed the predicted category forward into the region pathway, keeping the representation, outputs, and training configuration unchanged. This isolates the coupling topology (chain versus parallel) at no additional parameter or compute cost: the chain form drops the shared trunk and adds a conditioning projection of about 1,000 parameters. A final control, the standard place-level embedding [34], isolates what the per-visit representation adds. All other choices, including the nine-visit window, were fixed during development; the full training configuration for every model is in the released code (footnote 1).

5.6 Results

5.6.1 The representation makes the category task learnable

The check-in-level representation outperforms a standard place embedding (HGI [34]) on next-category macro-F1 by a wide gap on every dataset (Table 9). The comparison is controlled: target, single-task model, training configuration, windowing, and folds are identical; only the input representation changes (a vector per visit versus a vector per place), so the difference isolates the representation. The check-in-level column keeps one fixed configuration, not the per-dataset-tuned dedicated model of Table 10. The gaps range from +27.63 (Arizona) to +39.62 (Florida), with Istanbul at +28.09. A controlled test isolates this: averaging the per-visit vectors into one vector per place removes roughly 64 to 90 percent of the gain (state-dependent), so most of the gain is the context that each visit carries, not extra training signal (more training vectors per place).

The geometry of the vectors shows why the representation helps this task in particular: the per-visit vectors' silhouette score by category (how tight and well separated the seven labeled groups are, on a -1 to 1 scale) is about 0.57 against about 0.00 for the place embedding, and their nearest-neighbor category purity (the share of nearest neighbors with the vector's own category) is about 0.98 against about 0.78 (both averaged over the five U.S. states). The same geometry does not separate regions, so the benefit is category-only (the joint model's spatial stream instead reads the region-level vectors of the same graph, Section 5.4.2). CTLE [20], the closest prior contextual embedding, also gives each visit its own vector, but it pretrains on place identifiers and timestamps alone, so the category vocabulary never enters its training, whereas our graph reads each visit's category and time of day as input features. At Florida, we fine-tune CTLE together with the task model, using its authors' defaults and the same 64 dimensions and windowing as ours. It reaches 33.45 macro-F1 at its best epoch and 29.69 at its final one (an epoch is one pass over the training data), about two points below the place embedding under the same best-epoch rule, and far below the check-in-level representation's 75.15. With frozen weights, fed to the same single-task model, CTLE repeats the ordering at Alabama, Arizona, and Istanbul, with our representation ahead by +37.8, +37.0, and +28.7 macro-F1. A feature-concatenation control joins the place embedding with the same raw per-visit features that our graph reads (the category one-hot plus hour and day-of-week encodings) and feeds the result to the same single-task model. It does not close the gap either: at Alabama, Arizona, and Florida it raises the place embedding by only +2.0, +1.7, and +0.8 macro-F1, under a tenth of the place-to-check-in gap at each state. The gain therefore comes from the hierarchical per-visit representation, not from contextualization alone or feature injection.

Table 9 – Next-category macro-F1: the check-in-level representation against a standard place embedding (HGI), same single-task model, training configuration, and folds (mean and fold sd, seed 0).

Dataset	**Check-in level**	**Place level**	**Δ**
Istanbul	**54.65** ±0.6	26.56 ±0.5	+28.09
AL	**55.87** ±2.4	26.56 ±1.0	+29.31
AZ	**57.13** ±1.3	29.50 ±1.0	+27.63
FL	**75.15** ±0.5	35.53 ±0.3	+39.62
TX	**69.95** ±0.2	32.48 ±0.6	+37.47
CA	**70.26** ±0.3	32.31 ±0.4	+37.95

The matching place-level value at Alabama and Istanbul (26.56) is a coincidence of two independent runs; their per-fold values differ.

Figure 6 shows the same separability contrast graphically.

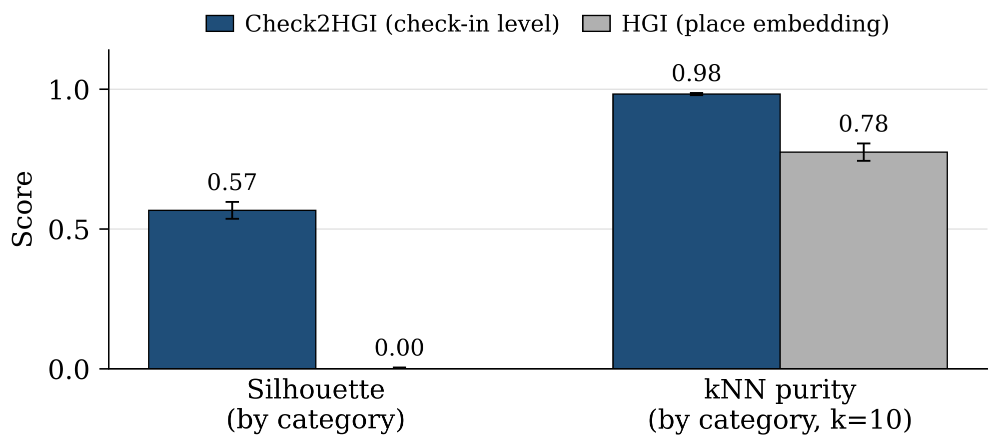

Figure 6 – Class separability of the representations, grouping each vector by its ground-truth category label (cosine silhouette; leave-one-out nearest-neighbor (kNN) purity, k=10), averaged over the five U.S. states. The check-in-level representation separates the seven categories; the place embedding does not.

5.6.2 One model, two tasks

One model outperforms or matches the dedicated models on both tasks. For scale: always predicting the most common category reaches only about 7 percent macro-F1 (the majority-class floor), and a random region top-ten guess is right at most about two percent of the time (Section 5.5.3).

Table 10 reports the single joint model against the dedicated single-task ceilings (the level a joint model is usually expected, at best, to match). The dedicated category model is tuned per dataset over batch size and learning rate (the dedicated region models use the strongest fixed configuration). Throughout, every reported model is one saved artifact per fold, read at its validation-selected epoch: each dedicated model at its task's best epoch, and the joint model at the epoch selected by its joint validation score (the geometric mean of the two task metrics), with both tasks read from that one model. Reading each joint task at its own best epoch instead would change every joint result by at most 0.06 (category) and 0.11 (region) points.

Table 10 – One model, two tasks: the single joint model against the dedicated single-task models and the external baselines, ordered by region count. The upper block reports next-category (macro-F1) and the lower block next-region (Acc@10); the two share the dataset and region-count columns, and the same six datasets appear in the same order in both. Bold marks a statistically supported improvement over the dedicated model; in the region block, ↑ marks that supported improvement and ≈ a non-inferior match (TOST, ±2 pp; Section 5.6.2).

		Next-category (macro-F1)			
Dataset	**Regions**	**Markov-K**	**POI-RGNN**	**Dedicated**	**Joint (ours)**
Istanbul	520	24.55	30.12	54.74 ±0.09	**63.32** ±0.02
AL	1,109	20.50	23.80	56.82 ±0.03	**64.51** ±0.09
AZ	1,547	23.92	27.64	56.43 ±0.10	**65.79** ±0.02
FL	4,703	29.74	34.49	74.51 ±0.03	**79.84** ±0.02
TX	6,553	28.67	33.03	69.79 ±0.08	**77.24** ±0.01
CA	8,501	27.58	31.78	70.60 ±0.07	**77.05** ±0.01

		Next-region (Acc@10)				
Dataset	**Regions**	**HMT-GRN**	**ReHDM**	**STAN**	**Dedicated**	**Joint (ours)**
Istanbul	520	60.4	69.33	61.86	75.16 ±0.01	**75.35** ±0.04↑
AL	1,109	57.05	65.38	60.72	70.11 ±0.10	69.70 ±0.09≈
AZ	1,547	43.70	53.00	49.86	59.46 ±0.08	59.46 ±0.04≈
FL	4,703	63.74	64.49	72.99	76.70 ±0.02	**77.41** ±0.02↑
TX	6,553	53.85	48.81‡	61.67†	64.95 ±0.01	**67.06** ±0.01↑
CA	8,501	49.61	50.26‡	58.52†	63.49 ±0.02	**65.69** ±0.02↑

HMT-GRN (region-native, on our folds, scored on visits whose region appears in training, >99% of test visits in every dataset and fold) is the primary external comparison; STAN (our re-implementation) and ReHDM (its own protocol) are references. Markov-K: strongest order per dataset. †STAN partial folds: TX 4/5, CA 2/5 (seed 0). ‡ReHDM at TX and CA: a single seed. Joint and dedicated entries: four seeds × five folds; ±: sd across seeds.

Figure 7 plots the signed gains ordered by region count. On category, the joint model outperforms the dedicated ceiling on every dataset, by +5.33 to +9.35 macro-F1 (smallest at Florida, largest at Arizona, and +8.58 at Istanbul). On region (Acc@10), the joint model outperforms the dedicated ceiling at Florida, Texas, California, and Istanbul, and stays a non-inferior match (TOST, ±2 pp) at Alabama and Arizona. Across the five U.S. states, the region gain rises with region count, from −0.41 at the smallest count to +2.20 at the largest; region count and corpus size co-vary here, so we read the trend across the points rather than as a precise law.

On category, all four seeds favor the joint model on every dataset, and each gain is significant after a Holm correction across the six datasets (paired t, corrected $p < 0.001$); the four next-region gains hold under their own Holm correction as well (corrected $p < 0.001$). The registered Wilcoxon test on the individual fold differences agrees at every dataset, with all 20 folds favoring the joint model (corrected $p < 0.001$). On region, Alabama and Arizona are tested for equivalence (TOST, ±2 pp) and pass with 90% confidence intervals well inside two points (each entry: point estimate; interval): Alabama (−0.41; −0.63 to −0.20) and Arizona (0.00; −0.08 to +0.07). At Arizona, the interval is centered on zero, so we report a match, not a gain. At Alabama, the whole interval lies below zero, a small but statistically significant deficit, still well within the two-point margin. At Florida and Istanbul, the 90% intervals lie entirely above zero (Florida +0.71; +0.67 to +0.76; Istanbul +0.19; +0.15 to +0.23), so the joint model outperforms there, though both gains are smaller than the two-point margin of Section 5.5.3. Texas and California exceed that margin: their 90% intervals lie entirely above it (+2.10 to +2.13 at Texas, +2.19 to +2.21 at California).

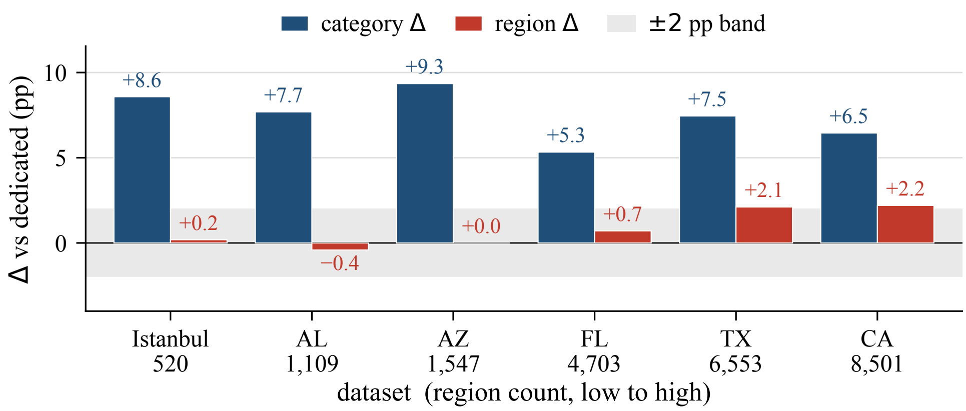

Figure 7 – Signed gains of the joint model over the dedicated single-task models, ordered by region count. The category gain (macro-F1) is positive at every dataset; the region gain (Acc@10) rises across the five U.S. states and is also positive at Istanbul. The ±2-point band applies to region only.

A reader used to multi-task learning [7] expects the harder task, here next-region with its hundreds to several thousand classes, to teach the easier one; a control shows otherwise. We fix the region pathway at its initial values at the start of training so it can neither learn nor teach the category task, yet the full category gain survives at Alabama, Arizona, and Florida: the control reaches 63.50, 63.67, and 79.79 macro-F1, above the single-task category score of the one fixed configuration (Table 9) by +7.63, +6.54, and +4.64 points. That comparison is the one the control was built to make, and it is the one to read. The control predates the results of Table 10 and was measured at one random initialization over five folds, so its second comparison is to the joint scores of the development configuration current at the time (63.56, 63.39, 79.82), which it matched to within 0.3, and not to the joint entries reported here. The control therefore rules out one reading and narrows the rest. It rules out the region task teaching the category one, since the gain survives with the region pathway untrained. What it leaves is the joint architecture itself, and the control does not say which part of it: freezing the region stream removes region training, not the category stream's own encoder, the per-stream feed-forward blocks, or the added depth. We therefore report the negative result, that the gain is not cross-task transfer, as a finding, and we attribute the gain to the joint architecture rather than to any named component of it. A separate ablation in our own development record, at Florida only, removed the cross-attention stack while holding the rest fixed and moved next-category macro-F1 by -0.04 ± 0.13, which a paired test cannot separate from zero. Two limits keep us from reading that as an absence of contribution. The ablation ran on an earlier configuration whose region output was driven by a region-transition prior the models reported here do not use, and the development record that produced it reads the null as a compensation effect, in which the category stream absorbs what the shared stack would otherwise supply, rather than as a component that does nothing. We therefore do not name the shared trunk as the source, and we do not present the ablation as evidence against it either.

Either way the gain requires no second model at serving time (one model, one forward pass).

The joint model is also above every external baseline reported, on both tasks, across all six datasets (Table 10): on region, the primary region-native comparison HMT-GRN [21], STAN [17] trained from the raw check-ins, and a ReHDM reference [98]; on category, POI-RGNN [24] and the Markov-K floor. These externals run on their own embeddings, so this comparison also includes the representation advantage of Section 5.6.1; the controlled comparison for the joint model remains the Dedicated column of Table 10, which uses the same representation, windows, and folds. The first-order Markov region floor of Section 5.5.4, computed under our windows and folds, reaches 51 to 72 Acc@10 across the datasets; the joint model exceeds it by 4.9 to 10.3 points on all six datasets.

That floor is also above the three external region systems at most datasets, and the comparison deserves a direct word rather than being left for the reader to assemble. HMT-GRN falls below it at all six, the ReHDM reference at three, and STAN at four. Two facts about how these numbers were produced bear on that comparison. The first concerns the floor: it is computed under our own sliding-window protocol and fold splits. Those windows advance one visit at a time, so the region of the last visit is a strong predictor of the next one, and a first-order transition table reads exactly that signal. At Alabama the target region is the last visited region in 32.9 percent of windows. The second fact concerns the three systems, which do not meet the floor on equal terms. HMT-GRN is evaluated on the same data, folds, and initialization as our models. STAN runs on the same folds, but builds its own embeddings and its own sequences from the raw check-ins. The ReHDM reference runs under its own published protocol, so its result is not measured on our windows or folds at all.

Neither fact establishes why the floor lies above the three systems, and we do not claim a single explanation. We treat the floor, not the external systems, as the reference the region task has to clear, and we read the external columns as a comparison against published designs rather than as the standard for this task. The dedicated and joint models are above the floor at every dataset.

We prefer coupling the tasks in parallel rather than in a chain: neither target is subordinate to the other, so a chain must pick one to predict first and pass its errors to the other. The choice still had to be checked, because CSLSL reports its chain outperforming a shared-trunk parallel variant on its own benchmarks [61]. Rewired as a cascade (Section 5.5.4), our model reaches the same combined score, the geometric mean of the two task metrics, as its parallel form: at Alabama, Arizona, Florida, and Istanbul, with both forms trained under the same configuration and initialization, the difference is about zero and neither task moves by more than a quarter of a point. We read this as a defense of the parallel design, not a claim that we outperform the cascade: on this representation the coupling topology makes no measurable difference, and the simpler parallel structure loses nothing. Two qualifications bound this reading: the cascade runs under the configuration tuned for the parallel model, and its form was fixed in advance, so tuning the cascade itself remains future work.

5.6.3 External validity (Istanbul)

The joint-model results hold for a non-U.S. city under the same representation, sliding-window protocol, and training setup as the U.S. states. At Istanbul, the joint model outperforms the dedicated category ceiling by +8.58 macro-F1 (dedicated 54.74, joint 63.32) and is slightly above the dedicated region ceiling at +0.19 Acc@10 (dedicated 75.16, joint 75.35), a gain with statistical support (the whole 90% confidence interval of the paired difference lies above zero, and TOST, ±2 pp, also passes). The comparable quantity is the gain over the ceiling, not the absolute Acc@10, since region counts differ across datasets (520 mahalle here). The external baselines show the same ordering on both tasks (Table 10). The U.S. result repeats on a different continent and region unit.

5.7 Discussion and Limitations

One model serves both tasks. In one forward pass it outperforms the dedicated category model at all six datasets, and on region it outperforms the dedicated model at four of them and matches it within a two-point margin at the other two, with the region output keeping a private spatial path (Table 10; Figure 7). Which part of the joint architecture produces the category gain is not settled by the controls reported here (Section 5.6.2).

A service could treat the model's top-ranked next regions as an anticipatory set: at California, ten regions out of 8,501 contain the true next region 65.69 percent of the time, over 500 times better than picking ten at random. On four datasets (Alabama, Arizona, Florida, and Istanbul; a single seed over five folds), the ten shortlisted regions lie a median of 3 to 8 kilometers from the shortlist's centroid, against 17 to 176 kilometers for ten regions drawn at random from the same candidate set (median over 10,000 draws). This remains motivation, not a measured service result.

Three limits qualify these results. First, our representation is trained once over all places; visits to places never seen during training are the single effect that we cannot fully isolate. A planned follow-up trains it on each fold's training places only and embeds unseen visits. Second, epoch selection consults the fold that the score is then read on (Section 5.5.2), so every absolute score reported here is optimistic. The comparison between the joint model and the dedicated models is affected far less, for two reasons we can state outright: the selection rule is the same for both models on the same folds, an epoch chosen on validation and read on that fold, with each model selected on its own validation objective (Section 5.6.2), and the dedicated model receives the wider search, a per-dataset sweep over batch size and learning rate against one configuration held fixed across all six datasets for the joint model. The residual therefore favors the comparator, which makes the reported difference conservative. It does not follow that the bias cancels exactly. The four seeds also reuse one fixed fold partition, so the reported intervals cover variation across random initializations and not across resampled user splits. Third, we do not build or evaluate a mobility-aware service; it is background motivation, and this chapter's claims are the prediction results themselves.

Our representation is a fixed per-visit vector that any downstream model can consume. Moura et al. [93], whose structural analysis of tourist check-ins reads past visits only, name machine learning as a next step; this study moves in that direction.

5.8 Conclusion

We tested whether one model can predict both the category and region of a user's next visit. A check-in-level representation, one vector per visit rather than one per place, makes the next-category task far more learnable than the standard place-level representation does. On that representation, a single multi-task model outperforms a dedicated category model on every dataset. On region, it outperforms the dedicated model on four of the six datasets and remains non-inferior (TOST, ±2 pp) on the other two. The gains at Texas and California exceed the two-point margin; those at Florida and Istanbul are statistically supported but smaller. Across the U.S. states, the region gain rises with region count. The joint model also remains above every external baseline in Table 10 on all six datasets, by at least 4 Acc@10 points over the strongest region reference (HMT-GRN, STAN, or ReHDM; the first two on our folds, ReHDM under its own protocol) and by at least 33 macro-F1 points over POI-RGNN on category. One model with a shared semantic context and a private spatial path anticipates both what a user will do next and where.

6 Conclusion

This dissertation asked one question, whether multi-task learning helps point-of-interest prediction for the next category and next region tasks and what the answer depends on, and answered it across three studies: a joint model that did not outperform its dedicated counterparts (Chapter 3), the diagnosis that located the bottleneck in the input representation (Chapter 4), and a check-in-level representation with a redesigned sharing topology under which one joint model finally outperforms the dedicated single-task models on the category task at all six datasets and on the region task at four of the six, and is statistically non-inferior to them, within a two-point margin (TOST), at the other two (Chapter 5).

6.1 Contributions by chapter

Chapter 3 contributed MTLnet, the first joint model of this research, and an honest negative result. Built from standard components, a place-level graph embedding and hard parameter sharing beneath the two task heads, the joint model did not consistently outperform the dedicated single-task models on the two category tasks, and it cost more to train. The study's contribution is not the architecture but the finding and its analysis: three candidate explanations, task dissimilarity, an input representation too poor for both tasks at once, and the restrictiveness of hard sharing, that the rest of the dissertation put to the test. That conclusion held for the configuration of its time, a place-level input under hard sharing, and the later chapters revised it.

Chapter 4 contributed the controlled test of the representation explanation. Holding the MTLnet architecture fixed and replacing only the input, the monolithic 64-dimensional place embedding, with decomposed spatial, temporal, and categorical encoders, raised category macro-F1 by 20.2 to 22.0 percentage points across the three states tested. A change of input, with no change of architecture, produced a larger gain than any architectural variation tried before it. Two qualifications bound what that number licenses. It is measured on the static task, which classifies a place from that place's own representation, and Appendix B of the supplementary volume of this dissertation records that this task's input determines its target by construction, so the figure is not evidence about the sequential task. The comparison is also not width-matched: the decomposed input is wider than the place embedding it replaces, 192 dimensions against 64, and Chapter 4 states that an equal-dimension control would be needed to separate the semantic specialization of the encoders from the increase in input dimensionality alone. The diagnosis of the arc follows from this chapter: the input representation, not the sharing scheme, was the binding constraint at that stage of the research. What carries that diagnosis is the direction and the size of the effect on the sequential task, where no such identity between input and target exists, and Chapter 5 is what tests it.

Chapter 5 contributed the resolution: the check-in-level representation, which gives each visit its own vector instead of assigning every visit to a place the same one, and a joint model that shares through a cross-attention trunk between two task-specific streams, with a private spatial path for the region task. Under user-disjoint cross-validation with twenty fitted models per configuration, four seeds over one fixed set of five folds, and paired tests on the four per-seed means, the joint model outperforms the dedicated single-task models on the category task at all six datasets, by 5.3 to 9.4 macro-F1 points, and on the region task at four of six, Istanbul, Florida, California, and Texas, while remaining statistically non-inferior within a two-point margin (TOST) at Alabama and Arizona.

6.2 The consolidated answer

Does multi-task learning help point-of-interest prediction? The dissertation's answer is conditional, and the condition is the finding. With a place-level embedding and naive hard sharing, no: the joint model of Chapter 3 did not consistently outperform two dedicated models, and negative transfer between the static and the sequential task was among its candidate causes. With a check-in-level representation and a sharing topology built for it, yes: one model, one forward pass, both predictions, at quality that outperforms the dedicated models on the category task everywhere and outperforms or matches them on the region task. The representation, together with the sharing topology built on it, is what the answer depends on.

Two controls separate this claim from wishful attribution. First, the freeze control reported in Chapter 5: with the region pathway frozen, the category gain survives, at the three datasets where the control was run (Alabama, Arizona, Florida), so the gain does not come from the region task teaching the category task; it comes from something in the joint architecture that the dedicated model does not have. The control does not say which part. A development ablation at Florida removed the cross-attention stack alone and moved next-category macro-F1 by -0.04 ± 0.13, but that measurement was taken on an earlier configuration whose region head was driven by a transition prior the models reported here do not use, and its own record reads the null as a compensation effect rather than an absence of contribution. We therefore do not name the shared trunk as the source, and we do not offer the ablation as evidence that the trunk contributes nothing. Second, a capacity-matched dedicated baseline, run after the Chapter 5 manuscript was submitted and reported here as a frame-level analysis: a dedicated category model widened to the joint model's parameter budget, about 4.2 million parameters at Alabama against 0.6 million at its published width, does not recover the joint gain. At Alabama the widened model was fitted under three training configurations, twenty models each and sixty in total, and the strongest configuration averages 56.16 macro-F1, standard deviation 1.89, against 56.82 for the dedicated model at its own tuned width and 64.51 for the joint model. The California run, completed since, repeats the pattern. At the matching width, a hidden dimension of 752 and 101.9 percent of the joint model's parameter count, the widened model's better training configuration reaches 69.88 macro-F1, standard deviation 0.26 over its twenty fitted models, against 70.60, standard deviation 0.07, for the same model at its own tuned narrow width. At both datasets, then, widening leaves the dedicated model at or slightly below its narrow-width optimum, 0.72 points below at California and 0.66 at Alabama, and recovers none of the joint model's margin. One detail supports the fairness of the comparison: at both datasets the widened model's best training configuration uses a lower learning rate than the narrow width's, so the sweep found the wide model's own better setting rather than rerunning the narrow model's. Parameter count alone, without the second task's training signal, yields nothing here; the two experiments together license the sentence Chapter 5 already states as its finding, that the gain is not cross-task transfer.

The gradient-level picture is consistent with this reading, within its measurement scope. During development, on an earlier preparation of the data, the cosine similarity between the two tasks' gradients averaged +0.001 over four seeds on four Gowalla states, three of which are among the five we report, directional conflict only, a finding for this pair of tasks rather than a general rule. Under the check-in-level representation the two sequential tasks coexist with essentially orthogonal gradients: sharing stopped hurting. This also explains why gradient-balancing optimizers had little to correct, in this configuration, and why a tuned fixed weighting sufficed.

The negative result of Chapter 3 and the positive result of Chapter 5 do not contradict each other; read together, they bound the claim. Multi-task learning is not a free gain that any architecture collects, and it is not a dead end that no representation can rescue. What this dissertation adds to the record is one worked path between those poles, with the decisive variable named and tested.

6.3 Limitations

Six limitations bound the scope of these conclusions.

1. **Data vintage.** The five state datasets come from Gowalla. Its authors report collecting public check-ins between February 2009 and October 2010 [12], while the extraction used here spans January 2009 to August 2011 across the five states; mobility patterns, place inventories, and check-in behavior have changed since either date.

2. **Taxonomy coarseness.** The category task uses seven top-level classes; finer-grained taxonomies may behave differently under joint training.

3. **Transductive representation.** The check-in-level representation is trained on the check-in graph of each dataset; it does not embed unseen places or users without retraining.

4. **No next-place task.** The exact next place is not predicted anywhere in this work; the conclusions concern the next category and next region targets only.

5. **Geographic coverage.** Outside the United States the evidence rests on a single city, Istanbul.

6. **The task-pair confound.** The task pair changed together with the representation and the sharing topology: Chapters 3 and 4 paired static category classification with next category, while Chapter 5 pairs next category with next region, two sequential tasks. No single controlled ablation separates the representation-and-topology change from the task-pair change in the final result. Chapter 4 is the fixed-pair control for the diagnosis, and the capacity-matched baseline above closes the parameter-count explanation in the setting it tested, the category task at two of the six datasets, one width point per dataset, and width scaling rather than depth, but the homogeneity of the final pair remains a possible contributor to the size of the improvement.

6.4 Future work

Each limitation names its own next step. Extending the pipeline to newer and denser traces would test the conclusions beyond the Gowalla vintage (limitation 1), and finer-grained taxonomies would test them below the seven-class ceiling (limitation 2). An inductive variant of the check-in-level representation, one that embeds unseen places and users without retraining, would remove the transductive constraint and open deployment to growing cities (limitation 3). Adding the exact next place as a third target, on top of the two properties this work predicts, would reconnect the joint model to the field's hardest task, where the cascade architectures reviewed in Chapter 5 suggest category and region predictions can serve as structure rather than competition (limitation 4). Evaluation on further cities outside the United States would widen the geographic base (limitation 5). And the fixed-pair ablation, training the Chapter 5 joint model on the Chapter 3 task pair under the check-in-level representation, is the experiment that would close the task-pair confound (limitation 6).

6.5 Final remarks

The practical output of this dissertation is one deployable model whose single forward pass answers two questions a mobility-aware service asks about the next visit, what kind of place and in which part of the city. The methodological output is an honest record: a published negative result, the diagnosis that turned it into a question about representations, and the resolution that answered the question. The negative result was not an obstacle on the way to the contribution; worked through, it was the contribution's first half.

References

[1] SILVA, V. H. O. et al. An investigation into multi-task learning for point-of-interest category classification and next-POI prediction. In: **Anais do XVII Congresso Brasileiro de Inteligência Computacional (CBIC 2025)**. [S.l.: s.n.], 2025. p. 1–8.

[2] SONG, C. et al. Limits of predictability in human mobility. **Science**, v. 327, n. 5968, p. 1018–1021, 2010.

[3] LUCA, M. et al. A survey on deep learning for human mobility. **ACM Comput. Surv.**, v. 55, n. 1, p. 7:1–7:44, 2021.

[4] XU, R. et al. Tme: Tree-guided multi-task embedding learning towards semantic venue annotation. **ACM Trans. Inf. Syst.**, v. 41, n. 4, p. 112, 2023. ISSN 1046-8188.

[5] MAI, G. et al. Sphere2vec: A general-purpose location representation learning over a spherical surface for large-scale geospatial predictions. **ISPRS J. Photogramm. Remote Sens.**, v. 202, p. 439–462, 2023. ArXiv:2306.17624.

[6] WU, N. et al. Torchspatial: A location encoding framework and benchmark for spatial representation learning. **Advances in Neural Information Processing Systems**, v. 37, p. 81437–81460, 2024. ArXiv:2406.15658.

[7] CARUANA, R. Multitask learning. **Mach. Learn.**, v. 28, n. 1, p. 41–75, 1997.

[8] KOKKINOS, I. Ubernet: Training a ‘universal’ convolutional neural network for low-, mid-, and high-level vision using diverse datasets and limited memory. In: **Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)**. [S.l.: s.n.], 2017. p. 5454–5463. ArXiv:1609.02132.

[9] LIPTON, Z. C. et al. Learning to diagnose with lstm recurrent neural networks. **arXiv preprint arXiv:1511.03677**, 2015.

[10] WEI, J. et al. Finetuned language models are zero-shot learners. In: **Proceedings of the 10th International Conference on Learning Representations (ICLR)**. [s.n.], 2022. Disponível em: <https://openreview.net/forum?id=gEZrGCozdqR>.

[11] SILVA, T. H. et al. Urban computing leveraging location-based social network data: A survey. **ACM Comput. Surv.**, v. 52, n. 1, p. 17:1–17:39, 2019.

[12] CHO, E.; MYERS, S. A.; LESKOVEC, J. Friendship and mobility: User movement in location-based social networks. In: **Proc. ACM SIGKDD (KDD)**. [S.l.]: ACM, 2011. p. 1082–1090.

[13] LIU, Q. et al. Predicting the next location: A recurrent model with spatial and temporal contexts. In: **Proc. AAAI Conf. Artif. Intell.** [S.l.: s.n.], 2016. p. 194–200.

[14] FENG, J. et al. DeepMove: Predicting human mobility with attentional recurrent networks. In: **Proc. Web Conf. (WWW)**. [S.l.: s.n.], 2018. p. 1459–1468.

[15] KONG, D.; WU, F. HST-LSTM: A hierarchical spatial-temporal long-short term memory network for location prediction. In: **Proc. IJCAI**. [S.l.: s.n.], 2018. p. 2341–2347.

[16] YANG, D. et al. Location prediction over sparse user mobility traces using RNNs: Flashback in hidden states! In: **Proc. IJCAI**. [S.l.: s.n.], 2020. p. 2184–2190.

[17] LUO, Y.; LIU, Q.; LIU, Z. STAN: Spatio-temporal attention network for next location recommendation. In: **Proc. Web Conf. (WWW)**. [S.l.: s.n.], 2021. p. 2177–2185.

[18] LIAN, D. et al. Geography-aware sequential location recommendation. In: **Proc. KDD**. [S.l.: s.n.], 2020. p. 2009–2019.

[19] YANG, S.; LIU, J.; ZHAO, K. GETNext: Trajectory flow map enhanced transformer for next POI recommendation. In: **Proc. ACM SIGIR**. [S.l.: s.n.], 2022. p. 1144–1153.

[20] LIN, Y. et al. Pre-training context and time aware location embeddings from spatial-temporal trajectories for user next location prediction. In: **Proc. AAAI Conf. Artif. Intell.** [S.l.: s.n.], 2021. v. 35, p. 4241–4248.

[21] LIM, N. et al. Hierarchical multi-task graph recurrent network for next POI recommendation. In: **Proc. ACM SIGIR**. [S.l.: s.n.], 2022. p. 1133–1143.

[22] YU, F. et al. A category-aware deep model for successive POI recommendation on sparse check-in data. In: **Proc. Web Conf. (WWW)**. [S.l.: s.n.], 2020. p. 1264–1274.

[23] ZHU, N. et al. Predicting a person's next activity region with a dynamic region-relation-aware graph neural network. **ACM Trans. Knowl. Discov. Data**, v. 16, n. 6, p. 1–23, 2022.

[24] CAPANEMA, C. G. S. et al. Combining recurrent and graph neural networks to predict the next place's category. **Ad Hoc Netw.**, v. 138, p. 103016, 2023.

[25] MIKOLOV, T. et al. Efficient estimation of word representations in vector space. In: **Proc. ICLR (Workshop)**. [S.l.: s.n.], 2013.

[26] PEROZZI, B.; AL-RFOU, R.; SKIENA, S. DeepWalk: Online learning of social representations. In: **Proc. KDD**. [S.l.: s.n.], 2014. p. 701–710.

[27] GROVER, A.; LESKOVEC, J. node2vec: Scalable feature learning for networks. In: **Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining**. [S.l.: s.n.], 2016. p. 855–864.

[28] KIPF, T. N.; WELLING, M. Semi-supervised classification with graph convolutional networks. In: **Proc. ICLR**. [S.l.: s.n.], 2017.

[29] VELIČKOVIĆ, P. et al. Graph attention networks. In: **Proc. ICLR**. [S.l.: s.n.], 2018. ArXiv:1710.10903.

[30] HAMILTON, W. L.; YING, R.; LESKOVEC, J. Inductive representation learning on large graphs. In: **Proc. NeurIPS**. [S.l.: s.n.], 2017.

[31] BELGHAZI, M. I. et al. Mutual information neural estimation. In: **Proc. ICML**. [S.l.: s.n.], 2018. p. 531–540.

[32] HJELM, R. D. et al. Learning deep representations by mutual information estimation and maximization. In: **Proc. ICLR**. [S.l.: s.n.], 2019.

[33] VELIČKOVIĆ, P. et al. Deep graph infomax. In: **Proc. ICLR**. [S.l.: s.n.], 2019.

[34] HUANG, W. et al. Learning urban region representations with POIs and hierarchical graph infomax. **ISPRS J. Photogramm. Remote Sens.**, v. 196, p. 134–145, 2023.

[35] KAZEMI, S. M. et al. Time2vec: Learning a vector representation of time. **arXiv preprint arXiv:1907.05321**, 2019.

[36] SITZMANN, V. et al. Implicit neural representations with periodic activation functions. In: **Advances in Neural Information Processing Systems**. [S.l.: s.n.], 2020. v. 33, p. 7462–7473. ArXiv:2006.09661.

[37] MAI, G. et al. **Multi-Scale Representation Learning for Spatial Feature Distributions using Grid Cells**. 2020. Disponível em: <https://arxiv.org/abs/2003.00824>.

[38] RUßWURM, M. et al. Geographic location encoding with spherical harmonics and sinusoidal representation networks. In: **Proc. ICLR**. [s.n.], 2024. ArXiv:2310.06743. Disponível em: <https://openreview.net/forum?id=PudduufFLa>.

[39] PEREZ, E. et al. FiLM: Visual reasoning with a general conditioning layer. In: **Proc. AAAI Conf. Artificial Intelligence**. [S.l.: s.n.], 2018. p. 3942–3951.

[40] RUDER, S. **An Overview of Multi-Task Learning in Deep Neural Networks**. 2017. ArXiv:1706.05098.

[41] MISRA, I. et al. Cross-stitch networks for multi-task learning. In: **Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)**. [S.l.: s.n.], 2016. p. 3994–4003.

[42] MA, J. et al. Modeling task relationships in multi-task learning with multi-gate mixture-of-experts. In: **Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining**. [S.l.: s.n.], 2018. p. 1930–1939.

[43] TANG, H. et al. Progressive layered extraction (PLE): A novel multi-task learning (MTL) model for personalized recommendations. In: **Proc. RecSys**. [S.l.: s.n.], 2020. p. 269–278.

[44] HAZIMEH, H. et al. DSelect-k: Differentiable selection in the mixture of experts with applications to multi-task learning. In: **Proc. NeurIPS**. [S.l.: s.n.], 2021.

[45] STANDLEY, T. et al. Which tasks should be learned together in multi-task learning? In: **Proc. Int. Conf. Machine Learning (ICML)**. [S.l.: s.n.], 2020. ArXiv:1905.07553.

[46] SENER, O.; KOLTUN, V. Multi-task learning as multi-objective optimization. In: **Advances in Neural Information Processing Systems (NeurIPS)**. [S.l.: s.n.], 2018. v. 31, p. 527–538. ArXiv:1810.04650.

[47] KENDALL, A.; GAL, Y.; CIPOLLA, R. Multi-task learning using uncertainty to weigh losses for scene geometry and semantics. In: **Proc. IEEE/CVF CVPR**. [S.l.: s.n.], 2018. p. 7482–7491.

[48] CHEN, Z. et al. GradNorm: Gradient normalization for adaptive loss balancing in deep multitask networks. In: **Proc. ICML**. [S.l.: s.n.], 2018. p. 794–803.

[49] LIU, S.; JOHNS, E.; DAVISON, A. J. End-to-end multi-task learning with attention. In: **Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)**. [S.l.: s.n.], 2019. Introduces Dynamic Weight Averaging (DWA).

[50] YU, T. et al. Gradient surgery for multi-task learning. In: **Proc. NeurIPS**. [S.l.: s.n.], 2020. v. 33.

[51] LIU, B. et al. Conflict-averse gradient descent for multi-task learning. In: **Proc. NeurIPS**. [S.l.: s.n.], 2021.

[52] NAVON, A. et al. Multi-task learning as a bargaining game. In: **Proc. ICML**. [S.l.: s.n.], 2022. p. 16428–16446.

[53] SENUSHKIN, D. et al. Independent component alignment for multi-task learning. In: **Proc. IEEE/CVF CVPR**. [S.l.: s.n.], 2023. p. 20083–20093.

[54] LIU, B. et al. FAMO: Fast adaptive multitask optimization. In: **Proc. NeurIPS**. [S.l.: s.n.], 2023. v. 36.

[55] LIN, B. et al. Reasonable effectiveness of random weighting: A litmus test for multi-task learning. **Transactions on Machine Learning Research**, 2022.

[56] XIN, D. et al. Do current multi-task optimization methods in deep learning even help? In: **Proc. NeurIPS**. [S.l.: s.n.], 2022. v. 35.

[57] KURIN, V. et al. In defense of the unitary scalarization for deep multi-task learning. In: **Proc. NeurIPS**. [S.l.: s.n.], 2022. v. 35.

[58] VANDENHENDE, S. et al. Multi-task learning for dense prediction tasks: A survey. **IEEE Transactions on Pattern Analysis and Machine Intelligence**, v. 44, n. 7, p. 3614–3633, 2022.

[59] YU, J. et al. Unleashing the power of multi-task learning: A comprehensive survey spanning traditional, deep, and pretrained foundation model eras. **arXiv preprint arXiv:2404.18961**, 2024. Disponível em: <https://arxiv.org/abs/2404.18961>.

[60] LIAO, D. et al. Predicting activity and location with multi-task context aware recurrent neural network. In: **Proc. IJCAI**. [S.l.: s.n.], 2018. p. 3435–3441.

[61] HUANG, Z. et al. Human mobility prediction with causal and spatial-constrained multi-task network. **EPJ Data Sci.**, v. 13, n. 1, p. 22, 2024.

[62] WONGSO, W.; XUE, H.; SALIM, F. D. **Massive-STEPS: Massive Semantic Trajectories for Understanding POI Check-ins — Dataset and Benchmarks**. 2025. ArXiv:2505.11239.

[63] YANG, D. et al. Modeling user activity preference by leveraging user spatial temporal characteristics in LBSNs. **IEEE Transactions on Systems, Man, and Cybernetics: Systems**, v. 45, n. 1, p. 129–142, 2015.

[64] SOKOLOVA, M.; LAPALME, G. A systematic analysis of performance measures for classification tasks. **Information Processing & Management**, v. 45, n. 4, p. 427–437, 2009.

[65] GAMBS, S.; KILLIJIAN, M.-O.; CORTEZ, M. N. del P. Next place prediction using mobility markov chains. In: **Proc. MPM (EuroSys Workshop)**. [S.l.: s.n.], 2012. p. 1–6.

[66] KOHAVI, R. A study of cross-validation and bootstrap for accuracy estimation and model selection. In: **Proc. IJCAI**. [S.l.: s.n.], 1995. v. 14, p. 1137–1143.

[67] PEDREGOSA, F. et al. Scikit-learn: Machine learning in Python. **Journal of Machine Learning Research**, v. 12, p. 2825–2830, 2011.

[68] WILCOXON, F. Individual comparisons by ranking methods. **Biometrics Bulletin**, v. 1, n. 6, p. 80–83, 1945.

[69] HOLM, S. A simple sequentially rejective multiple test procedure. **Scand. J. Statist.**, v. 6, n. 2, p. 65–70, 1979.

[70] LAKENS, D. Equivalence tests: A practical primer for t tests, correlations, and meta-analyses. **Soc. Psychol. Personal. Sci.**, v. 8, n. 4, p. 355–362, 2017.

[71] ZHANG, L. et al. An interactive multi-task learning framework for next POI recommendation with uncertain check-ins. In: **Proc. IJCAI**. [S.l.: s.n.], 2020. p. 3551–3557.

[72] LESKOVEC, J.; KREVL, A. **SNAP Datasets: Stanford Large Network Dataset Collection**. 2014. <https://snap.stanford.edu/data/loc-gowalla.html>. Accessed: 2024-05-21.

[73] CHEN, M. et al. Modeling spatial trajectories with attribute representation learning. **IEEE Transactions on Knowledge and Data Engineering**, v. 34, n. 4, p. 1902–1914, 2022.

[74] ZENG, J. et al. A next location predicting approach based on a recurrent neural network and self-attention. In: **Collaborative Computing: Networking, Applications and Worksharing (CollaborateCom)**. [S.l.]: Springer, 2019. p. 309–322.

[75] ZHANG, Y.; YANG, Q. A survey on multi-task learning. **IEEE Transactions on Knowledge and Data Engineering**, v. 34, n. 12, p. 5586–5609, 2022.

[76] RUDER, S. et al. Sluice networks: Learning what to share between loosely related tasks. **arXiv preprint arXiv:1705.08142**, 2017.

[77] HALDER, S. et al. Transformer-based multi-task learning for queuing time aware next poi recommendation. In: **Advances in Knowledge Discovery and Data Mining**. Cham: Springer International Publishing, 2021. p. 510–523.

[78] XIA, B. et al. Mtpr: A multi-task learning based poi recommendation considering temporal check-ins and geographical locations. **Applied Sciences**, v. 10, n. 19, 2020. ISSN 2076-3417. Disponível em: <https://www.mdpi.com/2076-3417/10/19/6664>.

[79] DU, J. et al. Beyond geo-first law: Learning spatial representations via integrated autocorrelations and complementarity. In: IEEE. **2019 IEEE international conference on data mining (ICDM)**. [S.l.], 2019. p. 160–169.

[80] HUANG, W. et al. Estimating urban functional distributions with semantics preserved poi embedding. **International Journal of Geographical Information Science**, Taylor & Francis, v. 36, n. 10, p. 1905–1930, 2022.

[81] BAXTER, J. A model of inductive bias learning. **Journal of Artificial Intelligence Research**, v. 12, p. 149–198, 2000.

[82] VASWANI, A. et al. Attention is all you need. In: **Advances in Neural Information Processing Systems**. [S.l.: s.n.], 2017. p. 5998–6008. ArXiv:1706.03762.

[83] PAIVA, T. S. et al. ST-MTLNet: Representações espaço-temporais de pontos de interesse para aprendizado multitarefa. In: **Anais do X Workshop de Computação Urbana (CoUrb 2026)**. [S.l.]: Sociedade Brasileira de Computação (SBC), 2026. p. 323–336.

[84] FENG, S. et al. POI2Vec: Geographical latent representation for predicting future visitors. In: **Proc. AAAI Conf. Artif. Intell.** [S.l.: s.n.], 2017. v. 31, n. 1, p. 102–108.

[85] RAHMANI, H. A. et al. Category-aware location embedding for point-of-interest recommendation. In: **Proceedings of the 2019 ACM SIGIR international conference on theory of information retrieval**. [S.l.: s.n.], 2019. p. 173–176.

[86] SUN, K. et al. Where to go next: Modeling long-and short-term user preferences for point-of-interest recommendation. In: **Proceedings of the AAAI conference on artificial intelligence**. [S.l.: s.n.], 2020. v. 34, n. 01, p. 214–221.

[87] SUN, Y. Transtarec: Time-adaptive translating embedding model for next poi recommendation. In: IEEE. **2024 5th International Conference on Computer Engineering and Application (ICCEA)**. [S.l.], 2024. p. 647–651.

[88] HALDER, S. et al. Poi recommendation with queuing time and user interest awareness. **Data Mining and Knowledge Discovery**, v. 36, p. 2379–2409, 2022.

[89] MIKOLOV, T. et al. Distributed representations of words and phrases and their compositionality. In: **Advances in Neural Information Processing Systems (NIPS)**. [S.l.: s.n.], 2013.

[90] BELKIN, M.; NIYOGI, P. Laplacian eigenmaps for dimensionality reduction and data representation. **Neural Computation**, v. 15, n. 6, p. 1373–1396, 2003.

[91] BASTUG, E.; BENNIS, M.; DEBBAH, M. Living on the edge: The role of proactive caching in 5G wireless networks. **IEEE Commun. Mag.**, v. 52, n. 8, p. 82–89, 2014.

[92] VIELHAUS, C. L. et al. Handover predictions as an enabler for anticipatory service adaptations in next-generation cellular networks. In: **Proc. ACM MobiWac**. [S.l.]: ACM, 2022. p. 19–27.

[93] MOURA, D. L. L.; AQUINO, A. L. L.; LOUREIRO, A. A. F. On the design of mobility-aware systems: A tourist's perspective. In: **Proc. IEEE/ACM MSWiM**. [S.l.: s.n.], 2025. p. 667–674.

[94] SUN, Z. et al. A multi-channel next POI recommendation framework with multi-granularity check-in signals. **ACM Trans. Inf. Syst.**, v. 42, n. 1, p. 1–28, 2024.

[95] SUN, K.; XU, M. **Knowledge Graph Tokenization for Behavior-Aware Generative Next POI Recommendation**. 2025. ArXiv:2509.12350.

[96] WANG, Y. et al. Hierarchy aware-based multi-task learning for user location prediction. **The Journal of Supercomputing**, v. 81, n. 11, 2025.

[97] YE, J.; ZHU, Z.; CHENG, H. What's your next move: User activity prediction in location-based social networks. In: **Proc. SIAM SDM**. [S.l.: s.n.], 2013. p. 171–179.

[98] LI, X. et al. Beyond individual and point: Next POI recommendation via region-aware dynamic hypergraph with dual-level modeling. In: **Proc. IJCAI**. [S.l.: s.n.], 2025. p. 3081–3089.

[99] SANTOS, G. B. dos. **Urban Region Representation Learning: A Positional and Structural Graph Approach**. Dissertação (Master's Dissertation) — Universidade Federal de Viçosa, Florestal, Minas Gerais, Brazil, 2024. Programa de Pós-Graduação em Ciência da Computação. Advisor: Fabrício Aguiar Silva.

Appendix

APPENDIX A – Other Scientific Contributions

A.1 The experimental platform

Beyond the three studies of this collection, this research produced a reusable software platform that supported the experiments of Chapters 3 and 5 and is released with the code. It is organized as a registry-driven experimental framework rather than a collection of one-off scripts: name-keyed registries expose interchangeable implementations across each axis the dissertation studies. The core comprises a 192-module, 28,644-line source tree, a separate embedding-engine suite, and a test layer of comparable size, developed across roughly 1,700 commits. The platform provides implementations of three dataset families (Gowalla, Massive-STEPS, and Foursquare), eight point-of-interest and check-in embedding engines (including the place embedding of Deep Graph Infomax, its hierarchical variant, and the check-in-level Check2HGI of Chapter 5), twenty-one multi-task loss and gradient-balancing methods, and thirteen multi-task backbone architectures spanning the hard-to-soft parameter-sharing spectrum.

A refined extract-transform-load pipeline for the Gowalla check-in dump is a distinct part of this contribution. It geospatially partitions the public check-in record into fifty-six United States state-level datasets through a three-stage, resumable process: category mapping over a several-hundred-entry hierarchy, optional timezone localization, and a state spatial join against Census boundary data. The platform also holds the evaluation protocol of the final study: user-disjoint cross-validation, a region-transition prior rebuilt for each fold from training data alone, and the paired significance and non-inferiority tests reported in Chapter 5. That protocol is not retroactive. The study of Chapter 4 stratified its folds by sample rather than by user and ran in a separate repository, and neither it nor the study of Chapter 3 reported a significance test, so the measurement procedure strengthened over the arc instead of holding constant across it. What the tooling contributes is the means by which the stricter protocol became practical: the split, the fold-wise prior, and the tests are implemented once, in one place, and not once per study. No performance or novelty claim is made here; those belong to the results chapters, under the evidence protocol the frame follows.

A.2 Reproducing the reported numbers

This section names, for each part of the protocol, the code that implements it and the file its output lives in. Every path is relative to the working repository of this research. The protocol described is Chapter 5's, and Chapter 2 scopes it there rather than to the whole collection. The public snapshot that Chapter 5 footnotes carries the model, the representation, the baselines and the fold construction; the statistical scripts and their output files listed below are part of the working repository and are supplied on request.

The fold partition. Folds are built by <src/data/folds.py>, which calls scikit-learn's StratifiedGroupKFold with five splits, shuffling enabled, and a fixed partition seed of 42, grouping by user identifier and stratifying on the task label. Grouping by user is what makes the split user-disjoint: a user's check-ins fall on one side of every fold. The same routine builds the paired multi-task folds, where both tasks share one user partition so that a user cannot reach one task's training data and the other task's validation data in the same fold.

The seeds. Four random initializations, 0, 1, 7, and 100, each repeating the five-fold experiment over that one fixed partition, which is the twenty fitted models per configuration the chapter reports. The design is recorded in <docs/studies/closing_data/v17_completion/STATISTICAL_PROTOCOL.md>. Only the initialization varies across the four; the folds do not, which is why the reported intervals cover variation across initializations and not across resampled user splits, a limitation Chapter 5 states.

The region-transition prior. Rebuilt for each fold from that fold's training partition alone by <scripts/build_phase3_per_fold_transitions.sh>, which writes one file per fold. Only the external region baseline of Chapter 5 consumes it; the joint and dedicated models do not.

The reported scores. Read at one saved model per fold, the epoch selected on validation, which is the joint-best convention: both tasks are read at the same checkpoint rather than each at its own best epoch. The post-hoc scorer is <scripts/closing_data/score_joint_best.py> and its results are collected in <docs/studies/closing_data/joint_best/JOINT_BEST_RESULTS.md>.

The significance tests. The superiority and non-inferiority tests are implemented in <scripts/closing_data/superiority_wilcoxon.py> and <scripts/closing_data/region_match_tost.py>. The reported analysis was run by <m1_stats_n20.py>, with the pre-registered per-fold variant in <m2_prereg_perfold.py>; both scripts, their console output (<m1_full_output.txt> and <m2_prereg_output.txt>), and the verdict tables that Chapter 5 quotes sit together in <docs/studies/closing_data/v17_completion/stats_n20/>.

The label-history benchmark. Computed by <scripts/embedding_eval/autocorrelation_ceiling.py>; outputs and the reading of them are in Appendix D of the supplementary volume of this dissertation, which names the files.

Two qualifications belong with this list. The metric conventions it presupposes are defined where they are used, macro-F1 and accuracy at ten in Chapter 2 and again in Chapter 5, and this section does not restate them. And the list covers the final study alone. The experiments of Chapters 3 and 4 ran under the weaker protocol described earlier in this appendix, the second of them in a separate repository, so a reader reproducing those chapters is reproducing what they report and not what this section describes.

APPENDIX C – AI-Use Disclosure

This dissertation was written with the assistance of a generative artificial intelligence tool, Claude (Anthropic), in its Opus 4.8, Fable 5, and Opus 5 versions, used as a research and writing assistant under the author's direction. This appendix discloses the scope of that use, as the CNPq integrity policy requires (Portaria nº 2.664/2026) and the UFV/DPE disclosure guidelines of 2026 recommend.

The scope of use, per part of the work, was the following.

Drafting. The frame chapters (Chapters 1, 2, and 6), the appendices, and the front matter were drafted by the assistant from author-approved outlines, source documents, and decision records, section by section. The three article chapters (Chapters 3 to 5) are re-typeset reproductions of the articles' own texts; the assistant performed the re-typesetting and the English translation of the Chapter 4 article, which passed a separate sentence-level fidelity check against the published Portuguese text.

Editing and review. Assistant-run review passes checked citations, numbers, claims, cross-references, and style, each pass run by an agent that did not write the text under review. Their findings were corrections for the author to accept or reject, not approvals.

Formatting. The LaTeX structure, the two build modes, the bibliography consolidation, and figure and table re-setting were produced by the assistant.

Code. Analysis and experiment code was written by the author with assistant support; the assistant also wrote verification scripts used in the review gates.

Three rules governed the drafting. Every reference was checked against its source of record, with the cited claim located in the source. Every number was traced to the file it came from and quoted rather than computed. Every verdict verb was bound to the statistical test that licenses it. Passages that could not be verified were flagged for the author instead of being smoothed over. The author read, corrected, and approved the final text, and is responsible for every word of it.

APPENDIX E – Data Ethics and Governance

Every result in this dissertation is measured on check-ins published by location-based social networks, and neither collection used here was assembled by the author. This appendix states where each came from, under what terms, what the pipeline does with it, and what remains unsettled.

E.1 Where the data came from

The Gowalla check-ins are read from a category-annotated deposit published on Figshare, DOI <`10.6084/m9.figshare.22126586.v2`>, which carries the Creative Commons CC0 public-domain dedication.[1] Four of its files enter the pipeline: the check-in table, two place tables, and the category structure file. The seven place categories used throughout the dissertation are the top-level names in that structure file, so the taxonomy arrives with the deposit; what the pipeline adds is the mapping of each finer category up to its top-level parent, plus a short local supplement for names the structure file leaves uncovered.

Two qualifications belong with that label, and both come from the record itself. The dedication was applied by the depositor of this copy, identified there by a single name, and not by the party that collected the data. The record also names a further site as the origin of the files, and that address now redirects to an unrelated commercial site, so its terms could not be read. What is supportable is therefore a statement about the copy in hand, which is distributed under CC0; that Gowalla data as such is in the public domain is a broader claim, and nothing that could be opened supports it. A second and older Gowalla release, hosted by the Stanford Large Network Dataset Collection and cited here as the collection reference [12, 72], is a different artifact: it carries no place categories, and its page states no license.

The Istanbul check-ins come from the Massive-STEPS collection [62], distributed on Hugging Face under the Apache License 2.0.[2] The project repository carries the same license. Massive-STEPS is derived material, and its documentation names the Semantic Trails Dataset and Foursquare Open Source Places as its sources; among the records that could be opened, no upstream term more restrictive than the Apache License 2.0 appeared. Two facts qualify that. The Foursquare distribution is additionally access-gated behind an agreement prompt, so a reader who goes to that upstream must accept terms before downloading, while the Massive-

[1] <https://creativecommons.org/publicdomain/zero/1.0/>
[2] <https://huggingface.co/datasets/CRUISEResearchGroup/Massive-STEPS-Istanbul>

STEPS copy used here is not gated. And one check is outstanding: the Foursquare product terms themselves were not read, only the license tag on the distribution.

E.2 Real people, and how the traces are handled

A user identifier in these files is a pseudonym and not a name, but a sequence of timestamped visits is still a description of one person's movements. Pseudonymity is not anonymity. The mobility literature treats the residual risk as open, and reports that models trained on mobility data raise privacy questions during training as well as at prediction time, because the portions of information a model absorbs cannot be controlled directly [3]. Public availability settles who may hold the files, not what can be inferred from them.

Against that, this work adds no de-identification of its own. No coordinate is perturbed, rounded, generalized, or masked, and no formal privacy mechanism is applied. Latitude and longitude are used at the precision the sources publish, because the prediction target of Chapter 5 is itself spatial, and a coarsened coordinate would change the quantity being measured.

Exposure is limited by restriction instead. The Figshare deposit also contains a social-graph file and a user-profile file; neither is declared as an input, and no code reads either one, so the friendship network and the profile fields never enter the study. User identifiers are carried as opaque integers exactly as the sources supply them, and a non-numeric identifier is replaced by a position index that is not kept. Nothing in the code links a user across the two collections or to any outside source.

No check-in data is redistributed. The public repository publishes code and configuration, and the data directory is excluded from version control, so a reader reproducing the results obtains both collections from the sources named above.

E.3 The human-subjects question

The studies are a secondary analysis of two collections that were already public before the work began, and no participant was recruited, contacted, or observed by the author. On that basis the author's position is that review by a research ethics committee was not required. This appendix records that position and its basis. It records no approval and no exemption, because none was sought and none is claimed.

A comparable dissertation defended in this program in 2024, on location-based social network data and under the same advisor [99], was consulted on the point. It carries a statement on location privacy and says plainly which fields were left unmasked, and a search of it found no mention of a research ethics committee, an approval number, or a submission for review. That is how a close precedent handled the question, not a determination of the rule. Should the program require a formal determination for secondary analysis of public data, it belongs on file before deposit, and this appendix should then name it.

APPENDIX F – Why the Two Tasks Do Not Compete on the Shared Trunk

A joint model that shares parameters between two tasks can end up worse at both than two dedicated models are at one each. The usual reason is a disagreement, and it shows up in the gradients: if the update next-region asks for points against the update next-category asks for, one task improves at the other's expense. That has a direct measurement, and this appendix runs it on the joint model of Chapter 5.

The two tasks turn out not to disagree. Their gradients are statistically indistinguishable from orthogonal on every dataset measured, which is a stronger and stranger result than mere absence of conflict, and it carries a consequence for the whole investigation: a gradient balancer had nothing to balance. That is why replacing the sharing scheme changed so little in the first study, and why changing the representation changed so much in the second and third. The sections below give the measurement, the two findings that qualify it, and the boundary beyond which none of it is claimed.

F.1 What was measured, and on what unit

At every training epoch, each task loss was back-propagated to the shared cross-attention trunk, and the cosine of the angle between the two resulting gradient vectors was recorded. The cosine is the right quantity because it is scale-free: it reads how the two requested updates are aligned and ignores how large they are, which differs between the tasks and drifts during training. A cosine of $+1$ means the tasks ask for the same update, -1 for opposite ones, and 0 that the requests are orthogonal, each leaving the other's objective unchanged to first order.

The observations are 3,900 epoch-level cosines from four datasets. Florida contributes 3,150 across twelve configurations that vary the loss weight, the weighting schedule, and the training procedure; Alabama, Arizona, and Georgia contribute 250 each at the canonical configuration. Every run is five-fold user-disjoint cross-validation over fifty epochs, so one configuration on one dataset is five series of fifty values, and two of Florida's carry a partial re-run on top of theirs.

Three of the four, Florida, Alabama, and Arizona, are among the six Chapter 5 reports on. The fourth, Georgia, is a further Gowalla state the dissertation does not otherwise use; it enters because the diagnostic ran on it cheaply and stays because dropping a measured dataset would be a choice about the evidence. That chapter reaches the same conclusion from a smaller development-time measurement, on an earlier data preparation and over four seeds rather than per-epoch series, so the two sets of numbers are not interchangeable and this appendix supersedes nothing there.

Those fifty values are not fifty independent measurements. They are consecutive states of one training run, so treating them as independent draws would report more confidence than the data hold. The unit of independence is the fold, and for Florida, whose twelve configurations reuse the same five folds, the configuration. Every test below therefore runs on five fold means for Alabama, Arizona, and Georgia and on twelve configuration means for Florida. Where a count of observations appears it describes the data, not a test's sample size.

Equivalence to zero is assessed by two one-sided tests against a margin of ±0.05 [70]. The margin is a judgment about relevance and should be read as one: at $|\cos| < 0.05$ one task's update projects less than five percent of its length onto the other's direction, below the epoch-to-epoch variation in either gradient and below anything a balancing method could act on. Chapter 5's two-point margin for the region task measures a different quantity on a different scale and should not be read against this one.

F.2 The result

Table 11 – The cosine between the next-region and next-category gradients on the shared trunk, on four datasets. Equivalence to zero holds at every dataset by two one-sided tests (TOST) against a margin of ±0.05, and the TOST column gives the order of magnitude of its p-value. Every test runs on the independent unit named in the second column, never on the individual observations, whose count describes the data and is not any test's sample size; Florida's unit is the configuration because its twelve configurations reuse the same five folds. Alabama and Georgia show all five fold means positive with a t-test that rejects zero, but their exact sign test sits at its floor at five observations, marked †, so each is a consistent tendency and not an established effect.

		Sample		Mean cosine		p against zero	
Dataset	Unit	n	obs.	95% CI	mean	TOST	t / sign
Florida	configuration	12	3,150	[−0.0012, +0.0017]	+0.0003	10^{-16}	0.70 / 0.77
Alabama	fold	5	250	[+0.0040, +0.0184]	+0.0112	10^{-5}	0.013 / 0.063†
Arizona	fold	5	250	[−0.0051, +0.0081]	+0.0015	10^{-5}	0.56 / 1.00
Georgia	fold	5	250	[+0.0016, +0.0061]	+0.0039	10^{-7}	0.009 / 0.063†

† 0.0625 is the smallest value the exact sign test can return at $n = 5$.

Every dataset is equivalent to zero, and Table 11 gives each one's mean, confidence interval and tests. All four means lie within one and a half hundredths of zero, against a margin of five hundredths, and equivalence holds at every other level of aggregation as well, including the raw observations, so the conclusion does not depend on the choice of unit.

The distance between that result and a null result is the point of this appendix. A test that merely failed to reject zero would license one statement, that no conflict was detected, which is equally consistent with a conflict too small to see at this sample size. An equivalence test licenses the positive claim instead: the mean alignment lies inside a margin fixed in advance, so whatever alignment is there is too small to matter. That is a claim about the tasks, not about the experiment's power.

Figure 8 shows the distributions, and one feature needs saying plainly: the equivalence is a statement about the mean, not about every observation. Of the 3,900 cosines, 91.3 percent fall inside the margin, and the full range runs from −0.34 to +0.58, the largest value lying past the right edge of panel (a). Individual epochs drift out of the margin in both directions, ordinary noise in a quantity computed from one mini-batch. What never happens is a systematic pull to one side.

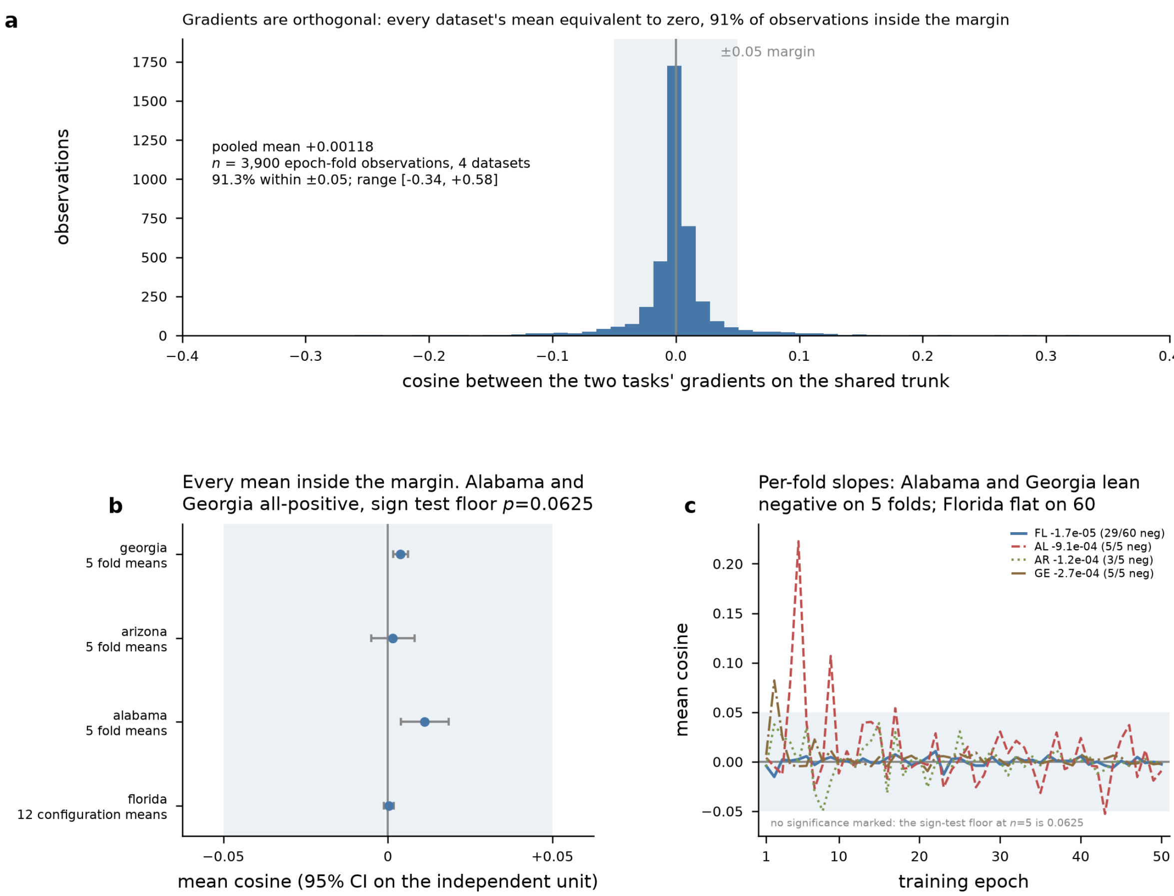

Figure 8 – The two tasks' gradients on the shared trunk are orthogonal, on four datasets. The shaded band is the equivalence margin of ±0.05; every mean falls inside it. Panels: (a) the pooled distribution, (b) each dataset's mean with its 95 percent interval, (c) the mean over training. No significance is marked anywhere in the figure.

Two departures from that flat picture appear on the smaller datasets, and both are worth reporting rather than smoothing. The first is a positive tendency that recurs: all five of Alabama's fold means are positive, averaging +0.0112, and so are all five of Georgia's, averaging +0.0039. The second is a decline over training, on the same two datasets and with the same unanimity, all five per-fold slopes negative in each; Arizona is mixed at three of five, and Florida is flat.

Neither is called significant here, and the reason is the design rather than the data. At five folds the exact sign test cannot return less than 0.0625, so no distribution-free test could reach significance however consistent the pattern. A t-test does reject in all four cases, but at five observations that rests entirely on assuming normality, and this appendix will not accept for one claim a basis it rejects for another. Both are therefore consistent tendencies rather than established effects. Florida, whose sixty independent series are the only estimate in the set with the resolution to settle either question, shows neither.

Both point away from trouble in any case. A positive cosine is mild cooperation, not conflict, and the decline stays inside the margin throughout while moving toward zero rather than away from it. No mechanism is proposed for either.

F.3 What orthogonality explains

Orthogonal gradients mean neither task's update reinforces or cancels the other's on the shared parameters, so the trunk can serve both without either improvement being paid for by the other. Two consequences follow.

The first is about gradient balancers. Nash-MTL and the family around it exist to resolve exactly the disagreement measured here, reweighting or reprojecting task gradients so a dominant task cannot overwrite a weaker one. Orthogonality leaves them nothing to resolve. That is why hard sharing costs nothing in this architecture, and why Chapter 5 finds no balancer improving on a fixed loss weighting: the measurement explains the finding.

The second is about the arc of the three studies. The investigation opened on the hypothesis that the sharing scheme limited joint training, and the first study's null result was read that way at the time. Had the tasks been in conflict, a better sharing scheme or a better balancer would have been the remedy. They were not, so the limit lay elsewhere, and the second and third studies located it in the input representation. There was little interference to fix, and therefore little for a change of architecture to recover.

F.4 How far this extends, and how far it does not

Four datasets and twelve configurations bound this result along two axes, each answering a different objection.

The first axis is the tuning. Every one of Florida's twelve configurations is equivalent to zero on its own five folds, and their twelve means span $[-0.00261, +0.00457]$ over the observations as recorded. Orthogonality survives every hyperparameter and procedure that was varied, so it is not an artifact of one choice among them. The second axis is the data. Among the three datasets Chapter 5 also reports, the check-in counts run from 113,846 at Alabama to

1,407,034 at Florida and the region counts from 1,109 to 4,703, so this axis spans an order of magnitude in volume, and Georgia adds a fourth state under the same protocol. The result is not a quirk of Florida either.

Together the two axes suggest orthogonality belongs to the task pair rather than to the tuning. Next category asks which of seven kinds of place a user visits next, next region asks which tract or neighborhood, and both read the same trace to predict a different projection of one event. Those projections appear close to statistically independent given the shared representation, which would explain why each task's update carries so little of the other's.

That is where the evidence stops, and the boundary matters. Every run measured here uses one architecture family, the cross-attention joint model of Chapter 5. The architecture decides where the two tasks meet and how much they share, which makes it at once the factor most likely to change the answer and the one this appendix did not vary. Nothing here says the gradients stay orthogonal in a model that shares more of its depth, couples the tasks in a cascade, or shares an output layer. Settling that needs the same diagnostic and nothing else changed: the per-epoch cosine on the candidate's shared parameters, under a fixed loss weighting, across one dataset's folds, with the fold means tested for equivalence against a margin chosen in advance. Equivalence there makes orthogonality the task pair's property; otherwise it belongs to this architecture, and Section F.3 applies only to models shaped like this one.

California, Texas, and Istanbul, the other three datasets of Chapter 5, are missing above for reasons of computational resource rather than of principle. California and Texas are the two largest of the six and the region task at that scale forces full single-precision training, and the attempt ran out of disk on the machine that would have done it; Istanbul, a smaller dataset, was queued behind them when the disk filled. The diagnostic needs no new method, so what these three lack is the run itself rather than a way to perform it. The coverage here is therefore three of the dissertation's six datasets plus Georgia, and a reader should not extrapolate to the rest.